D424 – Software Engineering

Task 3



Capstone Proposal Project Name : Master Blaster Hub Web Application

Student Name : James Kelly

Student ID : 011500695

Date : 02/25/2026

Table of Contents

Contents

Table of Contents 2
Master Blaster Hub Live URL 4
GitLab Repository & Branch History 4
Application Design & Testing 5
Class Design 5
Inheritance, Encapsulation and Polymorphism 6
UI Design 7
Login and Signup 8
Dynamic Header 9
MyPortal Page (Default State) 10
Add New Address Panel 11
Delete Address Panel 12
Request Service Panel 13
Check Request Status Panel 14
Admin Page (Default State) 16
Customers Panel 17
Service Jobs Panel 18
Reset Database Panel 19
Export Report Feature 21
Unit Test Plan 22
Introduction and Purpose 22
Overview of Testing Approach 22
Unit Test Scripts and Visual Evidence 23
Test Plan Description 25
Pass and Fail Criteria 26
Procedures 26
Results 27
User Guide – User Perspective 28
Customer Guide 28

Accessing the Application 28
Creating a New Account 29
Logging In 29
Overview of MyPortal 30
Adding a New Address 31
Deleting an Address 32
Requesting a Service 33
Checking Request Status 34
Technology Stack & Deployment Page 35
Logging Out 36
Additional Notes for Users 37
Administrator Guide 37
Accessing the System 37
Admin Dashboard Overview 38
Customers Panel 39
Service Jobs Panel 40
Report Export Feature 41
Database Reset Panel 42
Navigation and Logging Out 43
User Guide - Initial Setup & Running the Application 44
System Architecture Overview 44
Frontend Layer 44
Backend Layer 44
Database Layer 45
Continuous Integration and Deployment (CI/CD) 45
Infrastructure Provisioning 45
High-Level Traffic Flow 45
Required Accounts and Software Prerequisites 46
1. Domain Registration and DNS Delegation 46
1.a Register Domain Through Squarespace 46

1.b Create Public Hosted Zone 47
1.c Update Nameservers in Squarespace 48
1.d Verify Delegation 49
2. Required Cloud Service Accounts 50
2.a Amazon Web Services (AWS) Account 50
2.b AWS Identy and Access Management (IAM) 51
2.c Amazon S3 – Frontend Hosting 54
2.d AWS Certificate Manager (ACM) 57
2.e Amazon Cloudfront – Frontend Distribution 59
2.f Amazon Elastic Container Registry (ECR) 62
2.g Deploying the Backend with Amazon ECS (Fargate) 64
2.h MongoDB Atlas Account and Cluster Setup 73
2.i GitHub Repository, Secrets, and Workflows 80
2.j Local Development Environment Setup 86
2.k Summary of Access Permissions 90
APPENDIX A – Git Branch History 108
APPENDIX B – deploy-frontend.yml 109
APPENDIX C – deploy-backend.yml 110

Master Blaster Hub Live URL

https://www.masterblasterhub.com/

GitLab Repository & Branch History

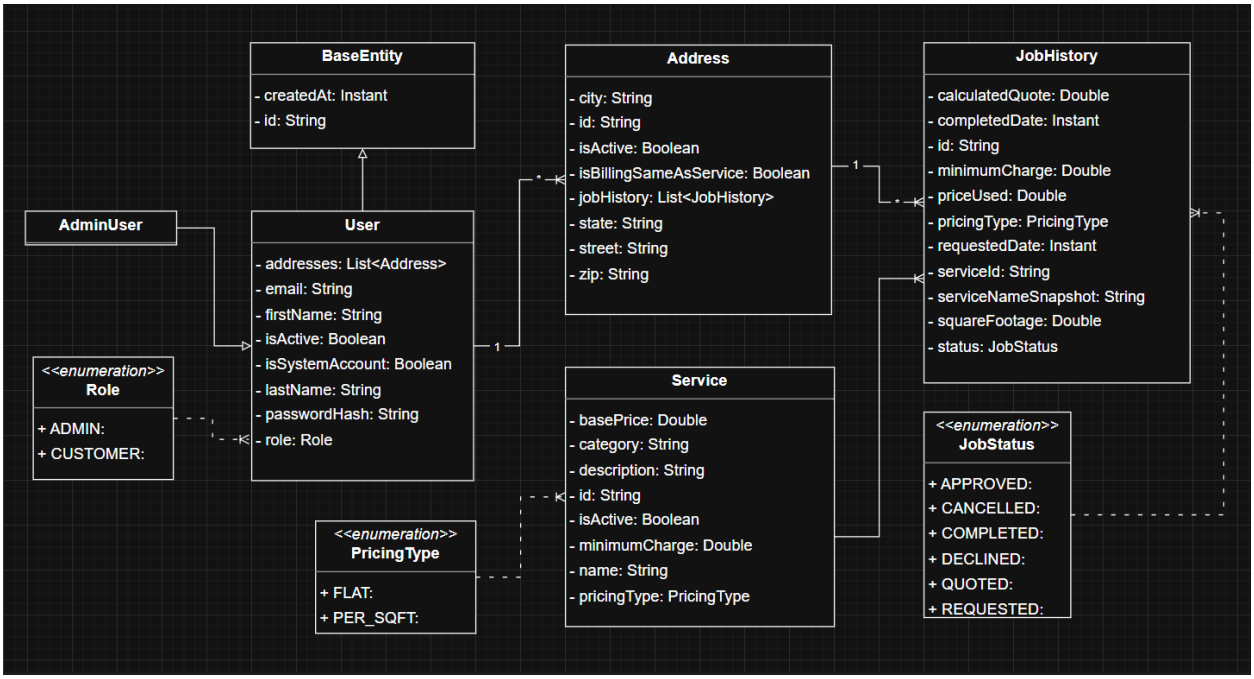
https://gitlab.com/wgu-gitlab-environment/student-repos/jkel829/d424-software-engineering-capstone/-/tree/working_branch?ref_type=heads

See Appendix A - branchHistory_SWECapstone_JamesKelly.pdf

Application Design & Testing

Class Design

The following class diagram represents the core structure of the Master Blaster Hub application and illustrates how the primary domain models relate to one another. The diagram provides a visual overview of the backend architecture and demonstrates how data is organized and managed throughout the system. Designing the class structure before and during development ensured a clear separation of responsibilities, consistent data modeling, and long-term scalability.



At the center of the design is the **User** class. This entity stores essential account information such as **firstName**, **lastName**, **email**, **passwordHash**, **isActive**, and **role**. Each **User** can have multiple **Address** objects, which represent service locations associated with that account. This relationship reflects real-world business logic, where one customer may manage multiple properties requiring separate service visits.

Each **Address** contains a collection of **JobHistory** records. This embedded structure was intentionally chosen because the application uses MongoDB, a document-based database. Instead of distributing related data across multiple relational tables, embedding **JobHistory** within an **Address** allows related information to be stored together within a single document. Since job records are always accessed in the context of a specific **User** and **Address**, embedding improves data locality and simplifies query logic without requiring relational joins.

The **Service** class defines the services offered by the business, including pricing rules such as **basePrice**, **minimumCharge**, and **pricingType**. Each **JobHistory** record stores both a reference to a **Service** and a calculated pricing snapshot at the time the request was created. This design ensures that historical records remain accurate even if service pricing changes in the future. By preserving calculated pricing values alongside the service reference, the system maintains financial consistency and audit reliability.

Enumerations such as **Role**, **PricingType**, and **JobStatus** restrict values to predefined options. For example, a user's role may only be **ADMIN** or **CUSTOMER**, and a job status must fall within defined lifecycle stages such as **REQUESTED**, **APPROVED**, or **COMPLETED**. Using enumerations strengthens type safety, prevents invalid input, and enforces consistent business rules throughout the application.

Inheritance, Encapsulation and Polymorphism

The class design intentionally demonstrates the three core object-oriented principles: inheritance, encapsulation, and polymorphism.

Inheritance is implemented through the **BaseEntity** class, which provides shared attributes such as **id** and **createdAt** to other domain models. This reduces redundancy and ensures structural consistency across entities. Additionally, the **AdminUser** class extends the **User** class, inheriting standard user properties while representing elevated administrative privileges. This hierarchical structure promotes code reuse and prevents duplication of shared fields and logic.

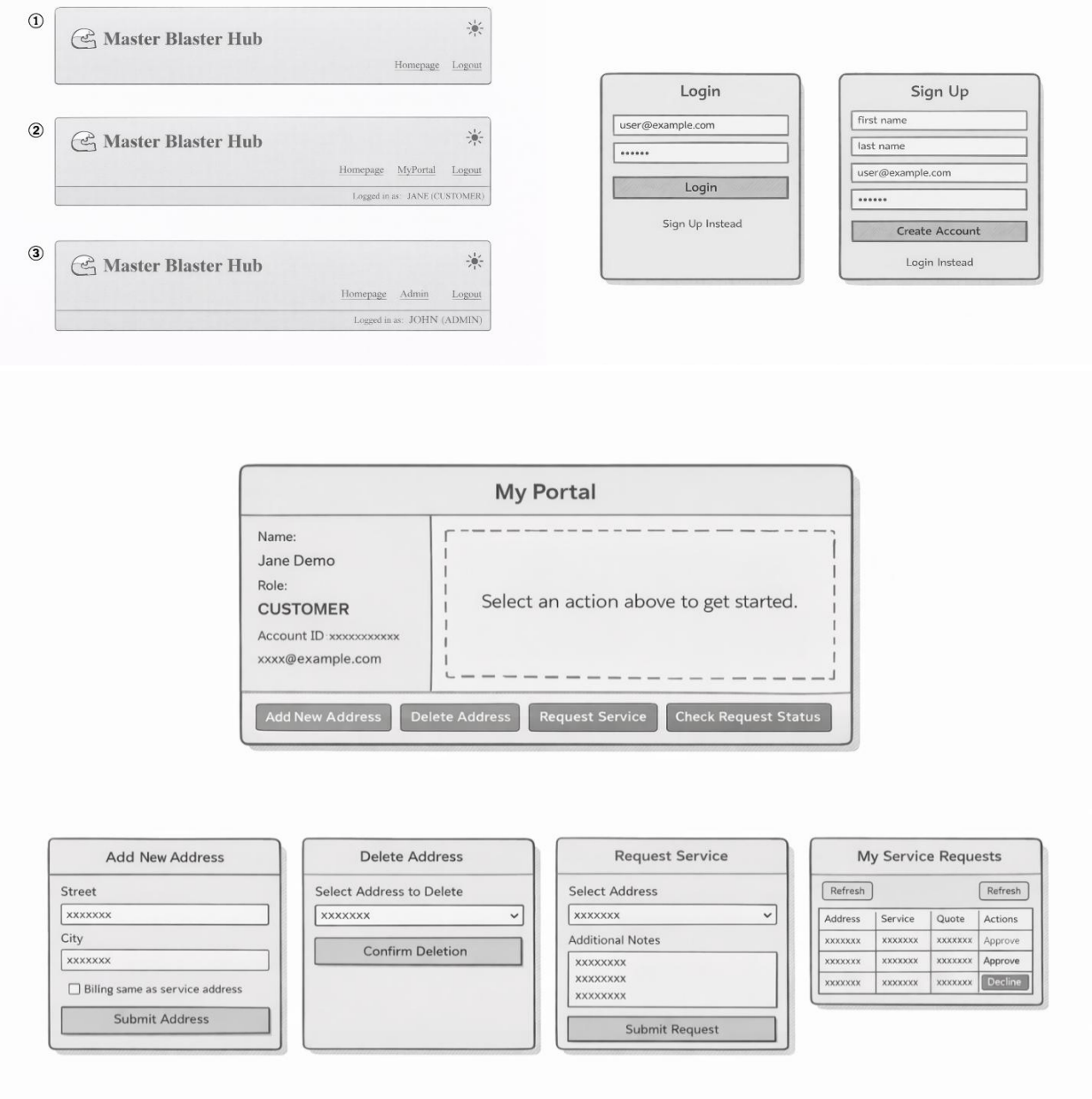
Encapsulation is enforced by declaring model fields as private and exposing them only through controlled getters, setters, and service-layer logic. Sensitive attributes, such as role assignments and password hashes, cannot be modified directly without passing through defined methods. This approach protects internal state, preserves data integrity, and ensures that changes occur in a predictable and controlled manner.

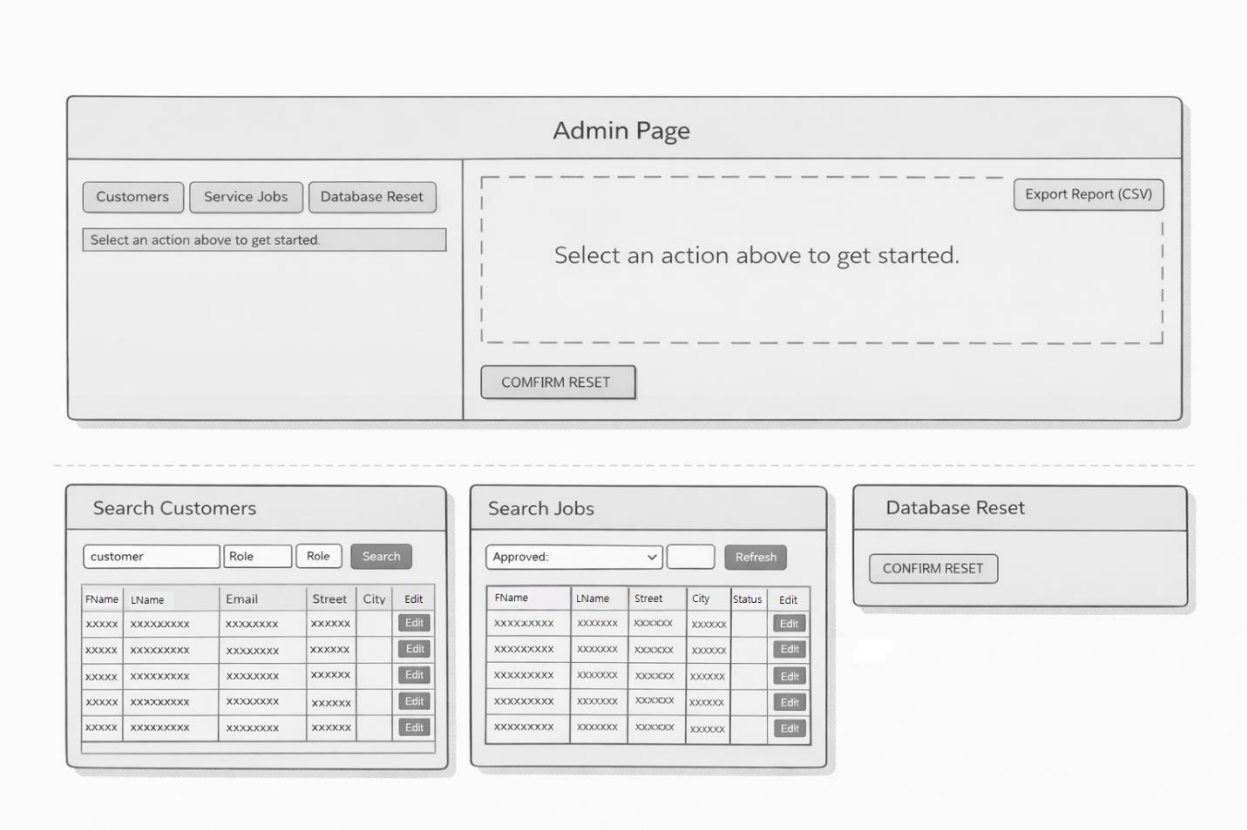
Polymorphism is reflected in how the system processes **User** objects based on their assigned **Role**. Although both administrators and customers are represented by the same base **User** type, the application evaluates role values at runtime to determine allowable actions. For example, permission methods such as **canDeleteUser()** and **canViewReports()** return different outcomes depending on whether the role is **ADMIN** or **CUSTOMER**. This allows shared object structures to exhibit distinct behavior while maintaining a unified type system.

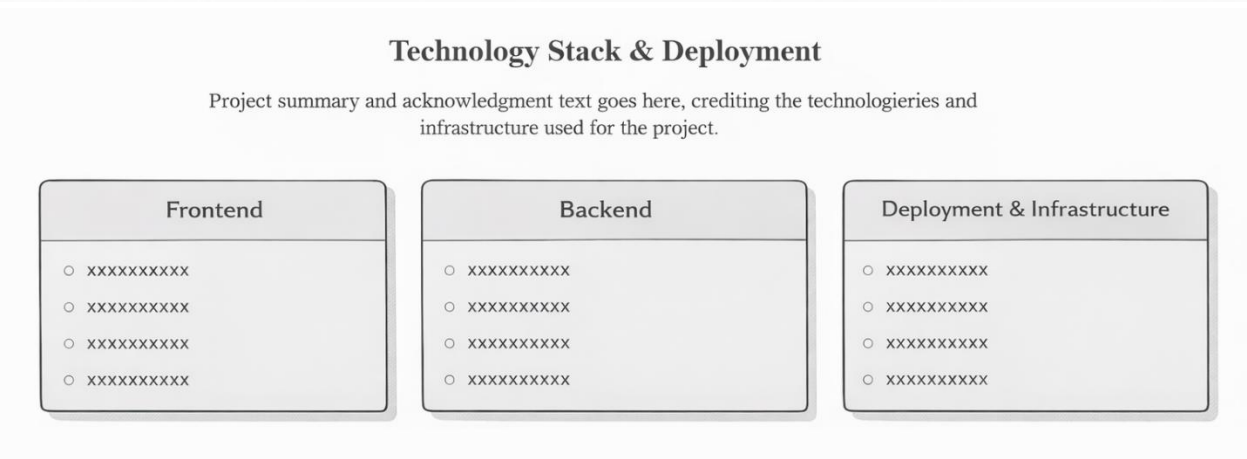
Together, these principles create a maintainable and scalable architecture that supports secure business rule enforcement and long-term system evolution.

UI Design

The user interface of Master Blaster Hub was developed using structured wireframes to define layout, interaction patterns, and role-based rendering behavior. These wireframes guided the implementation of a consistent and intuitive interface across both customer and administrator views. The design prioritizes clarity, usability, and security while minimizing unnecessary navigation. By maintaining consistent layout structures and dynamically rendering content based on role permissions, the interface supports efficient workflows while reinforcing backend access control policies.







Login and Signup

The **Login** and **Signup** screens were designed with clarity and simplicity in mind. Both forms are centered within a clean layout and contain clearly labeled input fields along with a single primary action button. This structure reduces cognitive load and directs the user's attention toward completing authentication efficiently.

Maintaining a consistent visual structure between the **Login** and **Signup** forms reinforces usability and allows users to switch between them without confusion. The

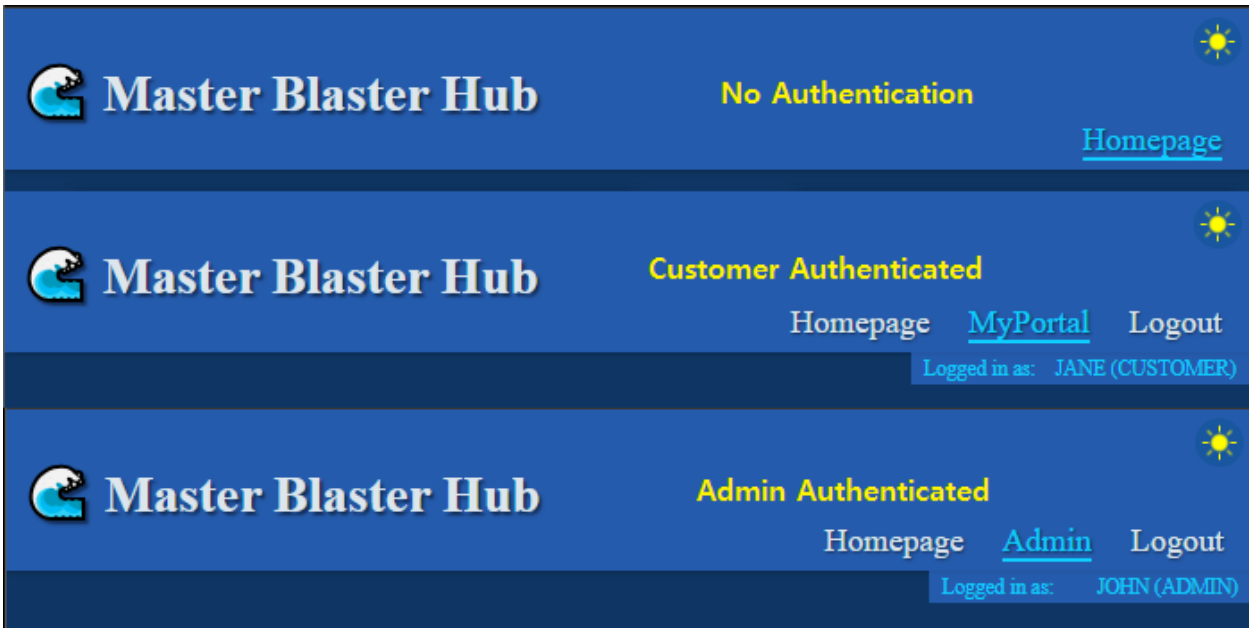
minimal layout supports accessibility and ensures that account creation and authentication remain straightforward processes.



Dynamic Header

The **Dynamic Header** plays a critical role in enforcing role-based access control at the interface level. When no **User** is logged in, only general navigation options are displayed. After authentication, the header updates automatically based on the user's assigned **Role**. Users with the **CUSTOMER** role gain access to **"MyPortal"**, while users with the **ADMIN** role gain access to **"Admin."** A **"Logout"** option appears for authenticated users, and the header displays the currently logged-in account and role.

This conditional rendering is implemented in the frontend so users only see navigation options relevant to their permissions. While backend security ultimately enforces access control, restricting visible navigation elements improves usability and reduces unnecessary or unauthorized access attempts.



MyPortal Page (Default State)

The **MyPortal** page is structured around a dynamic content panel that updates based on the customer's selected action. In its default state, the page displays account information on the left side, including name, role, account ID, and email address. Below this information are four primary action buttons: **"Add New Address," "Delete Address," "Request Service,"** and **"Check Request Status."**

The central panel initially displays a placeholder message prompting the user to select an action. This design allows multiple tasks to be completed within a single dashboard rather than navigating to separate pages. By maintaining a consistent layout and updating only the content panel, the system improves clarity and eliminates unnecessary page transitions.



Add New Address Panel

When the **“Add New Address”** button is selected, the dynamic panel updates to display a form for entering a new service location. The form includes fields for street, city, state, and zip code, along with an optional checkbox indicating whether the billing address is the same as the service address. A single submission button completes the action.

Rendering this form within the same dashboard allows customers to expand their account without leaving **MyPortal**. This approach preserves workflow continuity and maintains a consistent visual structure.



Delete Address Panel

Selecting the **“Delete Address”** button updates the dynamic panel to display a dropdown menu listing the customer’s existing service locations. The user selects an address and confirms deletion to complete the action.

Requiring deliberate selection from a controlled list reduces the likelihood of accidental deletions. Keeping address management within the same dashboard centralizes account maintenance and simplifies user interaction.

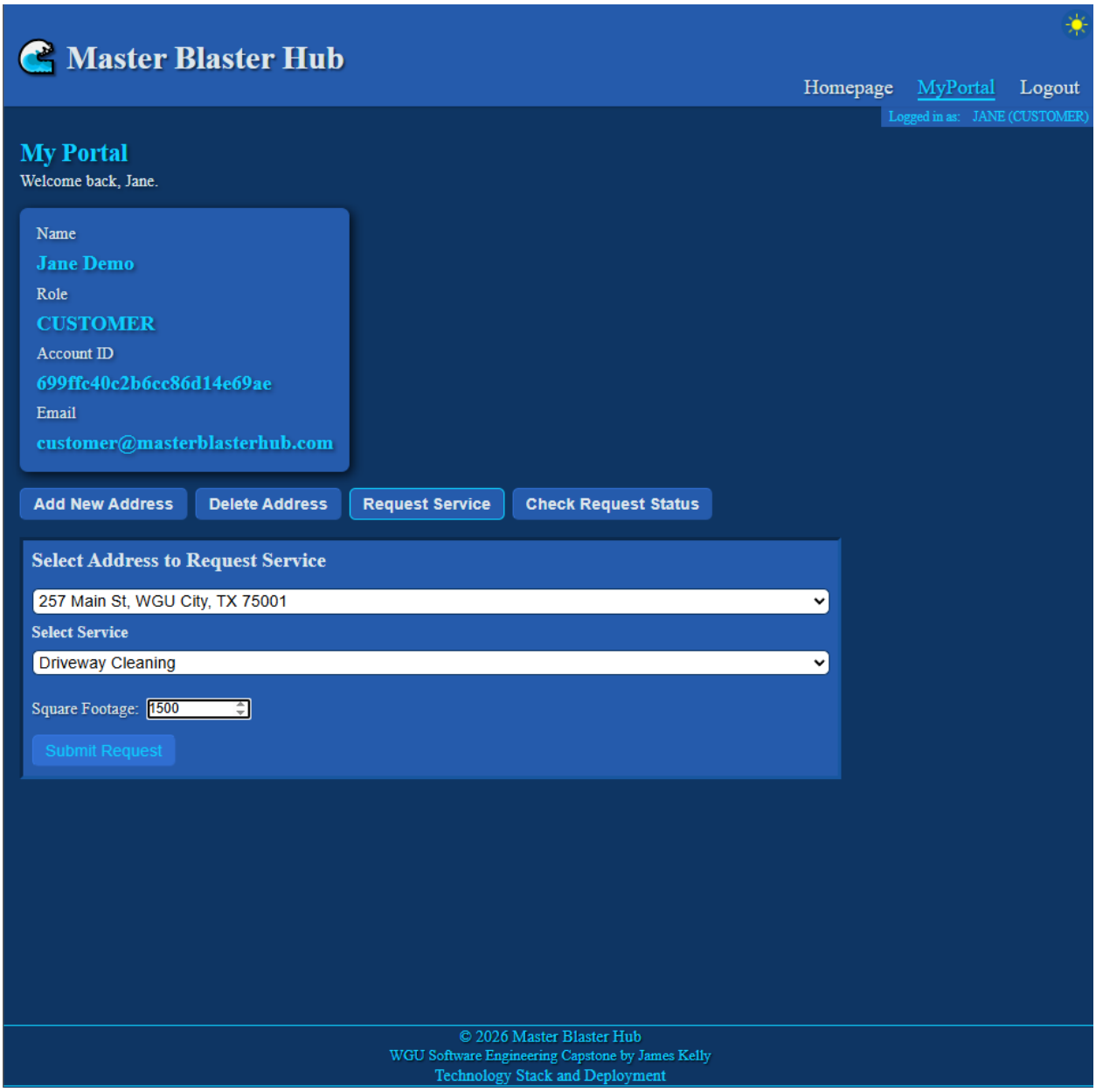


Request Service Panel

When **“Request Service”** is selected, the dynamic panel transforms into a service request form. The interface presents structured input controls that allow selection of a saved address, service type, and required service details such as square footage when applicable. A submission button sends the request for processing.

This panel guides the user through the required inputs in a structured manner, ensuring that all necessary information is provided before submission. By rendering the form

dynamically within **MyPortal**, the system maintains a streamlined and efficient service request workflow.



Check Request Status Panel

The **“Check Request Status”** button updates the panel to display a structured table of the customer’s service requests. The table includes columns for address, service type, quoted price, job status, and available actions. Status values such as **REQUESTED**, **QUOTED**, **APPROVED**, **COMPLETED**, **CANCELLED**, and **DECLINED** are visually distinct to improve readability.

This panel serves as the activity center of **MyPortal**, allowing customers to monitor progress and respond to quotes when necessary. By consolidating service lifecycle management within a single interface, the system enhances usability and maintains consistency.



Together, these dynamically rendered panels demonstrate a deliberate user interface design strategy. By consolidating related actions into one structured dashboard, the application minimizes navigation complexity, improves responsiveness, and maintains a consistent layout. This design enhances usability while supporting efficient interaction with backend services.

Admin Page (Default State)

The **Admin Page** follows the same dynamic panel structure as **MyPortal**, reinforcing consistency across user roles. In its default state, the page displays three primary action buttons: **“Customers,” “Service Jobs,”** and **“Database Reset.”** Beneath these controls, the central panel displays a placeholder message prompting the administrator to select an action.

Using a single dynamic content area keeps the interface clean and focused. Administrators can switch between management functions without reloading the page or navigating to separate routes, maintaining an efficient and organized workflow.

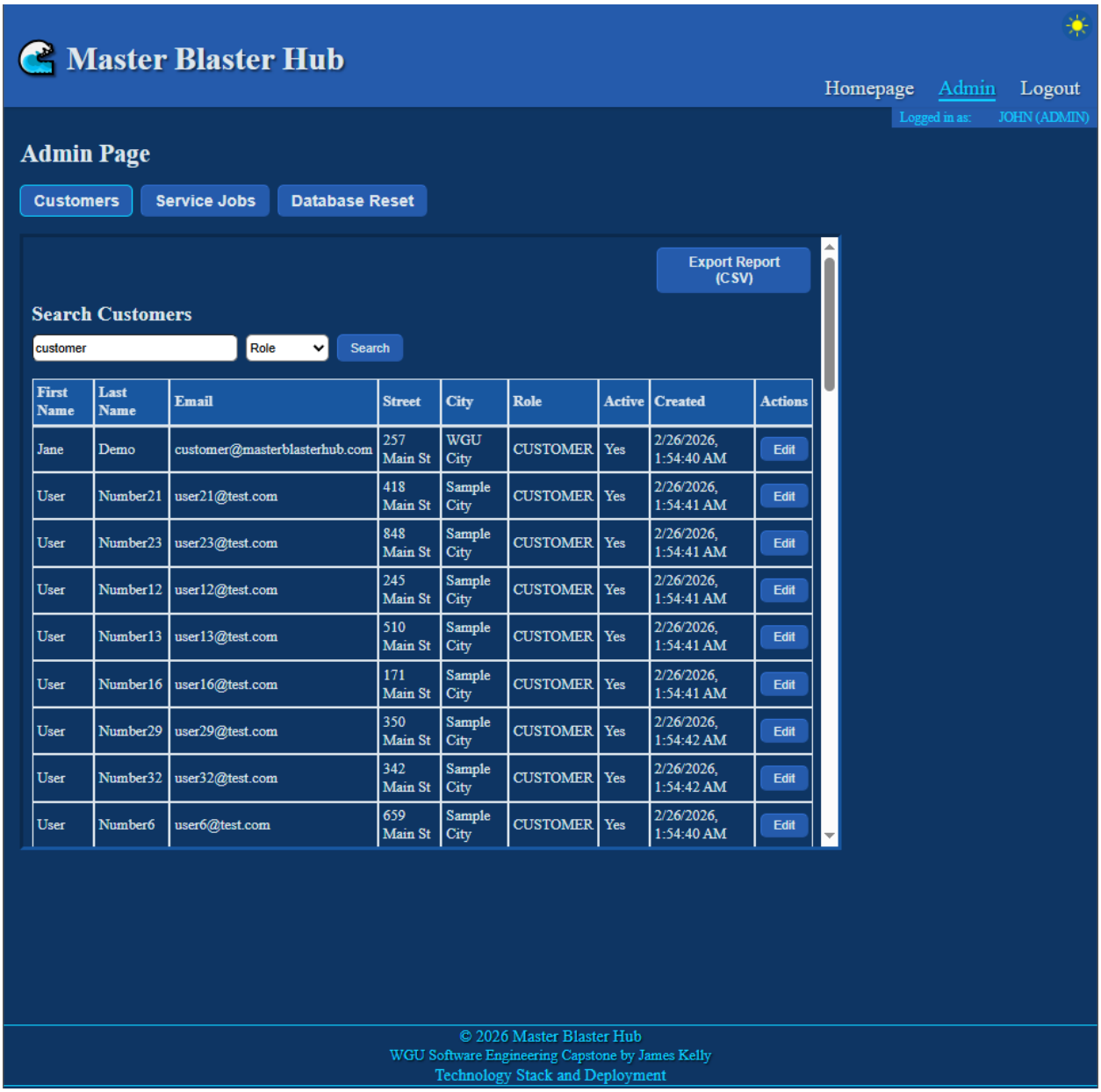


Customers Panel

When the **“Customers”** button is selected, the dynamic panel renders a searchable customer management interface. The layout includes filter controls for **email**, **last name**, and **role**, along with a **“Search”** button that dynamically retrieves matching records based on selected criteria.

Results appear in a structured table containing user details such as first name, last name, email, street address, city, **Role**, account status (active or inactive), and account creation date. Each row includes an **“Edit”** option, allowing administrators to review or modify individual accounts.

Providing multiple search filters improves efficiency when managing larger datasets. This panel centralizes user oversight while maintaining a structured and controlled administrative interface.



Service Jobs Panel

Selecting the **“Service Jobs”** button loads a searchable job management interface within the dynamic panel. Administrators can filter service requests by **email**, **city**, **service type**, or **job status**. The search controls dynamically retrieve and display matching records based on selected filter criteria. The **“Refresh”** button resets or reloads the current results.

The results table includes customer name, contact email, service address, selected service, quoted price, current job status, and available administrative actions. Status values such as **REQUESTED**, **QUOTED**, **APPROVED**, and **COMPLETED** are visually

highlighted to improve clarity. Depending on the job's state, action buttons such as **"Quote," "Complete,"** or **"Cancel"** may appear.

This panel functions as the operational control center of the system, allowing administrators to manage the full service lifecycle from a single, consistent interface.



Reset Database Panel

Selecting the **"Database Reset"** button loads the reset confirmation panel. This feature restores the application database to its original default state. It is intended strictly for emergency or maintenance scenarios, such as resolving data corruption, restoring data integrity, or returning the system to a known baseline configuration.

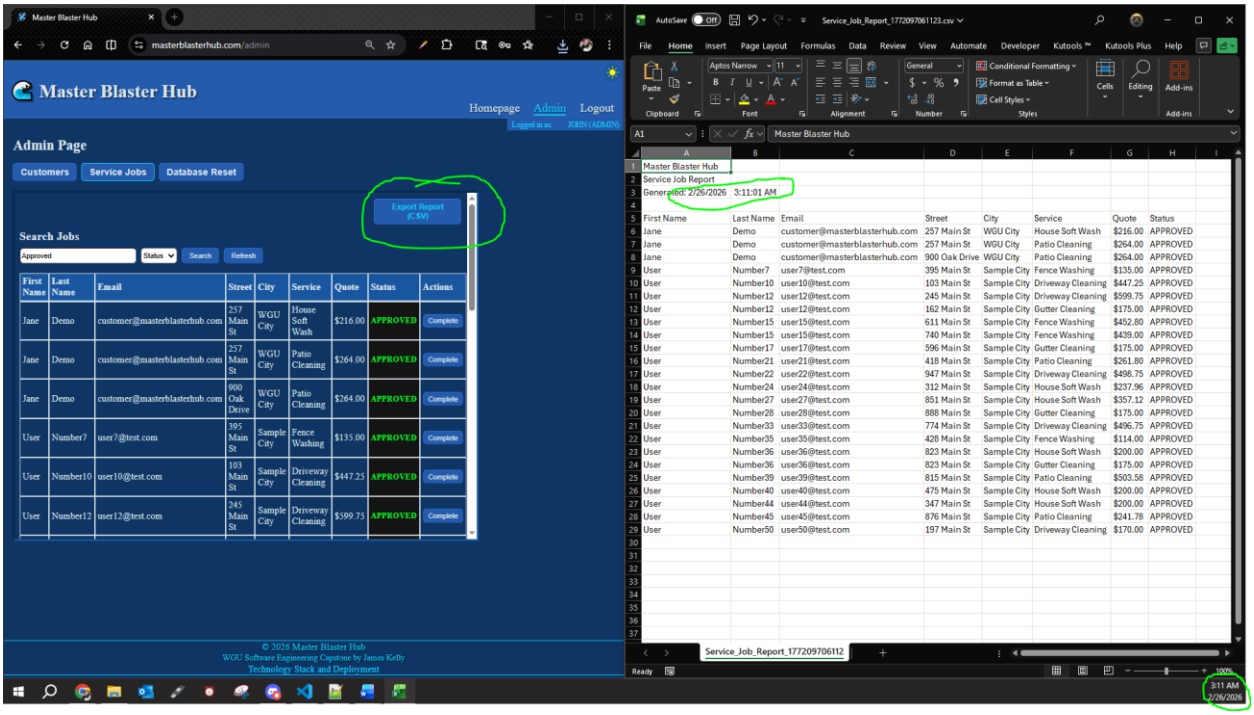
Because this action permanently removes existing data and replaces it with a predefined default dataset, it is irreversible and should only be used as a last resort. To prevent accidental execution, the reset process requires multiple confirmations. The interface incorporates a multi-step confirmation sequence, including an additional confirmation action and a browser alert, before execution is permitted. This layered confirmation process reinforces the seriousness of the action and minimizes the risk of unintended data loss.



Export Report Feature

The **Export Report** feature is integrated directly within the dynamic panel and remains visible when viewing customer or service job data. Selecting **“Export Report (CSV)”** generates a downloadable file containing the currently displayed dataset. Because exports are based on active filters, administrators can produce targeted reports by refining search criteria before generating the file.

The exported CSV includes structured columns such as customer information, service details, pricing data, and job status, along with a timestamp indicating when the report was created. This functionality supports auditing, record keeping, and external analysis without requiring a separate reporting interface.



Together, these dynamically rendered panels demonstrate a structured and efficient administrative interface. By centralizing system management functions within a consistent dashboard layout, the design reduces complexity while supporting powerful oversight capabilities. The combination of searchable tables, controlled system actions, and integrated reporting ensures that administrators can manage users and services efficiently while maintaining system integrity.

Unit Test Plan

Introduction and Purpose

The purpose of this unit test plan is to validate the correctness and reliability of the role-based permission system implemented in the Master Blaster Hub backend. Testing focused on ensuring that authorization rules governing administrative actions are enforced accurately. Specifically, the tests confirm that only properly authorized users may delete accounts or access reporting functionality. By evaluating both permitted and restricted scenarios, this process verifies that protected system accounts remain secure and that role-based access control operates as designed.

Overview of Testing Approach

Unit testing was conducted using the JUnit 5 framework within the IntelliJ IDEA development environment. The component selected for validation was the permission service layer, which controls authorization decisions related to user deletion and reporting access.

Five unit test methods were implemented within the **PermissionServiceTest** class. Three tests evaluate deletion permissions through the **canDeleteUser** method, ensuring that:

- ✓ A **CUSTOMER** cannot delete any account
- ✓ An **ADMIN** can delete a non-protected account
- ✓ A protected system account cannot be deleted

Two additional tests evaluate reporting access through the **canViewReports** method, confirming that only users assigned the **ADMIN** role may access reporting features.

Each test isolates a single business rule and executes independently without database dependencies. This approach ensures that authorization logic is evaluated strictly at the service layer before requests reach controllers or endpoints.

PermissionServiceTest raw code file:

```java
package com.mbh.backend.services;

import com.mbh.backend.models.Role;
import com.mbh.backend.models.User;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class PermissionServiceTest {

    // Verifies that a customer user cannot delete any user account
    @Test
    void customerCannotDeleteUser() {
        CustomerPermissionService service = new CustomerPermissionService();
        User current = new User();
        current.setRole(Role.CUSTOMER);
        User target = new User();
        assertFalse(service.canDeleteUser(current, target));
    }

    // Verifies that an admin can delete a regular (non-system) user
    @Test
    void adminCanDeleteNonSystemUser() {
        AdminPermissionService service = new AdminPermissionService();
        User current = new User();
        current.setRole(Role.ADMIN);
        User target = new User();
        target.setIsSystemAccount(systemAccount: false);
        assertTrue(service.canDeleteUser(current, target));
    }

    // Verifies that even an admin cannot delete the protected system account
    @Test
    void adminCannotDeleteSystemAccount() {
        AdminPermissionService service = new AdminPermissionService();
        User current = new User();
        current.setRole(Role.ADMIN);
        User target = new User();
        target.setIsSystemAccount(systemAccount: true);
        assertFalse(service.canDeleteUser(current, target));
    }

    // Verifies that a customer does not have access to admin reports
    @Test
    void customerCannotViewReports() {
        CustomerPermissionService service = new CustomerPermissionService();
        User current = new User();
        current.setRole(Role.CUSTOMER);
        assertFalse(service.canViewReports(current));
    }

    // Verifies that an admin has permission to view reports
    @Test
    void adminCanViewReports() {
        AdminPermissionService service = new AdminPermissionService();
        User current = new User();
        current.setRole(Role.ADMIN);
        assertTrue(service.canViewReports(current));
    }
}
```

Unit Test Scripts and Visual Evidence

The **PermissionServiceTest** class contains five distinct unit test methods, each validating a specific authorization rule.

Test Case 1: Customer Cannot Delete User

This test verifies that a **User** assigned the **CUSTOMER** role does not have permission to delete another account. A customer instance is created and evaluated against a target **User** object. The expected result is **false**, confirming that standard users cannot perform administrative deletion actions.

```
// Verifies that a customer user cannot delete any user account
@Test
void customerCannotDeleteUser() {
    CustomerPermissionService service = new CustomerPermissionService();
    User current = new User();
    current.setRole(Role.CUSTOMER);
    User target = new User();
    assertFalse(service.canDeleteUser(current, target));
}
```

Test Case 2: Admin Can Delete Non-System User

This test validates that a **User** assigned the **ADMIN** role is permitted to delete a regular account that is not designated as a protected system account. The target user's **isSystemAccount** property is explicitly set to **false** to simulate a standard account. The expected result is **true**, confirming appropriate administrative authority.

```
// Verifies that an admin can delete a regular (non-system) user
@Test
void adminCanDeleteNonSystemUser() {
    AdminPermissionService service = new AdminPermissionService();
    User current = new User();
    current.setRole(Role.ADMIN);
    User target = new User();
    target.setIsSystemAccount(systemAccount: false);
    assertTrue(service.canDeleteUser(current, target));
}
```

Test Case 3: Admin Cannot Delete Protected System Account

This test ensures that even an administrator cannot delete a protected system account. The target user's **isSystemAccount** property is set to **true**, representing a safeguarded account. The expected result is **false**, protecting critical system-level accounts from removal.

```java
// Verifies that even an admin cannot delete the protected system account
@Test
void adminCannotDeleteSystemAccount() {
    AdminPermissionService service = new AdminPermissionService();
    User current = new User();
    current.setRole(Role.ADMIN);
    User target = new User();
    target.setIsSystemAccount(systemAccount: true);
    assertFalse(service.canDeleteUser(current, target));
}
```

Test Case 4: Customer Cannot View Reports

This test confirms that a **User** assigned the **CUSTOMER** role is denied access to administrative reporting functionality. The expected result is **false**, reinforcing that reporting features are restricted to authorized roles.

```java
// Verifies that a customer does not have access to admin reports
@Test
void customerCannotViewReports() {
    CustomerPermissionService service = new CustomerPermissionService();
    User current = new User();
    current.setRole(Role.CUSTOMER);
    assertFalse(service.canViewReports(current));
}
```

Test Case 5: Admin Can View Reports

This final test verifies that a **User** assigned the **ADMIN** role is granted access to reporting functionality. The expected result is **true**, demonstrating that elevated privileges are correctly recognized and enforced.

```java
// Verifies that an admin has permission to view reports
@Test
void adminCanViewReports() {
    AdminPermissionService service = new AdminPermissionService();
    User current = new User();
    current.setRole(Role.ADMIN);
    assertTrue(service.canViewReports(current));
}
```

Test Plan Description

The unit testing process was performed within a controlled development environment using **Java 21**, **Spring Boot**, and **Maven** for dependency management. The **JUnit 5**

framework was used to define and execute test cases, and IntelliJ IDEA's integrated test runner provided real-time execution feedback. Prior to execution, the project was successfully compiled to verify configuration integrity and eliminate build errors.

The test plan focused specifically on validating permission enforcement within the service layer. Five scenarios were selected to represent realistic and security-sensitive use cases involving role-based decision logic. These scenarios ensured that administrative privileges are granted only when appropriate and denied when required by defined business rules.

The primary deliverable of this testing effort was the **PermissionServiceTest** class containing five clearly defined unit test methods. Supporting artifacts included execution output from the IntelliJ test runner and visual documentation in the form of screenshots. This structured testing approach ensured repeatability, traceability, and alignment with established quality assurance practices.

Pass and Fail Criteria

Each test case was evaluated using **assertTrue** and **assertFalse** assertions within the **JUnit 5** framework. A test was considered successful only if the returned boolean value matched the expected result and no runtime exceptions occurred.

The IntelliJ test runner displayed a green indicator for successful tests and would have displayed a red indicator with diagnostic details if a failure occurred.

A failure would result from:

- ✓ Returning an incorrect boolean value
- ✓ Violating a defined business rule
- ✓ Throwing an unexpected exception

If a failure had occurred, remediation would involve reviewing the permission service logic, correcting conditional evaluation related to **Role** or **isSystemAccount**, and re-running the full test suite to confirm resolution. Any defect would be documented prior to correction to maintain accountability.

Procedures

Testing began by implementing a dedicated class named **PermissionServiceTest** within the **services** package to isolate permission logic from higher-level application components.

Five test methods were created using the **@Test** annotation. For each scenario, new **User** objects were instantiated and assigned either the **ADMIN** or **CUSTOMER** role. In

deletion-related tests, the target user's **isSystemAccount** property was configured to simulate protected and non-protected accounts.

After configuration, either **canDeleteUser** or **canViewReports** was invoked, and the result was validated using **assertTrue** or **assertFalse**.

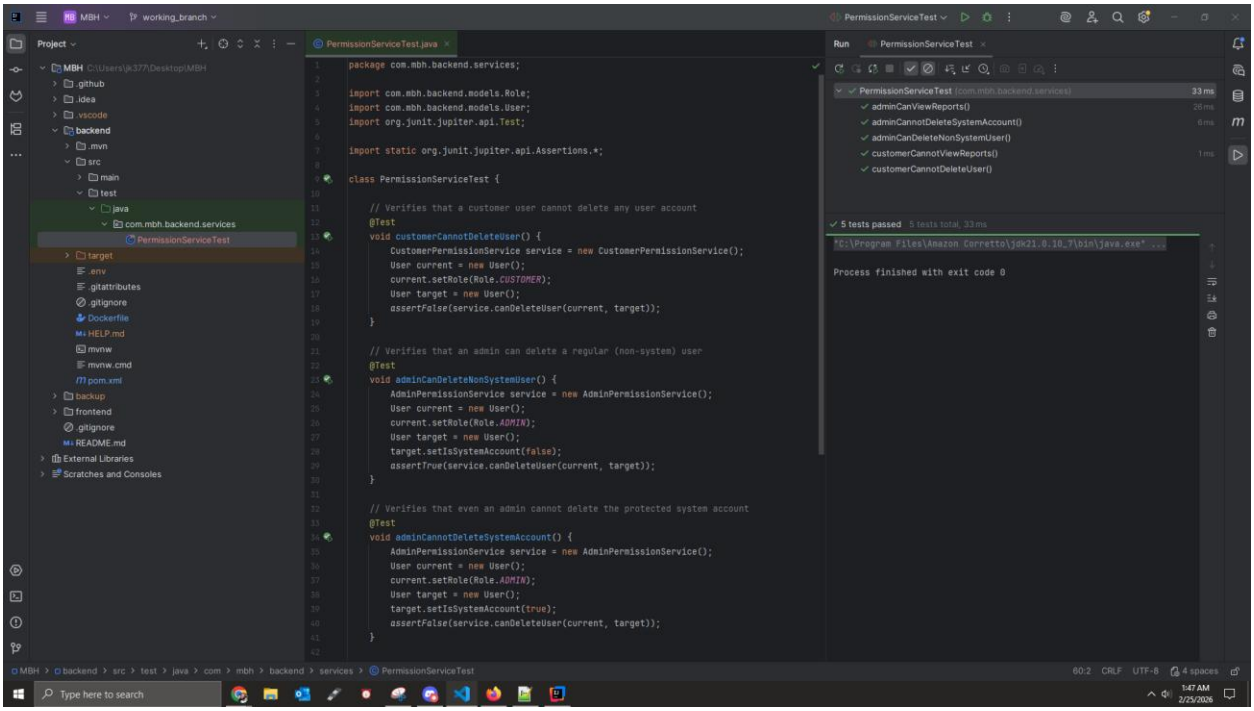
The full test suite was executed using IntelliJ IDEA's integrated runner. All tests compiled and executed independently. Because each scenario produced the expected result, no corrective iterations were required.

Results

The unit tests were executed using IntelliJ IDEA's integrated **JUnit 5** test runner. All five test cases passed successfully with zero failures and zero errors. The execution process concluded with an exit code of 0, confirming error-free completion.

The results verify that the permission service correctly evaluates **Role** assignments and **isSystemAccount** status when determining authorization. Each defined scenario produced the expected outcome, demonstrating consistent enforcement of business rules at the service layer.

The included IntelliJ execution panel screenshot provides visual confirmation of successful test completion.

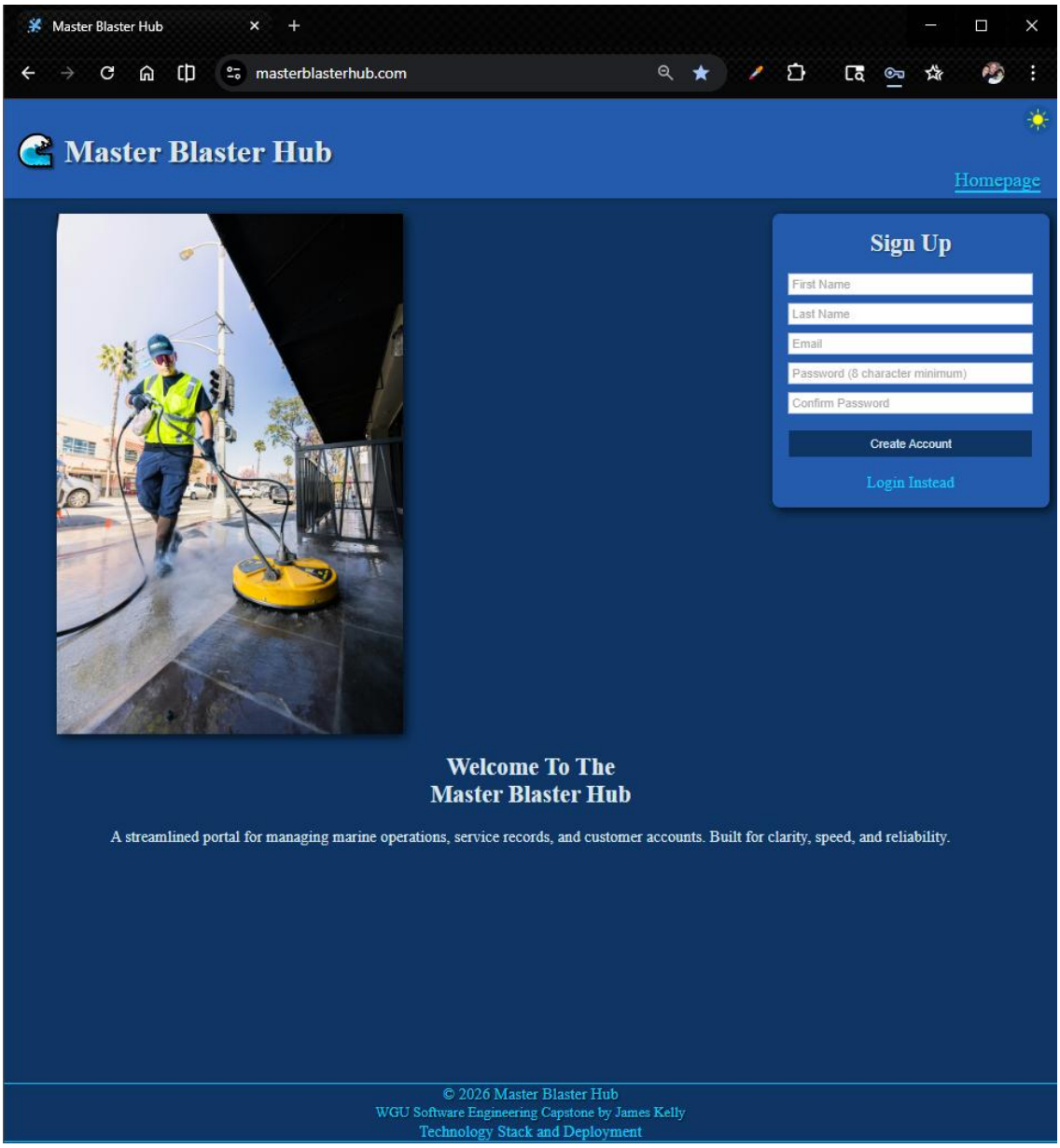


User Guide – User Perspective

Customer Guide

Accessing the Application

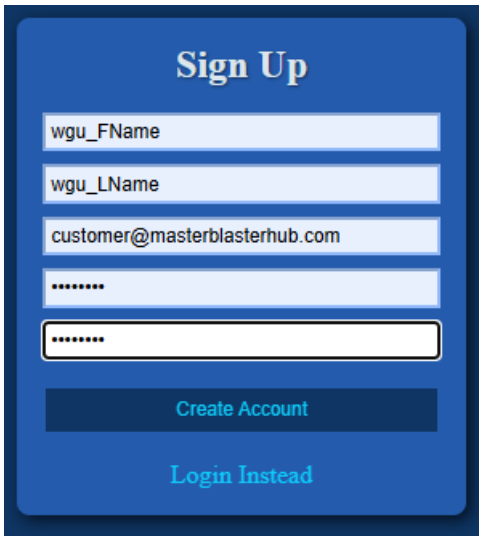
To begin using Master Blaster Hub, open a web browser such as Chrome, Edge, or Firefox and enter www.masterblasterhub.com in the address bar. When the site loads, you will arrive at the **Homepage**, which serves as the main entry point into the system. In the top-right corner, a light and dark mode toggle allows you to switch between display themes based on your preference. From the **Homepage**, new users may create an account, and returning users may log in to access their personalized dashboard.



Creating a New Account

If you are a new user, select **Create Account** from the **Homepage**. You will be prompted to enter your first name, last name, email address, and password. Ensure your email address is entered correctly, as it will be used for future logins.

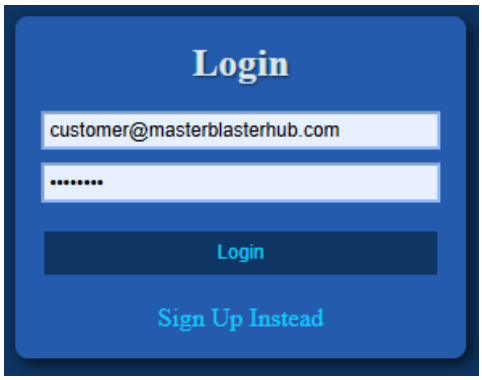
After completing all required fields, submit the form. If the information meets system validation requirements, your account will be created successfully, and you will be redirected to your **MyPortal** dashboard. If any required fields are missing or improperly formatted, an error message will appear prompting you to correct the information before continuing.



Logging In

Returning users may log in by selecting **Login** and entering their registered email address and password. After submitting valid credentials, you will be redirected to your **MyPortal** dashboard.

Once logged in, your name and assigned role will be displayed in the top navigation area. If incorrect credentials are entered, an error message will appear, and you will be prompted to try again.



Overview of MyPortal

After logging in, you will arrive at **MyPortal**, your central dashboard for managing service addresses and service requests. On the left side of the page, your account information is displayed, including your name, role, account ID, and email address.

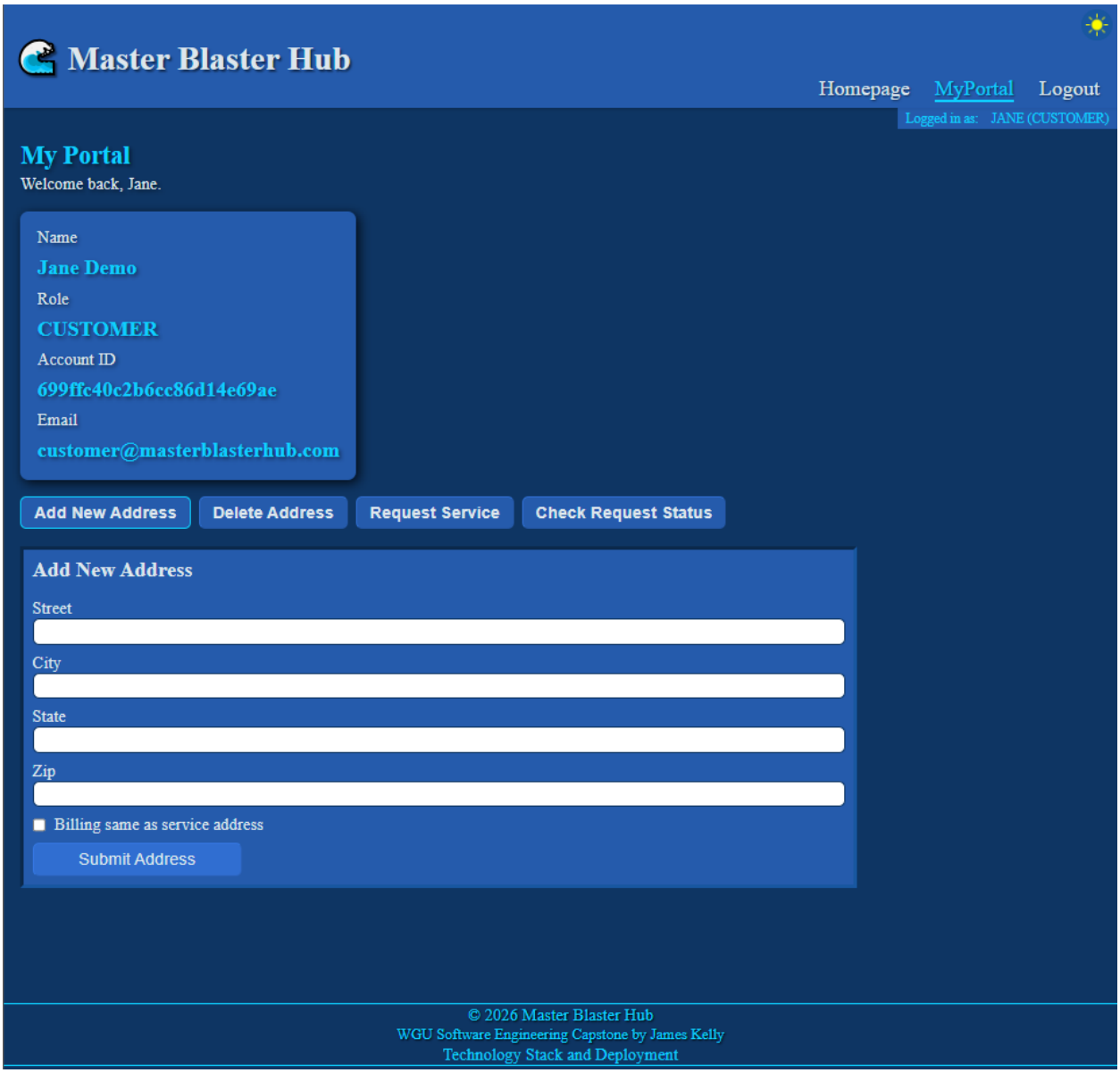
The lower portion of the page contains a dynamic content panel that updates based on the action selected. Available actions include **Add New Address**, **Delete Address**, **Request Service**, and **Check Request Status**. This layout allows you to manage all account-related tasks from a single location without navigating to separate pages.



Adding a New Address

To add a new service location, select **Add New Address**. A form will appear in the dynamic panel prompting you to enter the street address, city, state, and zip code. You may also indicate whether the billing address is the same as the service address.

After completing the form, select **Submit Address**. If the information is valid, the address will be saved to your account and will become available for future service requests.



Deleting an Address

To remove an address from your account, select **Delete Address**. A dropdown menu will display all addresses currently associated with your profile. Choose the address you wish to remove and confirm the action.

Once confirmed, the address will be permanently removed and will no longer be available for service requests.

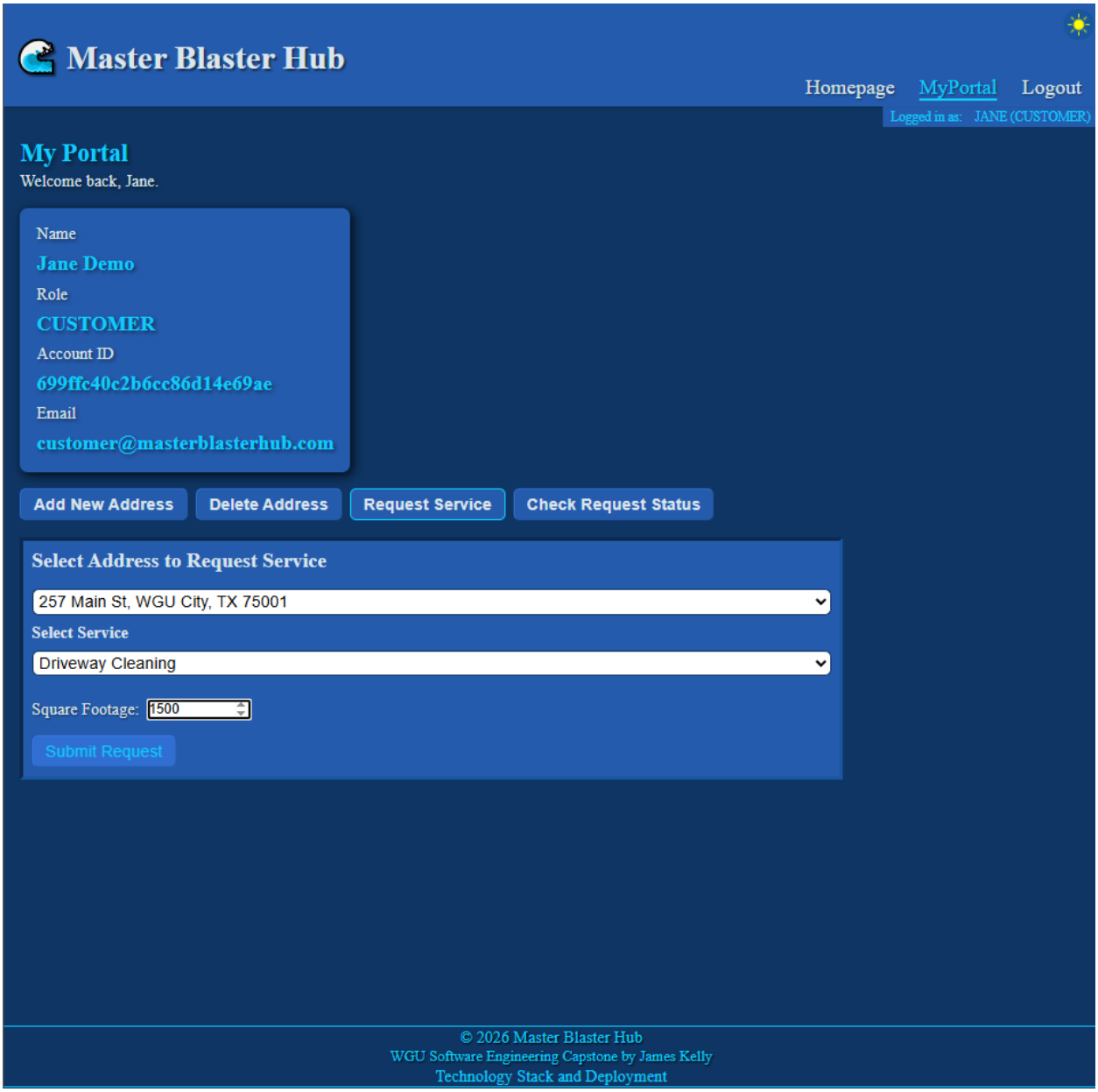


Requesting a Service

To submit a service request, select **Request Service**. Begin by choosing one of your saved addresses from the dropdown list. Next, select the type of service you would like to request and enter any required details, such as square footage for pricing calculations.

After completing all required fields, select **Submit Request**. Your request will be recorded and its status will update as it progresses through the review and approval process.

Note: A saved address is required in order to submit a service request.



Checking Request Status

To monitor your service requests, select **Check Request Status**. A table will display all submitted requests, including the service address, selected service type, quoted price, and current status.

Possible status values include **REQUESTED**, **QUOTED**, **APPROVED**, **COMPLETED**, **CANCELLED**, and **DECLINED**. If a request is marked **QUOTED**, you may have the option to approve or decline the quote directly from this panel.



Technology Stack & Deployment Page

At the bottom of the website, in the footer, you will find a link labeled **Technology Stack and Deployment**. Selecting this link opens a page that provides a categorized overview of the tools and platforms used to build and deploy the application.

Rather than offering in-depth technical explanations, this page presents a structured list of the frontend framework, backend components, database platform, and cloud infrastructure services that support the system.



Logging Out

When you are finished using the application, select **Logout** in the top navigation bar. Logging out securely ends your session and returns you to the **Homepage**.

For security reasons, especially when using a shared or public computer, it is recommended that you log out after each session.



Additional Notes for Users

You must be logged in to access **MyPortal**. Attempting to navigate directly to restricted pages without authentication will automatically redirect you to the login screen.

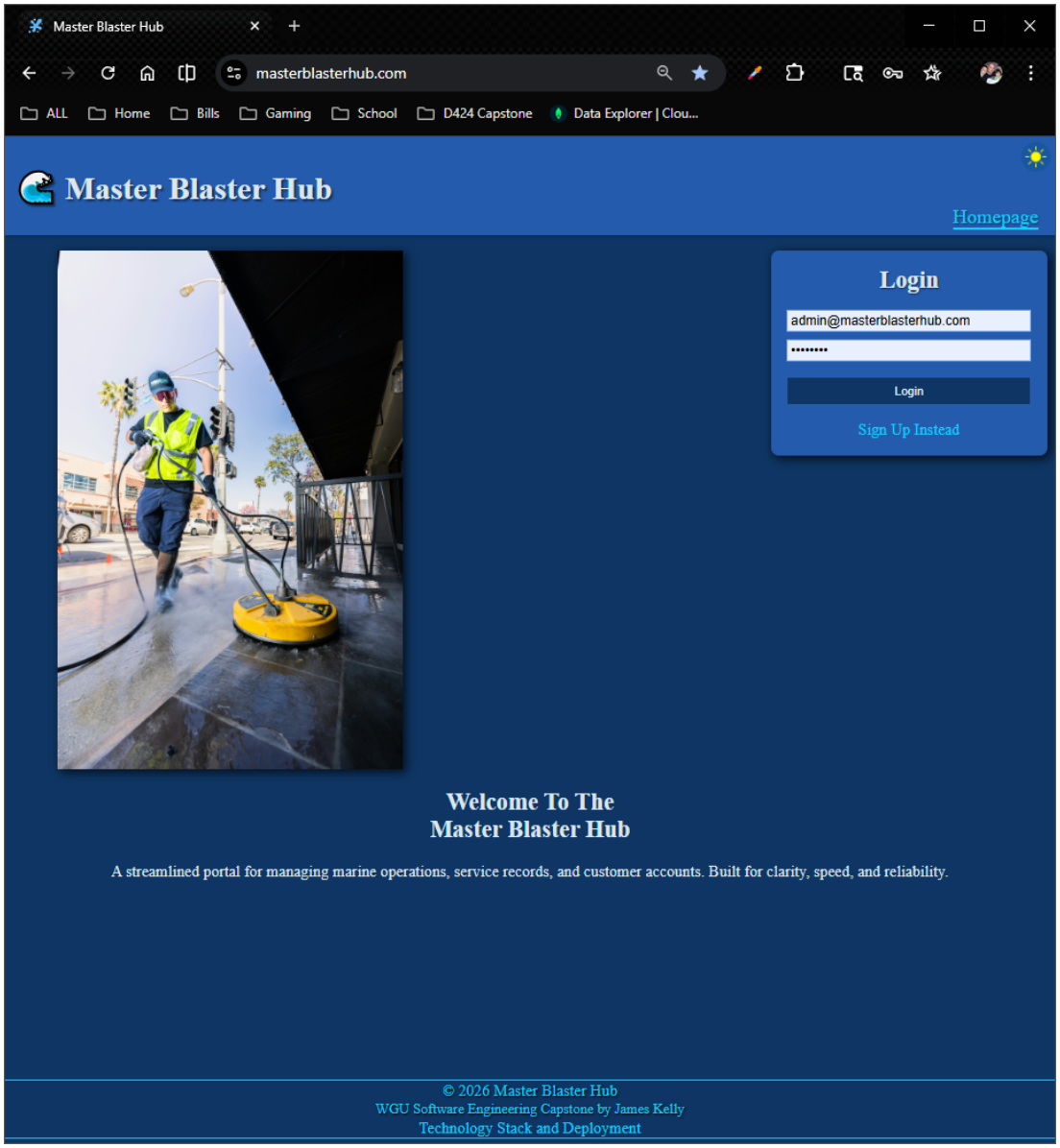
Certain features are controlled by your assigned role within the system. Standard customers have access to service management functions, while administrative features are reserved for authorized administrators.

Administrator Guide

Accessing the System

To begin, open a web browser and navigate to www.masterblasterhub.com. From the **Homepage**, select **Login** and enter your administrator credentials. After successful authentication, you will be redirected to the **Admin Page**.

Your assigned role will be displayed in the top-right corner to confirm administrator access. A light and dark mode toggle is also available in the upper-right corner for display preference.



Admin Dashboard Overview

After logging in, you will arrive at the **Admin Page**, which contains three primary action buttons: **Customers**, **Service Jobs**, and **Database Reset**. Beneath these controls is a dynamic content area that updates based on the selected action.

When first entering the page, the content area displays a prompt instructing you to select a function. This layout keeps administrative tools organized and allows you to focus on one management task at a time.



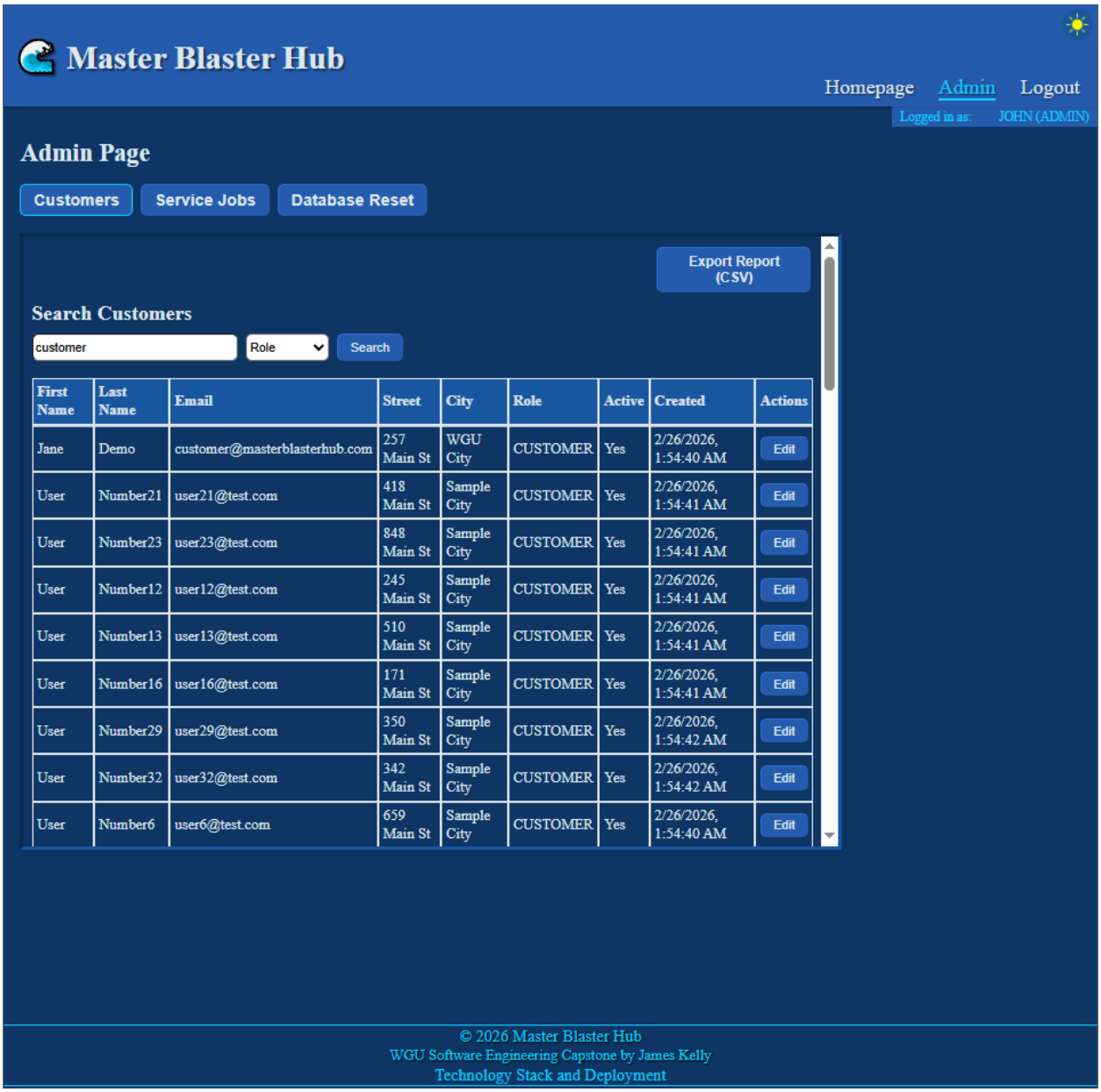
Customers Panel

Selecting **Customers** loads the customer management panel into the dynamic window. This panel allows administrators to locate users using one of three search criteria: email address, last name, or role.

After entering a value and selecting the desired filter, click **Search** to display matching records.

Results appear in a structured table containing first name, last name, email, street address, city, role, account status (active or inactive), account creation date, and available

actions. Each row includes an **Edit** option, enabling administrators to review or modify user account details when necessary.



Service Jobs Panel

Selecting **Service Jobs** loads the job management panel into the dynamic window. This section allows administrators to review and manage service requests submitted by customers.

Jobs may be searched using email address, city, service type, or job status. After entering the appropriate criteria, click **Search** to retrieve matching records. The **Refresh** button may be used to reload or reset the displayed results.

Search results are displayed in a detailed table showing customer name, contact email, service address, service type, quoted price, current job status, and available administrative actions. Depending on the job's state, action buttons such as **Quote**, **Complete**, or **Cancel** may appear. These controls allow administrators to update job progress and maintain accurate service records.

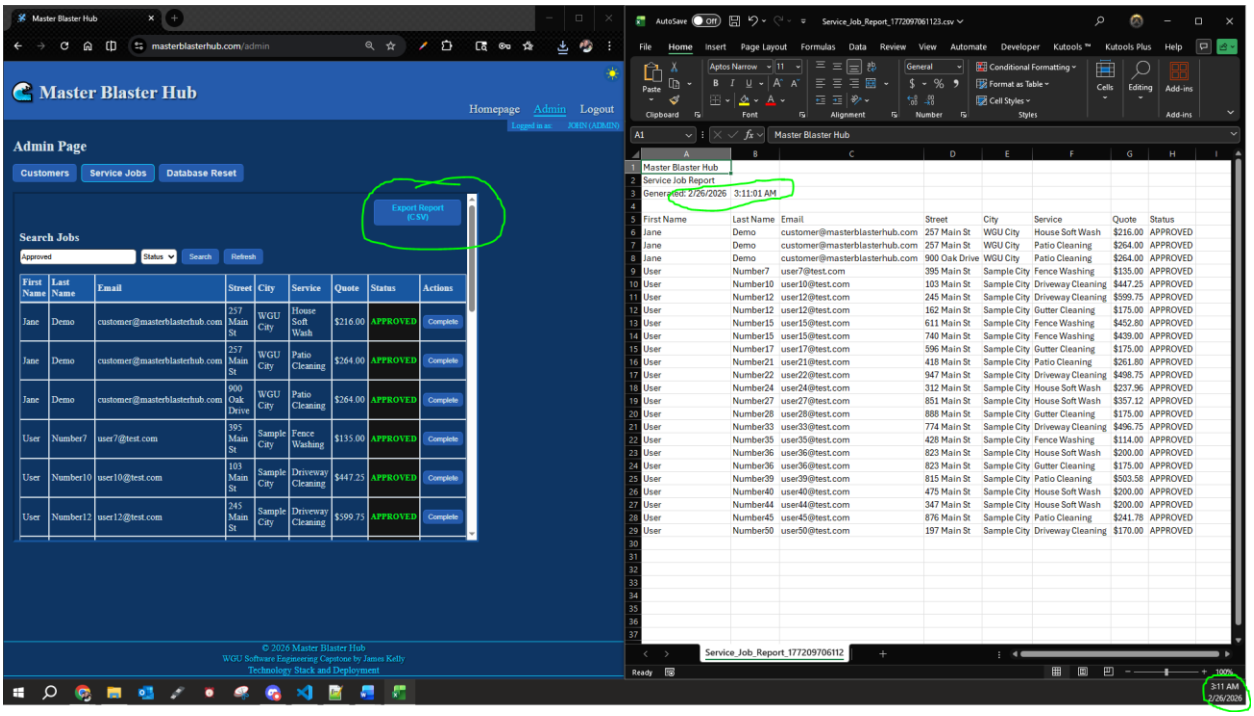


Report Export Feature

Within both the **Customers** and **Service Jobs** panels, administrators have access to the **Export Report (CSV)** button located in the dynamic window. This feature allows the currently displayed dataset to be downloaded as a CSV file.

When selected, the system generates a formatted report containing all visible records, along with a timestamp indicating when the report was created.

This export capability is particularly useful for record keeping, auditing, and sharing data externally. The generated CSV file can be opened in spreadsheet applications such as Microsoft Excel for sorting, filtering, and further analysis. By exporting only the filtered results currently displayed, administrators can produce targeted reports based on specific search criteria. This feature ensures that administrative data can be securely archived or distributed without manual transcription.



Database Reset Panel

Selecting **Database Reset** loads the reset confirmation panel. This feature restores the application database to its original default state and is intended strictly for emergency or maintenance scenarios.

Because this action permanently removes all existing data and replaces it with a predefined default dataset, **it is irreversible and should only be used as a last resort**. To prevent accidental data loss, the reset process requires deliberate confirmation. After initiating the reset, an additional confirmation step must be completed, and a system alert provides a final warning before execution.



Navigation and Logging Out

At any time, administrators may use the header navigation links to return to the **Homepage** or remain within the **Admin Page**. When administrative tasks are complete, select **Logout** in the top-right corner to securely end the session.

Logging out is an important security practice, particularly when using shared or public devices, as it prevents unauthorized access to administrative features.



User Guide - Initial Setup & Running the Application

1. System Architecture Overview

The Master Blaster Hub application is deployed using a cloud-based, containerized architecture designed to separate frontend delivery, backend processing, and data persistence into distinct layers. The system follows a client–server model with a statically hosted frontend, a containerized backend API, and an externally managed database service.

Frontend Layer

The frontend is a single-page application built with **React** using **Vite** as the build tool. The compiled static assets are deployed to an **Amazon S3** bucket configured for static website hosting. Content delivery is optimized through **Amazon CloudFront**, which distributes the frontend globally and provides HTTPS support via **AWS Certificate Manager (ACM)**.

Domain resolution is handled by **Amazon Route 53**, which routes traffic from **www.masterblasterhub.com** to the CloudFront distribution.

Frontend-to-backend communication occurs via HTTP requests using the configured environment variable **VITE_API_BASE_URL**, which targets the backend API endpoint.

Backend Layer

The backend is a RESTful API built with **Spring Boot** and packaged as a Docker container using a multi-stage **Dockerfile** build process.

Container images are stored in a private **Amazon Elastic Container Registry (ECR)** repository named **masterblasterhub-backend**. The backend is deployed using **Amazon Elastic Container Service (ECS)** with the **Fargate** launch type, which provides serverless container orchestration without requiring EC2 instance management.

The backend service runs within the ECS cluster **mbh-backend-cluster**, under the service **mbh-backend-service**, with a task definition versioned as **mbh-backend-task**.

Incoming traffic is routed through an internet-facing **Application Load Balancer (ALB)** named **mbh-backend-alb**, which forwards requests to the running ECS task. HTTPS termination is handled via certificates provisioned through **AWS Certificate Manager (ACM)**.

The backend exposes a health endpoint at **/api/health**, which is used for service validation and load balancer health checks.

Database Layer

Persistent data storage is provided by **MongoDB Atlas**, a managed cloud database service external to AWS. The backend connects to the database using environment variables:

- ✓ **SPRING_MONGODB_URI**
- ✓ **SPRING_MONGODB_DATABASE**

Database initialization and default data seeding are performed using the administrative **Reset Database** function within the application.

Continuous Integration and Deployment (CI/CD)

Automated deployment is implemented using **GitHub Actions** workflows.

Frontend workflow responsibilities:

- ✓ Install dependencies
- ✓ Build production assets
- ✓ Synchronize build output to **Amazon S3**
- ✓ Invalidate the associated **CloudFront** distribution

Backend workflow responsibilities:

- ✓ Build Docker image
- ✓ Authenticate with **Amazon ECR**
- ✓ Push container image to the private registry
- ✓ Trigger service update in **Amazon ECS**

Deployment authentication is managed using dedicated IAM users:

- ✓ **github-frontend-deployer**
- ✓ **github-backend-deployer**

Credentials are stored securely as repository secrets within GitHub.

Infrastructure Provisioning

Core infrastructure components (ECS cluster and ECS service) were created through the AWS console. As a result, supporting **CloudFormation** stacks were automatically generated to manage underlying infrastructure resources. No manually authored infrastructure-as-code templates are used in this deployment.

The system does not utilize EC2 virtual machines. All backend compute resources are provisioned through the **Fargate** serverless container model.

High-Level Traffic Flow

1. A user navigates to **https://masterblasterhub.com**.

2. **Route 53** resolves the domain to **CloudFront**.
3. **CloudFront** serves static frontend assets from **S3**.
4. The frontend issues API requests to **https://api.masterblasterhub.com/api/**.
5. **Route 53** resolves the API subdomain to the **Application Load Balancer**.
6. The ALB forwards traffic to the ECS service running in **Fargate**.
7. The backend communicates with **MongoDB Atlas** for data persistence.

2. Required Accounts and Software Prerequisites

1. Domain Registration and DNS Delegation

A publicly accessible domain is required for frontend and backend routing. The domain for this deployment was registered through **Squarespace Domains** and delegated to **Amazon Route 53** for DNS management.

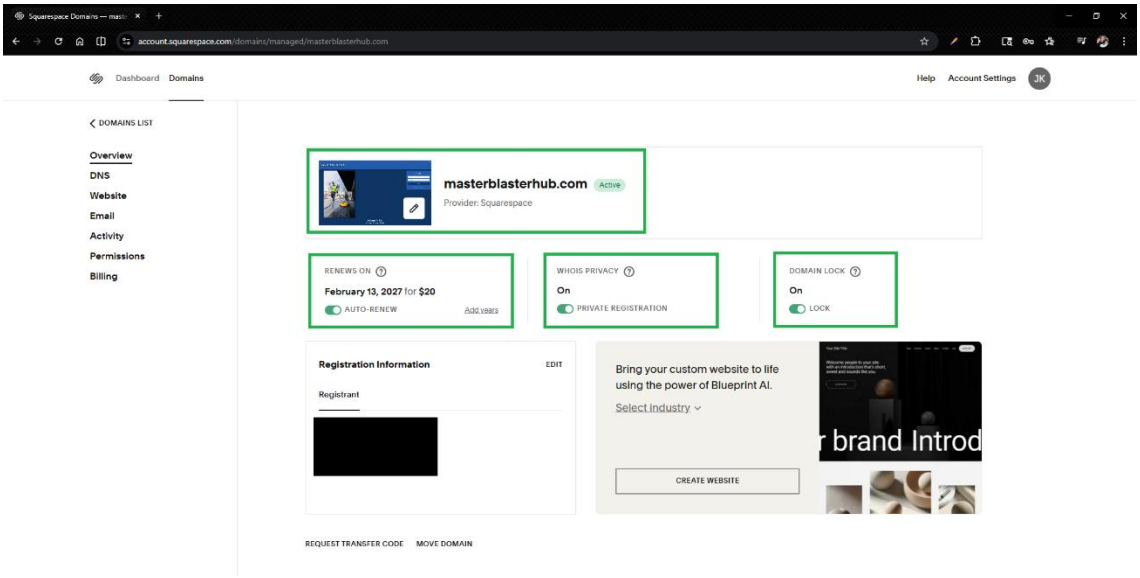
1.a Register Domain Through Squarespace

1. Navigate to **https://account.squarespace.com/domains**
2. Select **Register a Domain**
3. Search for the desired domain name (e.g., masterblasterhub.com)
4. Select the available domain
5. Proceed to checkout
6. Enable:
    - Auto-renew (recommended)
    - WHOIS privacy (recommended)
7. Complete purchase

After purchase, verify:

- ✓ Domain status displays as **Active**
- ✓ Domain lock is enabled
- ✓ Auto-renew is enabled

At this stage, DNS is still managed by Squarespace.



1.b Create Public Hosted Zone

An AWS account with administrative permissions is required to perform this step. The user must have access to the **AWS Management Console** with sufficient privileges to create and manage Route 53 hosted zones.

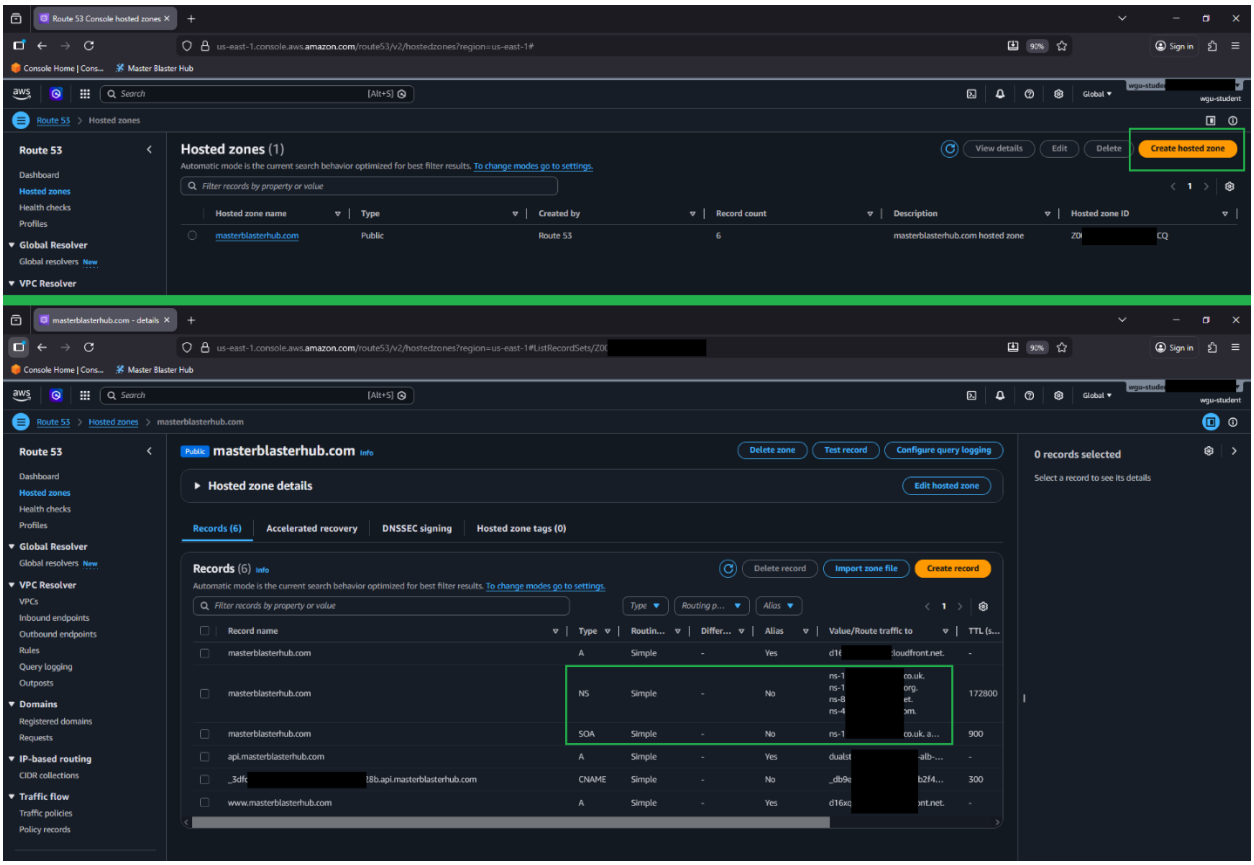
Create AWS Account (If Not Already Established)

1. Navigate to https://aws.amazon.com/
2. Select **Create an AWS Account**
3. Complete account registration
4. Log in to the **AWS Management Console**
5. Ensure the account has administrative access (either the root account or an IAM user with Administrator permissions)
6. In the AWS Management Console, navigate to **Route 53**
7. Select **Hosted zones**
8. Click **Create hosted zone**
9. Enter the following:
    a) Domain name: masterblasterhub.com
    b) Type: **Public hosted zone**
10. Click **Create hosted zone**

Upon creation, Route 53 will automatically generate:

- ✓ An **NS (Name Server)** record containing four AWS nameservers
- ✓ An **SOA (Start of Authority)** record

The four nameservers listed in the NS record must be copied and used in the domain registrar configuration (see Section 1.c).



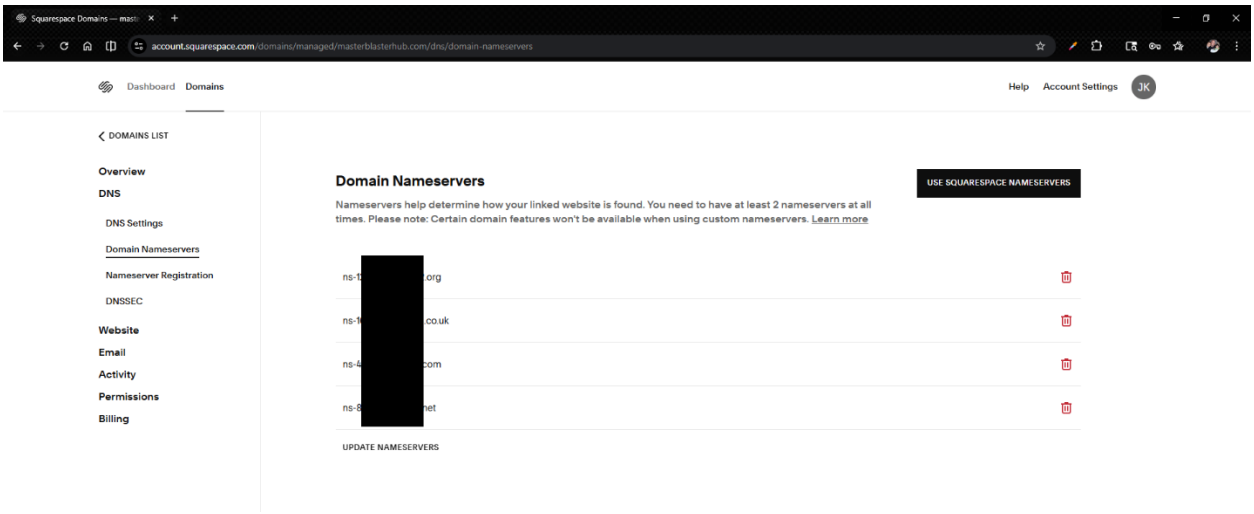
1.c Update Nameservers in Squarespace

1. Log in to **Squarespace Domains**
2. Select the registered domain
3. Navigate to **DNS Settings** or **Nameservers**
4. Replace the existing nameservers with the four AWS nameservers generated in Route 53
5. Save changes

DNS propagation may take several minutes to several hours.

After propagation:

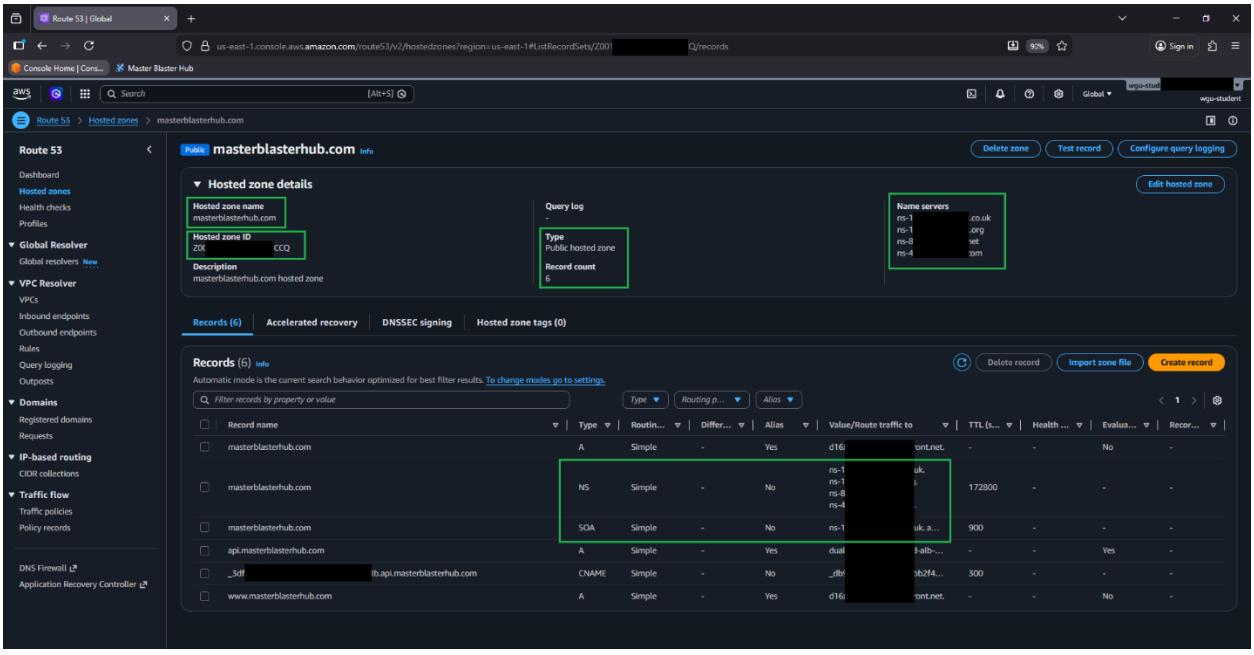
- ✓ Route 53 becomes the authoritative DNS provider
- ✓ All DNS records will be managed within AWS



1.d Verify Delegation

To verify successful delegation:

1. Navigate to **Route 53 > Hosted zones**
2. Select masterblasterhub.com
3. Confirm:
   - ✓ NS record matches the nameservers configured in Squarespace
   - ✓ Hosted zone is active
   - ✓ No DNS errors are present



Additionally, DNS delegation can be verified from a local machine by executing the command **nslookup masterblasterhub.com** from your terminal. A successful response

returning the AWS nameservers confirms that the domain is properly delegated to **Amazon Route 53** and that DNS resolution is functioning correctly.



2. Required Cloud Service Accounts

2.a Amazon Web Services (AWS) Account

Before any infrastructure components can be provisioned, an active **Amazon Web Services (AWS)** account must be established.

An AWS account with **administrative privileges** is required to create and configure the infrastructure resources used in this deployment. Administrative access may be provided through:

- ✓ The AWS root account (not recommended for daily use), or
- ✓ An **IAM user** with the **AdministratorAccess** policy attached
- ✓ Administrative permissions are necessary to:
- ✓ Create and manage **Amazon S3** buckets
- ✓ Configure **Amazon CloudFront** distributions
- ✓ Create and manage **Amazon Route 53** hosted zones
- ✓ Issue certificates through **AWS Certificate Manager (ACM)**
- ✓ Create repositories in **Amazon Elastic Container Registry (ECR)**
- ✓ Provision clusters and services in **Amazon Elastic Container Service (ECS)**
- ✓ Configure an **Application Load Balancer (ALB)**
- ✓ Manage IAM users, roles, and policies

AWS Region Consistency:

All services must be deployed within the same AWS region to ensure compatibility and simplified networking configuration.

For this deployment, the selected region is: **us-east-1 (N. Virginia)**

The region must remain consistent across:

- ✓ **ECR repositories**
- ✓ **ECS clusters and services**
- ✓ **Application Load Balancer**

- ✓ **ACM certificates**
- ✓ **CloudFront (certificate must exist in us-east-1)**

Region inconsistency can result in failed certificate validation, networking errors, or load balancer misconfiguration.

2.b AWS Identy and Access Management (IAM)

AWS Identity and Access Management (IAM) is used to securely control programmatic deployment access from GitHub Actions to AWS infrastructure resources.

Rather than using the root account for automation, two dedicated IAM users were created to isolate deployment responsibilities:

- ✓ **github-frontend-deployer**
- ✓ **github-backend-deployer**

Each user is configured with programmatic access (Access Key ID and Secret Access Key) and is assigned policies specific to its deployment role.

This approach follows the principle of least privilege and prevents unnecessary exposure of administrative credentials.

2.b.1 Frontend Deployment IAM User

The **github-frontend-deployer** IAM user is responsible for deploying static frontend build artifacts to **Amazon S3** and invalidating the **CloudFront** distribution.

This user has two attached policies:

a) frontend-deployer-github-actions-s3-policy

This custom policy grants restricted access to the production S3 bucket:

Allowed actions:

- ✓ **s3:ListBucket**
- ✓ **s3:GetObject**
- ✓ **s3:PutObject**
- ✓ **s3:DeleteObject**

Restricted to:

- ✓ arn:aws:s3:::masterblasterhub.com
- ✓ arn:aws:s3:::masterblasterhub.com/*

This policy limits access strictly to the production bucket and does not grant permissions to other S3 resources.

b)GitHubFrontendDeployPolicy

This policy extends frontend deployment capabilities to include CloudFront invalidation.

Additional allowed action:

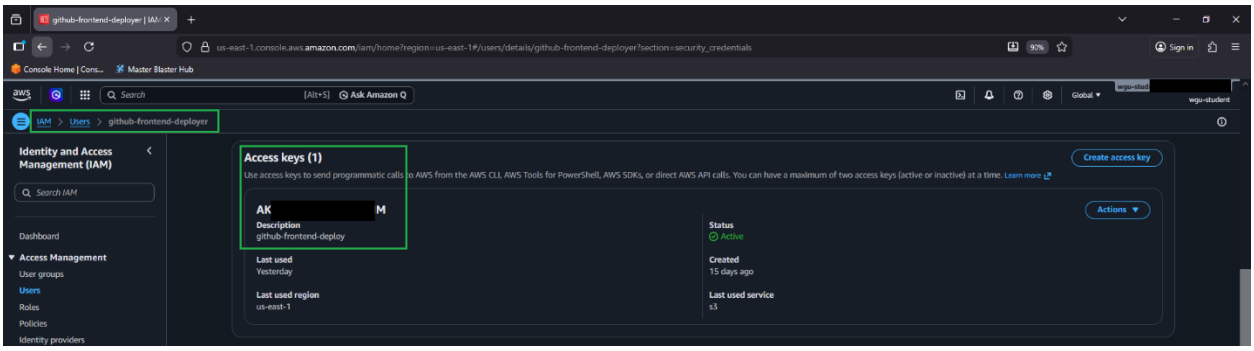
- **cloudfront:CreateInvalidation**

Restricted to:

- The specific CloudFront distribution ARN associated with the application

This permission is required because after uploading new frontend assets, the GitHub Actions workflow invalidates the CloudFront cache to ensure updated content is immediately served.

The Access Key ID and Secret Access Key for this IAM user can be generated and viewed under the **Security credentials** tab within the IAM user details page, specifically in the **Access keys** section.



2.b.2 Backend Deployment IAM User

The **github-backend-deployer** IAM user is responsible for:

- ✓ Building and pushing Docker images to **Amazon Elastic Container Registry (ECR)**
- ✓ Updating the running service in **Amazon Elastic Container Service (ECS)**

This user has the following policies attached:

a) AmazonEC2ContainerRegistryFullAccess

Allowed actions:

- ✓ Authenticate to ECR
- ✓ Push Docker images
- ✓ Manage repository layers

Includes:

- ✓ **ecr:*** permissions
- ✓ Required service-linked role creation for ECR replication

This policy enables the GitHub workflow to push newly built container images into the private ECR repository.

b) AmazonECS_FullAccess

Allowed actions:

- ✓ Register new task definitions
- ✓ Update ECS services
- ✓ Manage load balancer integrations
- ✓ Manage autoscaling policies
- ✓ Interact with CloudWatch logs
- ✓ Perform infrastructure integrations required by ECS

This policy allows the GitHub Actions workflow to trigger deployment updates to the running ECS service after a new container image is pushed.

c) AllowPassRoleForECS

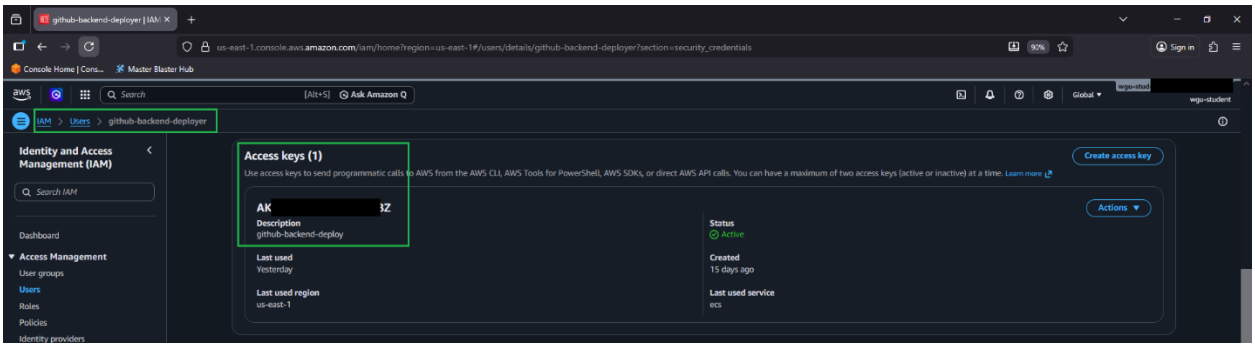
This custom policy permits:

- ✓ **iam:PassRole**

This is required because ECS services and tasks operate using execution roles. The deployment user must have permission to pass those roles to the ECS service during updates.

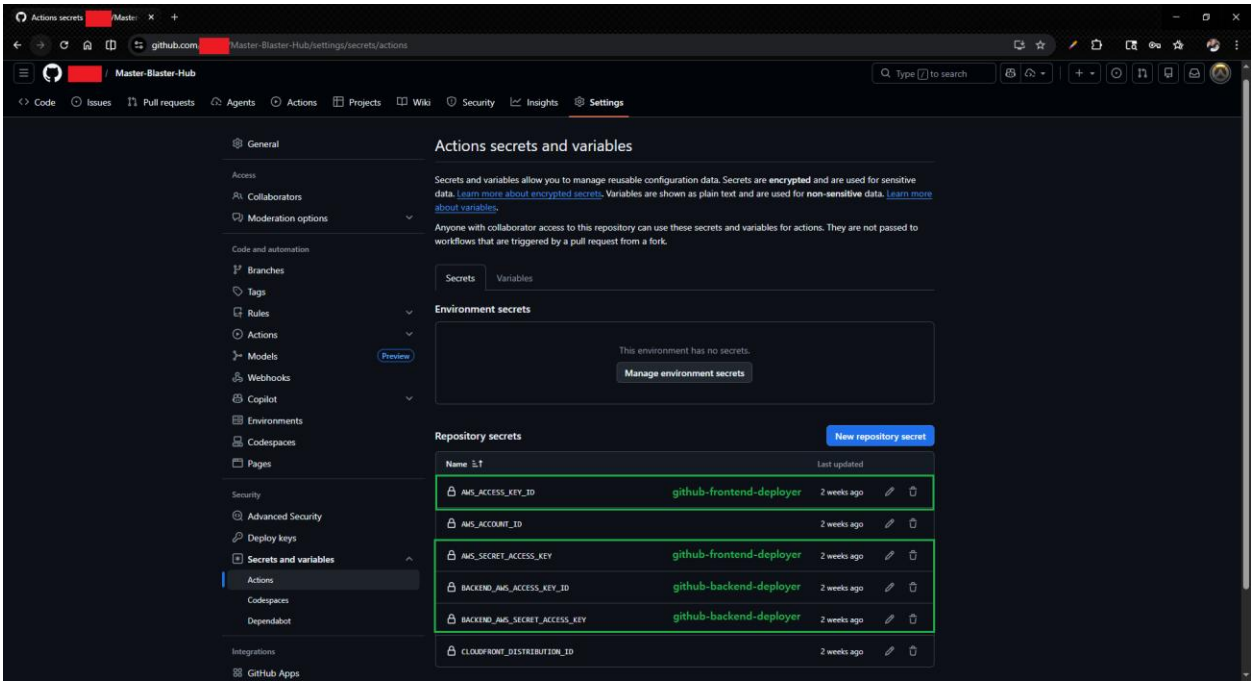
Without **iam:PassRole**, ECS service updates would fail during deployment.

The Access Key ID and Secret Access Key for this IAM user can be generated and viewed under the **Security credentials** tab within the IAM user details page, specifically in the **Access keys** section. You will need this for 2.b.3



2.b.3 Secure Credential Storage

The Access Key ID and Secret Access Key generated for each IAM user are stored securely as encrypted repository secrets within **GitHub Actions** under **Settings > Secrets and variables > Actions**. These credentials are referenced by the deployment workflows and are never committed to source control. Storing credentials as encrypted secrets ensures they are masked during workflow execution and protected from unauthorized access.



2.c Amazon S3 – Frontend Hosting

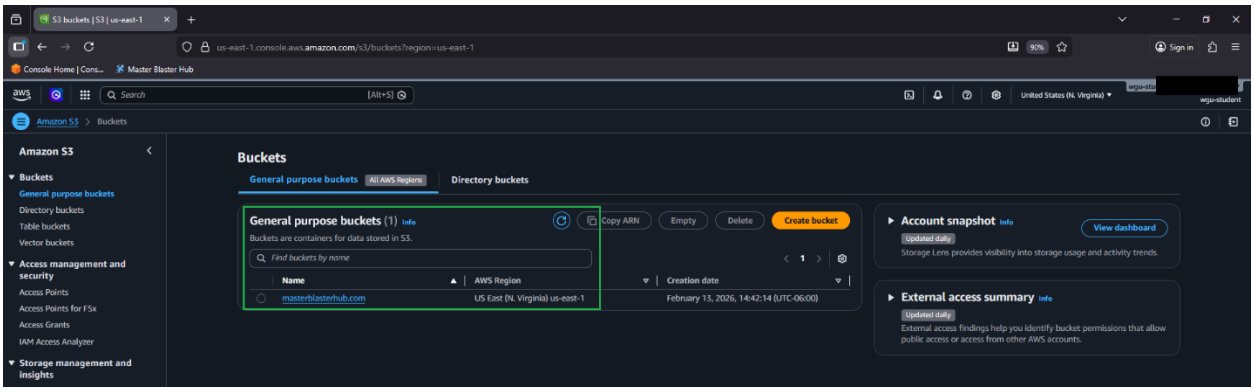
An Amazon S3 bucket was created to host the compiled React frontend as a static website.

2.c.1 Create S3 Bucket

1. Sign in to the AWS Management Console.
2. Navigate to **Amazon S3 > Buckets**.

3. Select **Create bucket**.
4. Configure the following settings:
   - ✓ **Bucket name:** masterblasterhub.com
   - ✓ **Region:** us-east-1 (must match other deployed services)
5. Leave default settings unless otherwise required.
6. Select **Create bucket**.

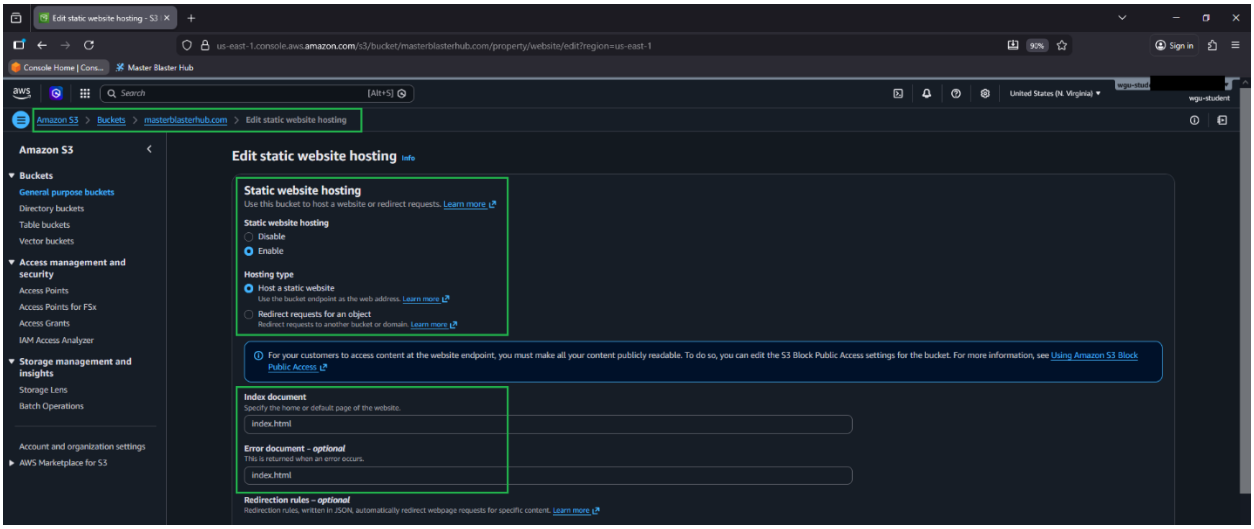
After creation, the bucket appears under **General purpose buckets**, confirming successful provisioning.



2.c.2 Configure Static Website Hosting

1. Open the newly created bucket.
2. Navigate to the **Properties** tab.
3. Scroll to **Static website hosting**.
4. Select **Enable**.
5. Specify:
   - **Index document:** index.html
   - **Error document:** index.html (required for SPA routing)
6. Save changes.

Using index.html as both the index and error document ensures that client-side routing within the React single-page application functions correctly.



2.c.3 Configure Bucket Policy for Deployment

The github-frontend-deployer IAM user was granted permission to:

- ✓ List the bucket
- ✓ Upload objects
- ✓ Retrieve objects
- ✓ Delete objects

This allows GitHub Actions to automatically deploy new frontend builds to the S3 bucket.

Public access to the bucket itself is controlled through CloudFront rather than exposing the bucket directly.

2.c.4 Region Consistency

The S3 bucket was created in:

US East (N. Virginia) – us-east-1

This region must remain consistent with:

- ✓ ACM certificates (for CloudFront)
- ✓ ECS services
- ✓ Application Load Balancer
- ✓ Route 53 configuration

Maintaining a single region avoids cross-region configuration conflicts and certificate limitations.

2.c.5 Frontend Deployment Automation

The frontend deployment is automated using a GitHub Actions workflow file located at:

.github/workflows/deploy-frontend.yml

This workflow performs the following steps:

1. Installs frontend dependencies.
2. Builds the React application using **npm run build**.
3. Authenticates to AWS using the **github-frontend-deployer** IAM user credentials.
4. Uploads the compiled **dist** folder to the S3 bucket.
5. Creates a CloudFront invalidation to clear cached content.

A simplified excerpt of the workflow: (for entire code see Appendix A)

```
      # Build production bundle
      - name: Build frontend
        working-directory: frontend
        run: npm run build

      # Configure AWS credentials
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      # Upload build output to S3
      - name: Deploy to S3
        run: aws s3 sync frontend/dist s3://masterblasterhub.com --delete

      # Clear CloudFront cache so new files are served
      - name: Invalidate CloudFront cache
        run: |
          aws cloudfront create-invalidation \
            --distribution-id ${{ secrets.CLOUDFRONT_DISTRIBUTION_ID }} \
            --paths "/*"
```

This ensures that any push to the designated branch automatically rebuilds and redeploys the frontend.

2.d AWS Certificate Manager (ACM)

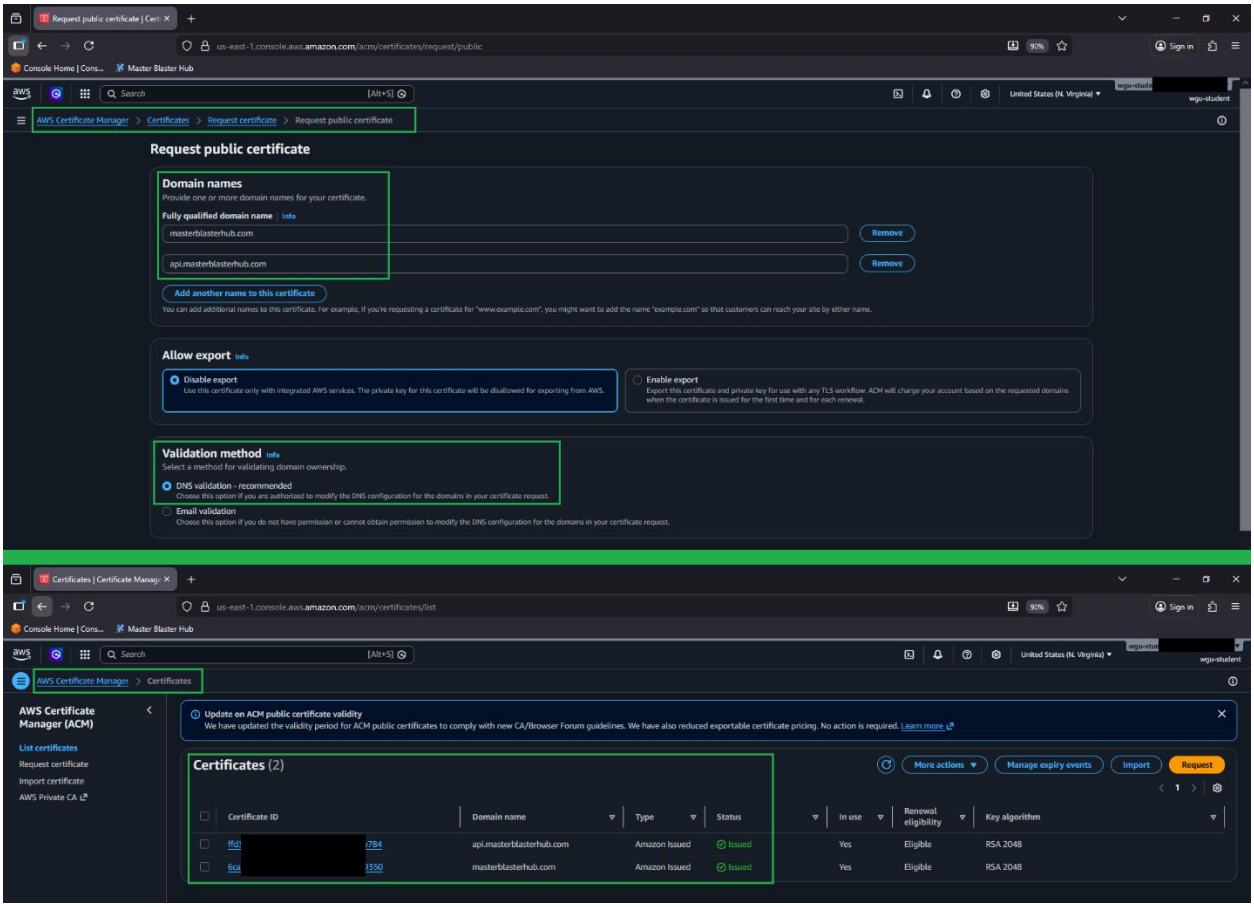
AWS Certificate Manager (ACM) was used to provision SSL/TLS certificates for secure HTTPS communication.

2.d.1 Request Public Certificates

1. Navigate to **AWS Certificate Manager (ACM)**.
2. Confirm region is set to **us-east-1**.
3. Select **Request certificate**.
4. Choose **Request a public certificate**.
5. Enter the following domain names:
   - ✓ masterblasterhub.com

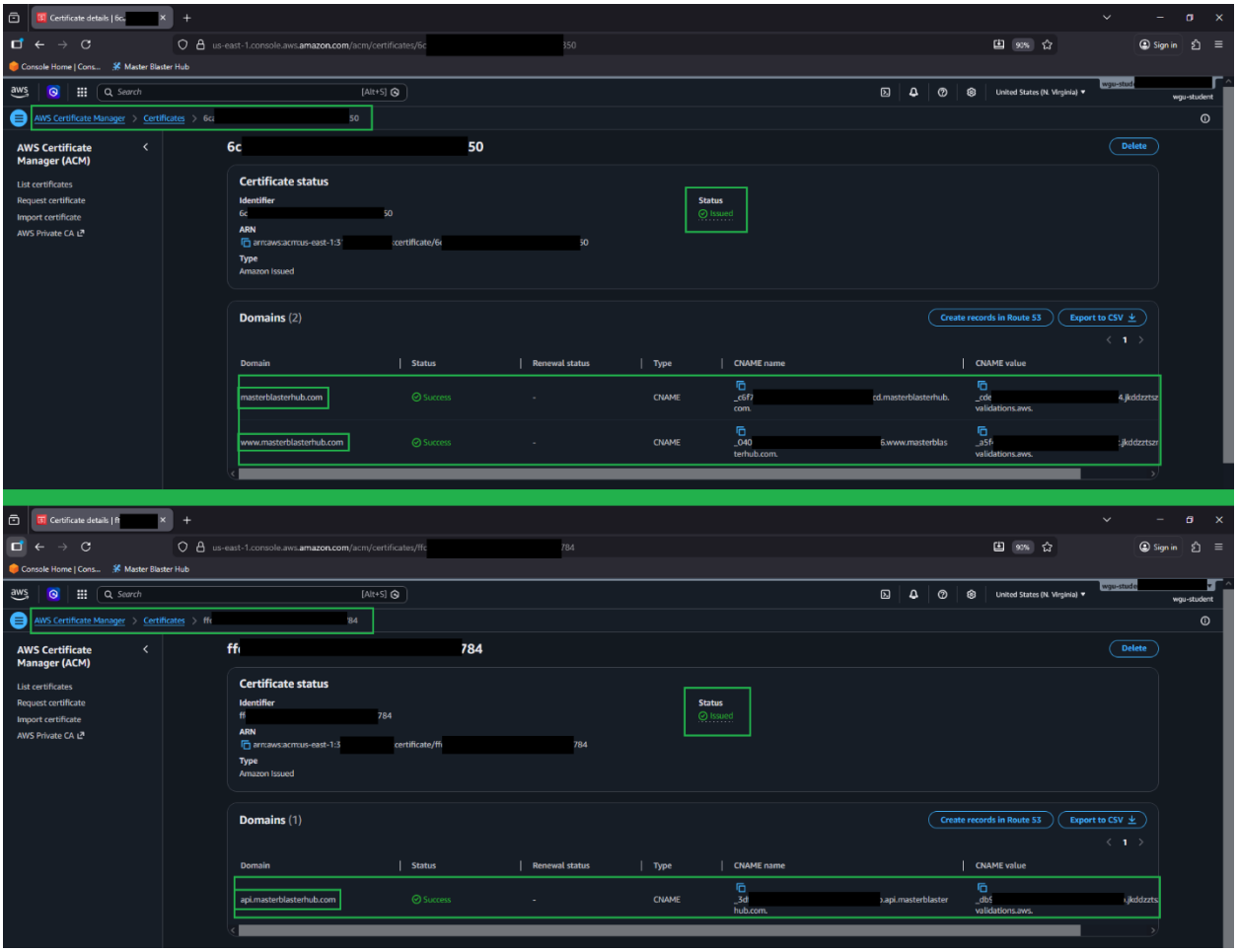
- ✓ api.masterblasterhub.com

6. Choose **DNS validation**.
7. Submit the request.



2.d.2 DNS Validation

1. After requesting the certificate, ACM provides CNAME validation records.
2. These CNAME records were added to the **Route 53 hosted zone**.
3. Once DNS propagation completed, certificate status changed to **Issued**.



2.d.3 Certificate Usage

The certificates are actively associated with:

- ✓ CloudFront distribution (frontend)
- ✓ Application Load Balancer (backend API)

ACM manages automatic renewal as long as DNS validation records remain in place.

2.e Amazon Cloudfront – Frontend Distribution

CloudFront requires certificates to be created in the **us-east-1 (N. Virginia)** region.

Amazon CloudFront was configured as a content delivery network (CDN) in front of the S3 bucket to:

- ✓ Enable HTTPS

- ✓ Improve global performance
- ✓ Secure bucket access
- ✓ Provide custom domain support

1) Create Distribution

1. Navigate to **CloudFront > Distributions**.
2. Select **Create distribution**.
3. Choose **Single website or app** as the distribution type.
4. Provide:
   - ✓ **Distribution name:** master-blaster-hub-cloudfront-distribution
   - ✓ **Description:** CloudFront distribution for masterblasterhub.com frontend
5. Select **Next**.

2) Specify Origin

1. Select **Origin type:** Amazon S3.
2. Choose the S3 bucket:
   - ✓ masterblasterhub.com
3. Leave **Origin path** blank.
4. Enable:
   - ✓ ✓ Allow private S3 bucket access to CloudFront (Origin Access Control).
5. Leave:
   - ✓ Origin settings > Use recommended origin settings.
   - ✓ Cache settings > Use recommended cache settings tailored to serving S3 content.
6. Select **Next**.

This configuration ensures:

- ✓ The S3 bucket is not publicly exposed.
- ✓ Only CloudFront can retrieve content from the bucket.

3) Enable Security

1. Leave Web Application Firewall (WAF) at default.
2. Do not enable monitor mode.
3. Proceed to the next step.

4) TLS and Domain Configuration

The following HTTPS configuration was applied:

1. **Alternate domain names (CNAMEs):**

- ✓ masterblasterhub.com
- ✓ www.masterblasterhub.com

2. **Custom SSL certificate:**
   - ✓ ACM certificate for masterblasterhub.com (Issued in us-east-1)
3. **Viewer protocol policy:**
   - ✓ Redirect HTTP to HTTPS
4. **Default root object:**
   - ✓ index.html
5. **Security policy:**
   - ✓ TLSv1.2_2021

These settings ensure:

- ✓ All traffic is encrypted.
- ✓ The custom domain uses a valid SSL certificate.
- ✓ React client-side routing functions properly.

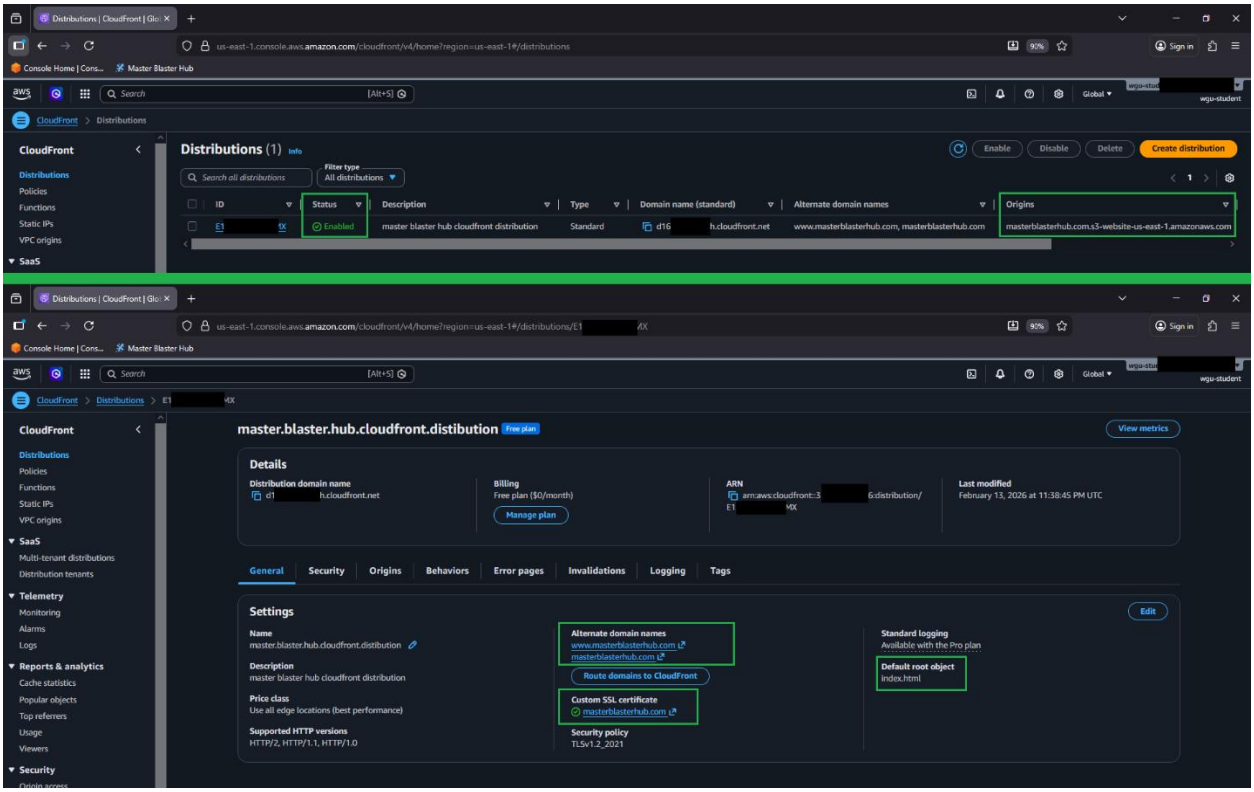
5) Create Distribution

After reviewing settings, the distribution was created.

Once deployed, the distribution status changed to:

- ✓ **Enabled**

The distribution domain name was automatically generated and later used in Route 53 Alias records.



2.f Amazon Elastic Container Registry (ECR)

Amazon Elastic Container Registry (ECR) was used to store Docker images for the backend Spring Boot application. This repository allows container images to be securely pushed from GitHub Actions and pulled by Amazon ECS during deployment.

The repository was created in: **US East (N. Virginia) – us-east-1**

2.f.1 Create Private Repository

1. Navigate to **Amazon ECR > Private registry > Repositories**.
2. Select **Create private repository**.
3. Configure the following:

General Settings

- ✓ **Repository name: masterblasterhub-backend**

Image Tag Settings

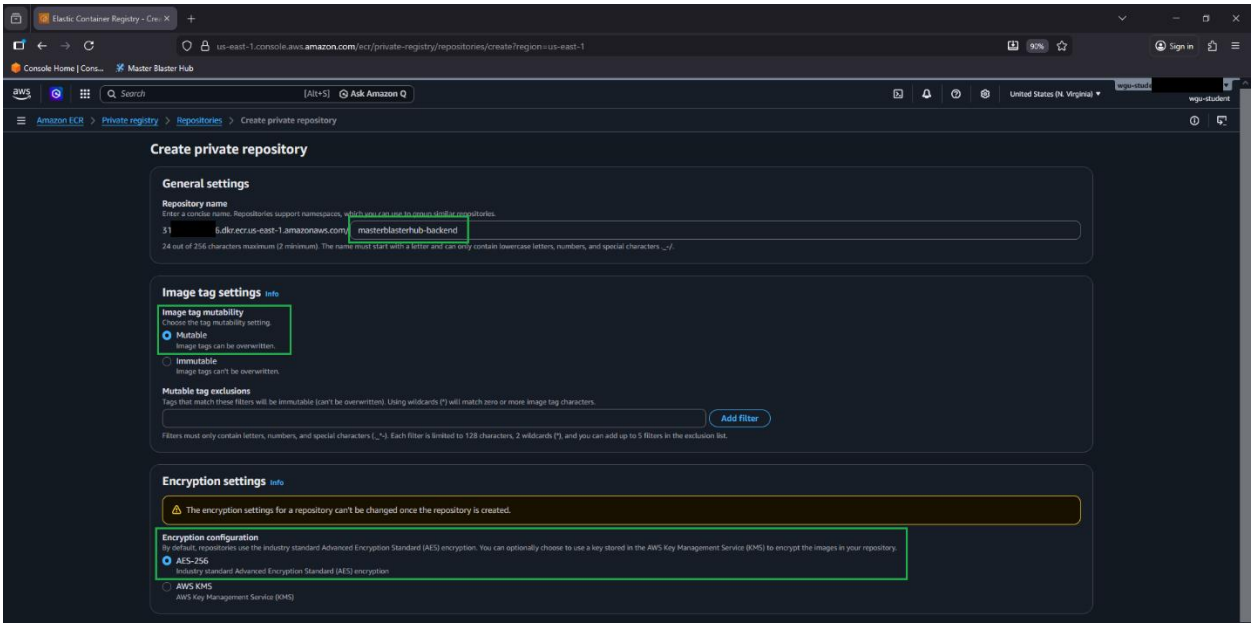
- ✓ **Image tag mutability:** Mutable

(This allows images such as **latest** to be overwritten during automated deployments.)

Encryption Settings

- ✓ **Encryption type:** AES-256 (Default)

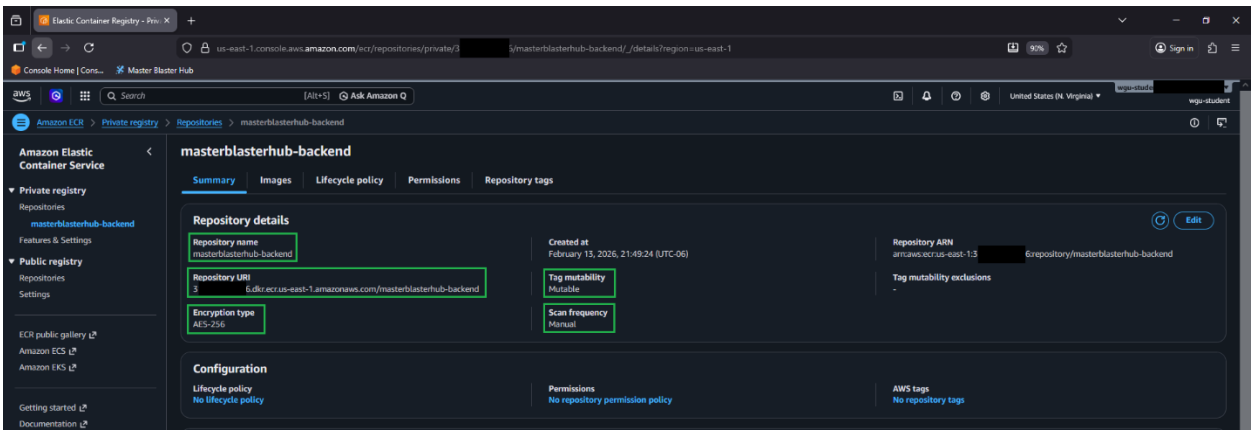
4. Select **Create repository**.



2.f.2 Repository Configuration

After creation, the repository displays the following:

- ✓ **Repository name: masterblasterhub-backend**
- ✓ **Repository URI:** xxxxxxxxxxxx.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-backend
- ✓ **Encryption type:** AES-256
- ✓ **Tag mutability:** Mutable
- ✓ **Scan frequency:** Manual



2.f.3 Deployment Integration

The backend deployment process is automated using a GitHub Actions workflow file located in the repository under:

.github/workflows/deploy-backend.yml

This workflow performs the following actions:

1. Authenticates to Amazon ECR using the github-backend-deployer IAM credentials.
2. Builds the Docker image from the backend source code.
3. Tags the image using the ECR repository URI.
4. Pushes the image to: xxxxxxxxxxxx.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-backend
5. Updates the Amazon ECS service to deploy the new container image.

A simplified excerpt of the workflow: (for entire code see Appendix B)

```
# Authenticate Docker to Amazon ECR
      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      # Build backend Docker image and tag for ECR
      - name: Build Docker image
        run: |
          docker build -t masterblasterhub-backend ./backend
          docker tag masterblasterhub-backend:latest \
            ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-backend:latest

      # Push image to Amazon ECR repository
      - name: Push Docker image to ECR
        run: |
          docker push \
            ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-backend:latest
```

2.g Deploying the Backend with Amazon ECS (Fargate)

This section describes how to deploy the Spring Boot backend container to Amazon ECS using Fargate and expose it through an Application Load Balancer.

2.g.1 Create Required Security Groups

Security groups must be created before the load balancer and ECS service.

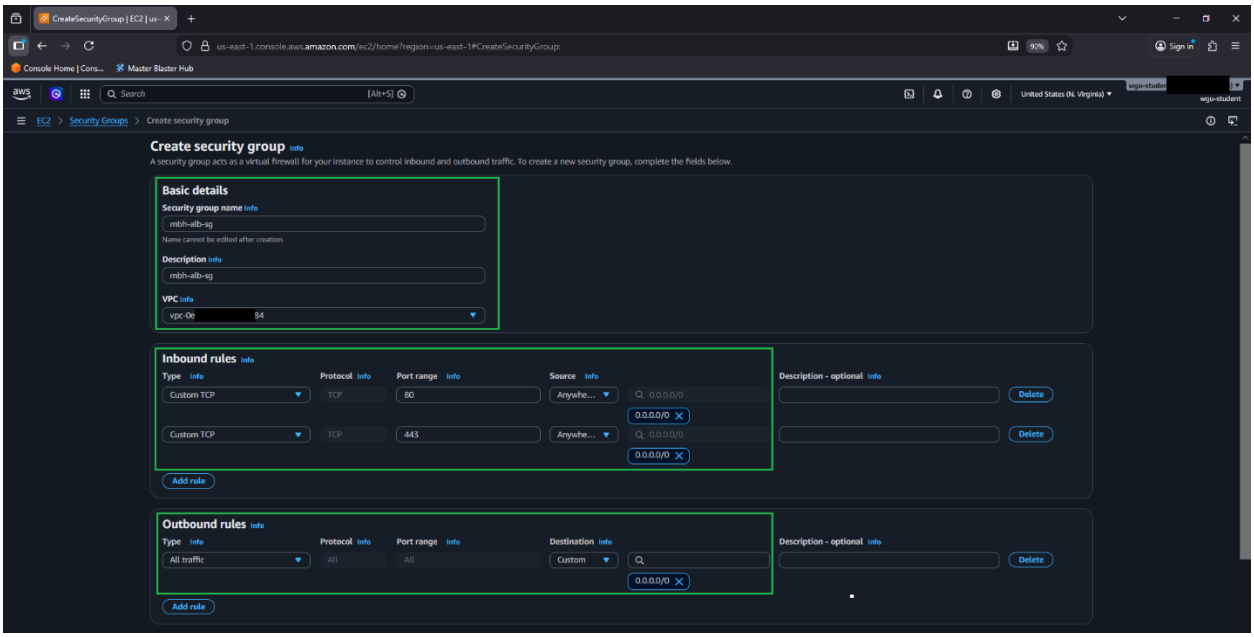
2.g.1.a Create the Application Load Balancer Security Group

1. Navigate to **EC2 > Security Groups**
2. Click **Create security group**
3. Configure:
   - ✓ **Security group name:** mbh-alb-sg
   - ✓ **Description:** mbh-alb-sg

- ✓ **VPC:** vpc-xxxxxxxxxxxx5ae84 (default)

4. Under **Inbound rules**, add:
   - ✓ HTTP – TCP – Port 80 – Source: 0.0.0.0/0
   - ✓ HTTPS – TCP – Port 443 – Source: 0.0.0.0/0
5. Leave **Outbound rules** as default:
   - ✓ All traffic > 0.0.0.0/0
6. Click **Create security group**

This allows public web traffic to reach the load balancer.

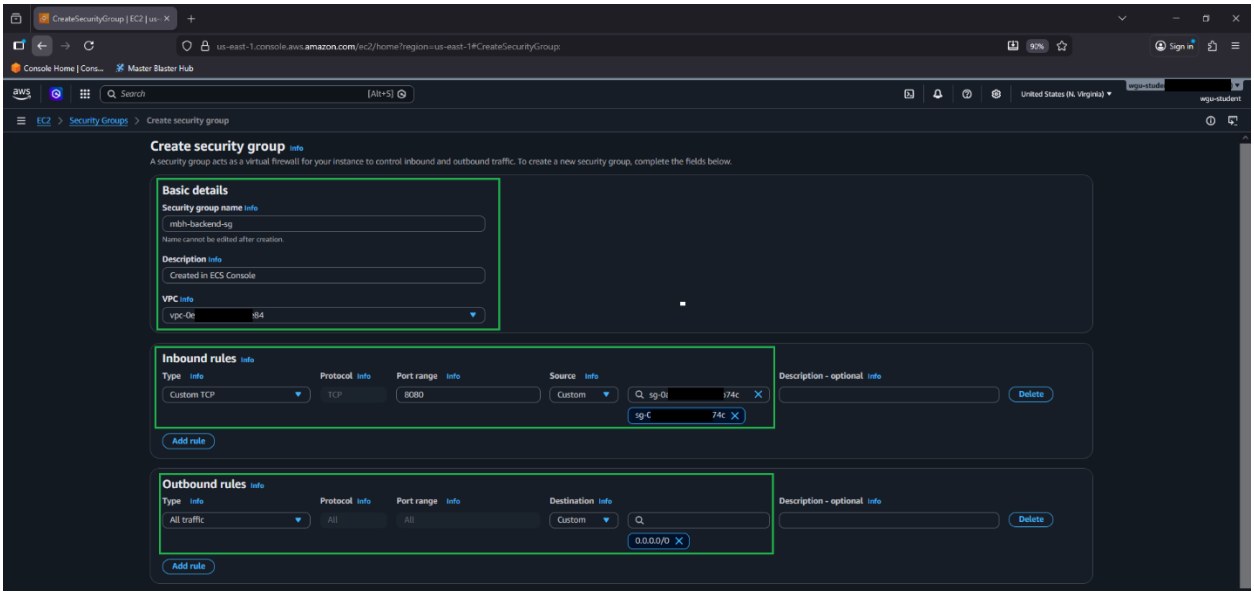


2.g.1.b Create the ECS Backend Security Group

1. Navigate to **EC2 > Security Groups**
2. Click **Create security group**
3. Configure:
   - ✓ **Security group name:** mbh-backend-sg
   - ✓ **Description:** Created in ECS Console
   - ✓ **VPC:** vpc-0xxxxxxxxxxxxae84
4. Under **Inbound rules**, add:
   - ✓ Type: Custom TCP
   - ✓ Port: 8080
   - ✓ Source: mbh-alb-sg

This restricts backend access to traffic coming only from the load balancer.

5. Leave **Outbound rules** as default:
   - ✓ All traffic > 0.0.0.0/0
6. Click **Create security group**



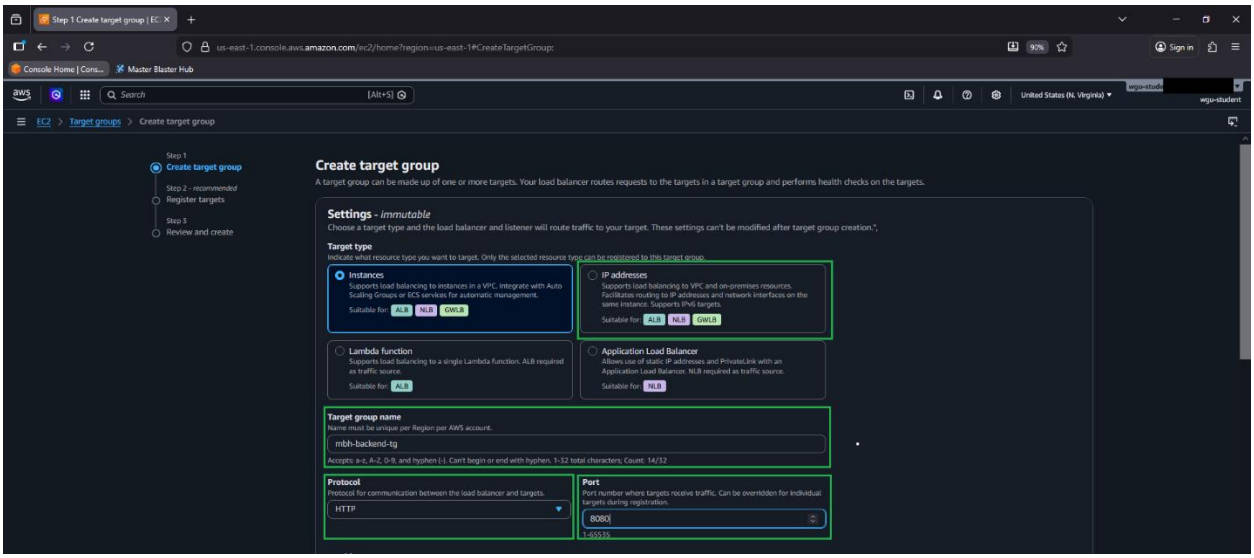
2.g.2 Create the Target Group

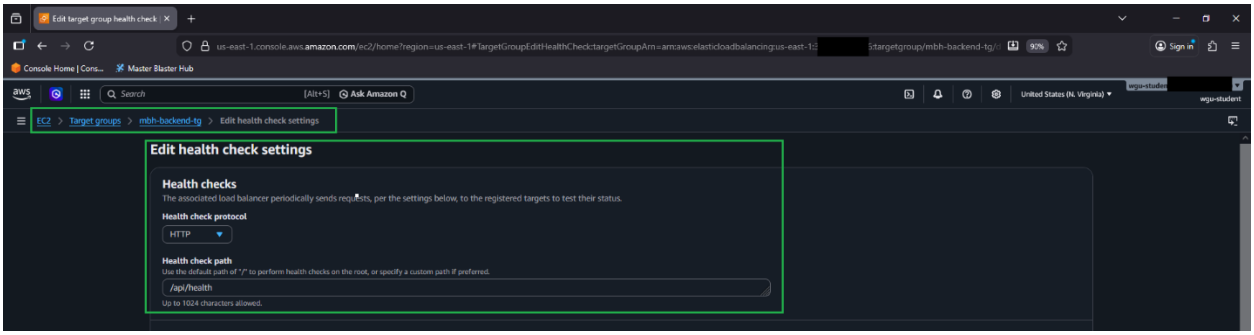
1. Navigate to **EC2 > Target Groups**
2. Click **Create target group**
3. Configure:
    - ✓ **Target type:** IP
    - ✓ **Protocol:** HTTP
    - ✓ **Port:** 8080
    - ✓ **Protocol version:** HTTP1
    - ✓ **VPC:** vpc-0xxxxxxxxxxxxae84
4. Click **Next**
5. Do not manually register targets (ECS will register automatically)
6. Click **Create target group**

Configure Health Check Settings

1. Open target group **mbh-backend-tg**
2. Go to **Health checks**
3. Configure:
    - ✓ Protocol: HTTP
    - ✓ Path: /api/health
    - ✓ Port: Traffic port
    - ✓ Healthy threshold: 5
    - ✓ Unhealthy threshold: 2
    - ✓ Timeout: 5 seconds
    - ✓ Interval: 30 seconds
    - ✓ Success codes: 200

4. Save changes



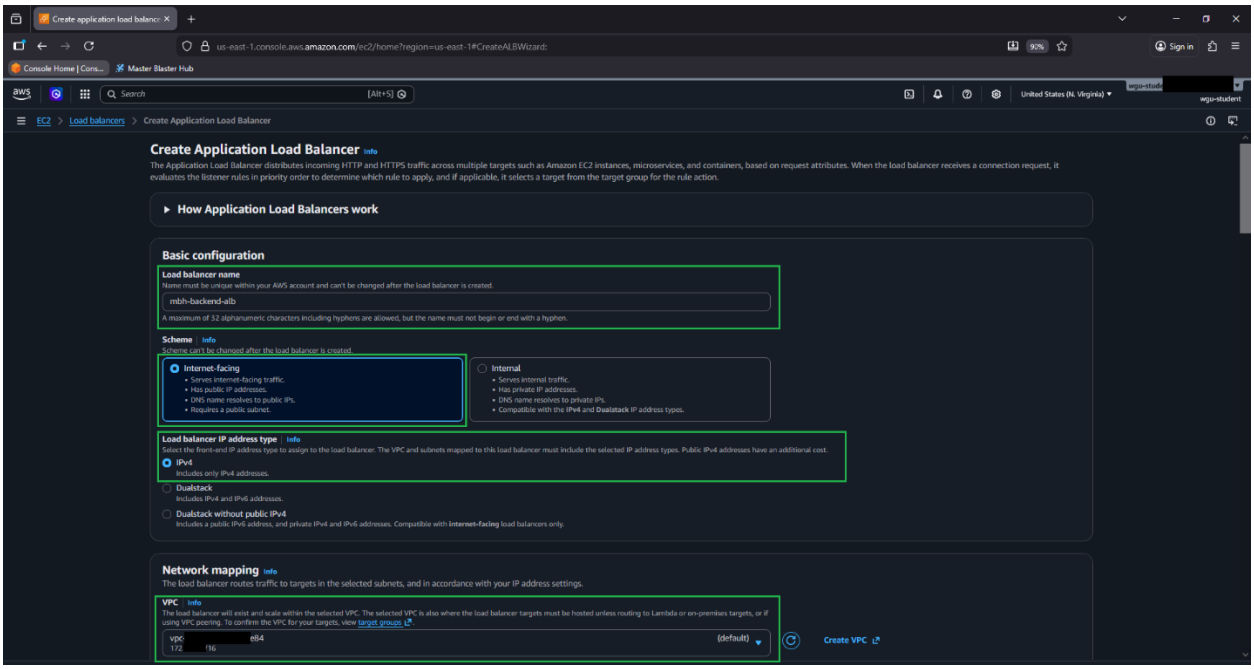


2.g.3 Create the Application Load Balancer

1. Navigate to **EC2 > Load Balancers**
2. Click **Create load balancer**
3. Select **Application Load Balancer**
4. Configure:
   - ✓ **Name:** mbh-backend-alb
   - ✓ **Scheme:** Internet-facing
   - ✓ **IP address type:** IPv4
   - ✓ **VPC:** vpc-0xxxxxxxxxxxe84
5. Under **Availability Zones**, select all available public subnets.
6. Under **Security groups**, select:
   - ✓ mbh-alb-sg
7. Under **Listeners**, create:
   - ✓ HTTP (80)
   - ✓ HTTPS (443)
8. For HTTPS:

- ✓ Attach ACM certificate for api.masterblasterhub.com
- ✓ Use ELBSecurityPolicy-TLS13-1-2-...

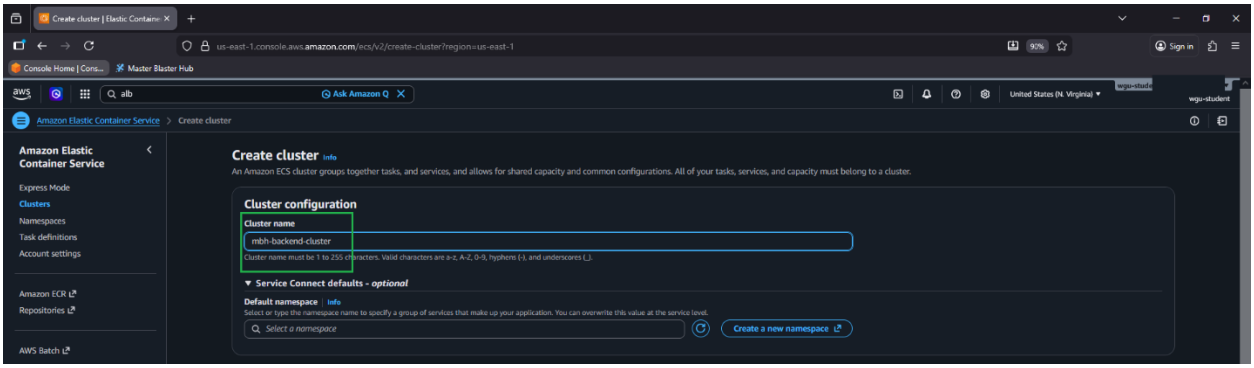
9. For default actions:
   - ✓ Forward to target group mbh-backend-tg
10. Click **Create load balancer**



2.g.4 Create an ECS Cluster

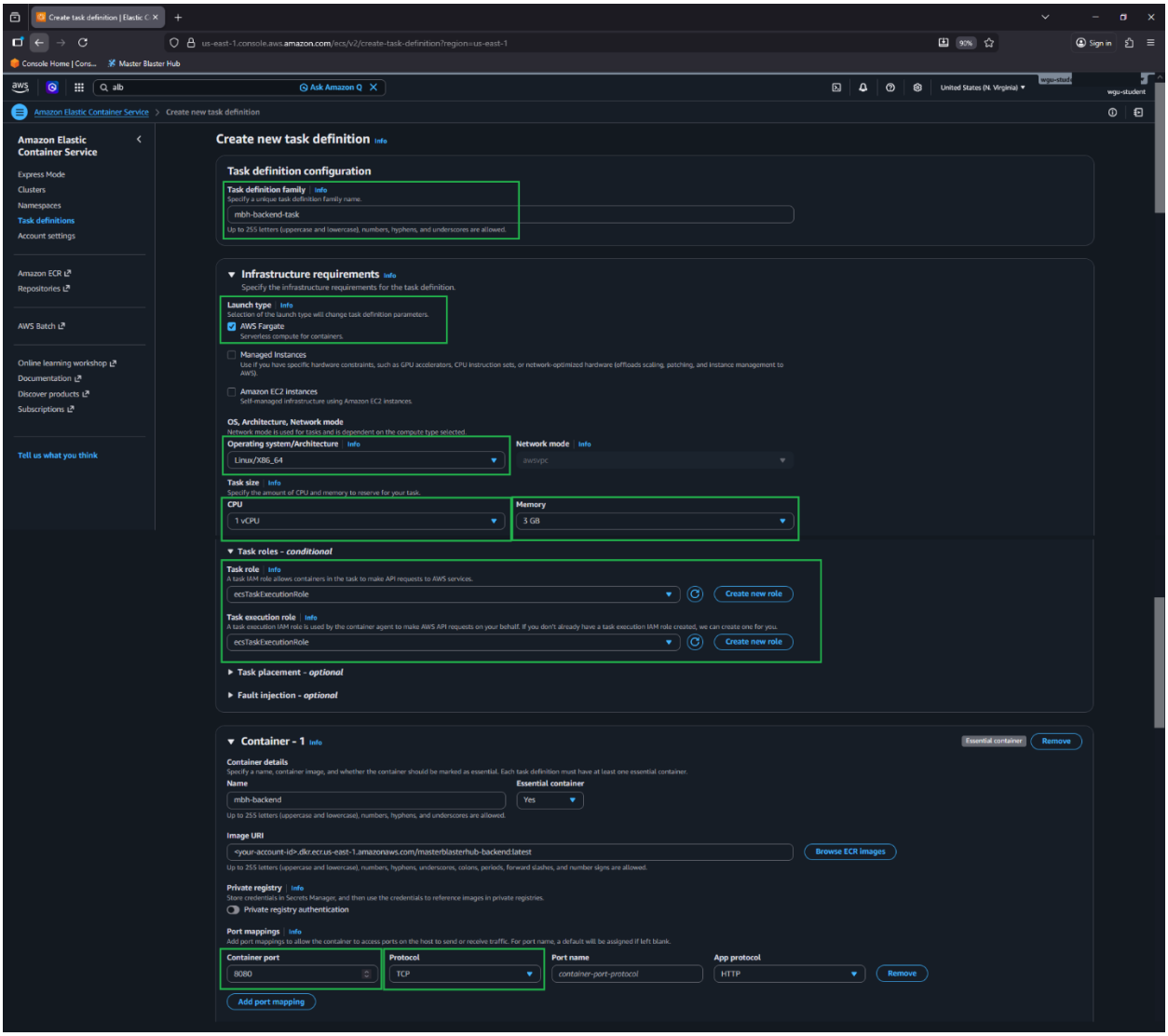
1. Navigate to **Amazon ECS > Clusters**
2. Click **Create Cluster**
3. Select **Networking only (Fargate)** or standard cluster creation
4. Enter:
   - ✓ Cluster name: mbh-backend-cluster
5. Leave monitoring as default
6. Click **Create**

Verify cluster status shows **Active**.



2.g.5 Create a Task Definition

1. Navigate to **Amazon ECS > Task Definitions**
2. Click **Create new Task Definition**
3. Select:
    - ✓ Launch type: Fargate
4. Configure:
    - ✓ Task Definition Name: mbh-backend-task
    - ✓ Operating System: Linux
    - ✓ CPU: 1 vCPU
    - ✓ Memory: 2 GB
    - ✓ Execution Role: ecsTaskExecutionRole
5. Under **Container Definitions**, click **Add container**
    - ✓ Container name: mbh-backend
    - ✓ Image URI: xxxxxxxxxxxx.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-backend:latest
    - ✓ Port mapping:
        - ✓ Container port: 8080
        - ✓ Protocol: TCP
6. Leave logging and advanced settings default
7. Click **Create**

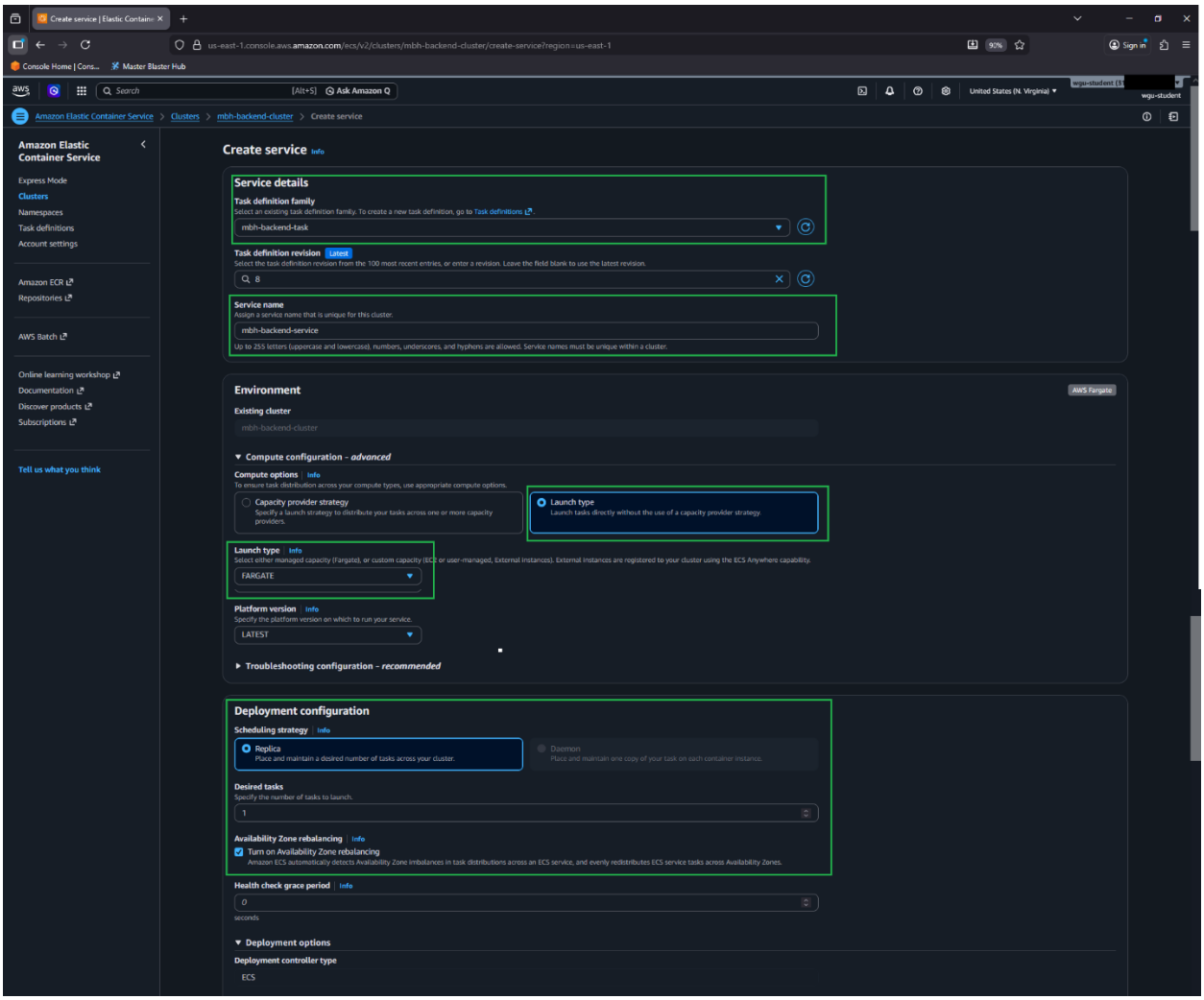


2.g.6 Create the ECS Service

1. Navigate to **ECS > Clusters > mbh-backend-cluster**
2. Select **Services > Create**
3. Configure:
   - ✓ Task definition: mbh-backend-task
   - ✓ Service name: mbh-backend-service
   - ✓ Launch type: AWS Fargate
4. Deployment configuration:
   - ✓ Scheduling strategy: Replica
   - ✓ Desired tasks: 1
   - ✓ Availability Zone rebalancing: Enabled
5. Networking:

- ✓ VPC: vpc-0xxxxxxxxxxxxae84
- ✓ Subnets: All subnets
- ✓ Security group: mbh-backend-sg
- ✓ Public IP: Turned on

6. Under **Load balancing**:
   - ✓ Enable load balancing
   - ✓ Load balancer type: Application Load Balancer
   - ✓ Load balancer: mbh-backend-alb
   - ✓ Target group: mbh-backend-tg
   - ✓ Container name: mbh-backend
   - ✓ Container port: 8080
7. Leave auto scaling disabled
8. Click **Create**



2.g.7 Verify Deployment

After deployment:

1. ECS > Cluster > Tasks
    - ✓ Confirm status: Running (1/1)
2. EC2 > Target Groups
    - ✓ Confirm 1 Healthy target
    - ✓ 0 Unhealthy targets
3. EC2 > Load Balancers
    - ✓ Confirm HTTP:80 > Forward to target group
    - ✓ Confirm HTTPS:443 > Forward to target group

At this point, the backend API should be accessible at:

https://api.masterblasterhub.com

2.g.8 Request Flow After Deployment

After the backend infrastructure has been successfully deployed and verified, client requests are processed through a defined sequence of AWS services before reaching the application container. When a user submits an HTTPS request to https://api.masterblasterhub.com, the domain name is resolved through DNS to the internet-facing Application Load Balancer (ALB).

The ALB terminates the TLS session using the associated certificate provisioned through AWS Certificate Manager (ACM). Once decrypted, the ALB evaluates its listener configuration and forwards the request to the configured target group. The target group distributes traffic to the registered Amazon ECS Fargate task using IP-based routing on port 8080.

Health checks are continuously performed against the /api/health endpoint to ensure that only healthy tasks receive traffic. If a task becomes unhealthy, it is automatically removed from rotation until it returns a successful health status. The Spring Boot application processes the request and returns the response through the same network path in reverse order.

This deployment architecture ensures secure TLS termination at the load balancer, restricted backend access through security group rules, and containerized application execution managed by Amazon ECS using the Fargate launch type.

2.h MongoDB Atlas Account and Cluster Setup

MongoDB Atlas is used as the managed cloud database service for persistent data storage. The backend application connects to MongoDB Atlas using a secure connection string stored as environment variables.

The cluster must be created before the backend application can successfully connect.

2.h.1 Create MongoDB Atlas Account

1. Navigate to:
   https://www.mongodb.com/cloud/atlas
2. Select **Try Free** or **Sign Up**.
3. Register using one of the following methods:
   - ✓ Email and password
   - ✓ Google account
   - ✓ GitHub account
4. Complete account verification if prompted.
5. Log in to the MongoDB Atlas dashboard.

Upon first login, Atlas may automatically create a default organization and prompt for project creation.

2.h.2 Create Organization

If an organization is not automatically created:

1. In the top-left corner, select the organization dropdown.
2. Click **New Organization**.
3. Enter:
   - ✓ Organization name: MasterBlasterHub
4. Click **Create Organization**.

The organization acts as a container for one or more projects.

2.h.3 Create Project

1. From the Atlas dashboard, select **New Project**.
2. Enter:
   - ✓ Project name: masterblasterhub
3. Click **Next**.
4. (Optional) Add additional members if required.
5. Click **Create Project**.

The project contains database clusters and related configuration.

2.h.4 Create Database Cluster

1. Within the project, select **Build a Database**.

2. Choose:
    - ✓ Deployment type: Shared (Free tier) or Dedicated (as required)
3. Select:
    - ✓ Cloud Provider: AWS
    - ✓ Region: us-east-1 (to match application deployment region)
4. Choose cluster tier:
    - ✓ M0 (Free tier) for development
5. Enter:
    - ✓ Cluster name: mbh-cluster
6. Click **Create Cluster**.

Cluster provisioning may take several minutes.

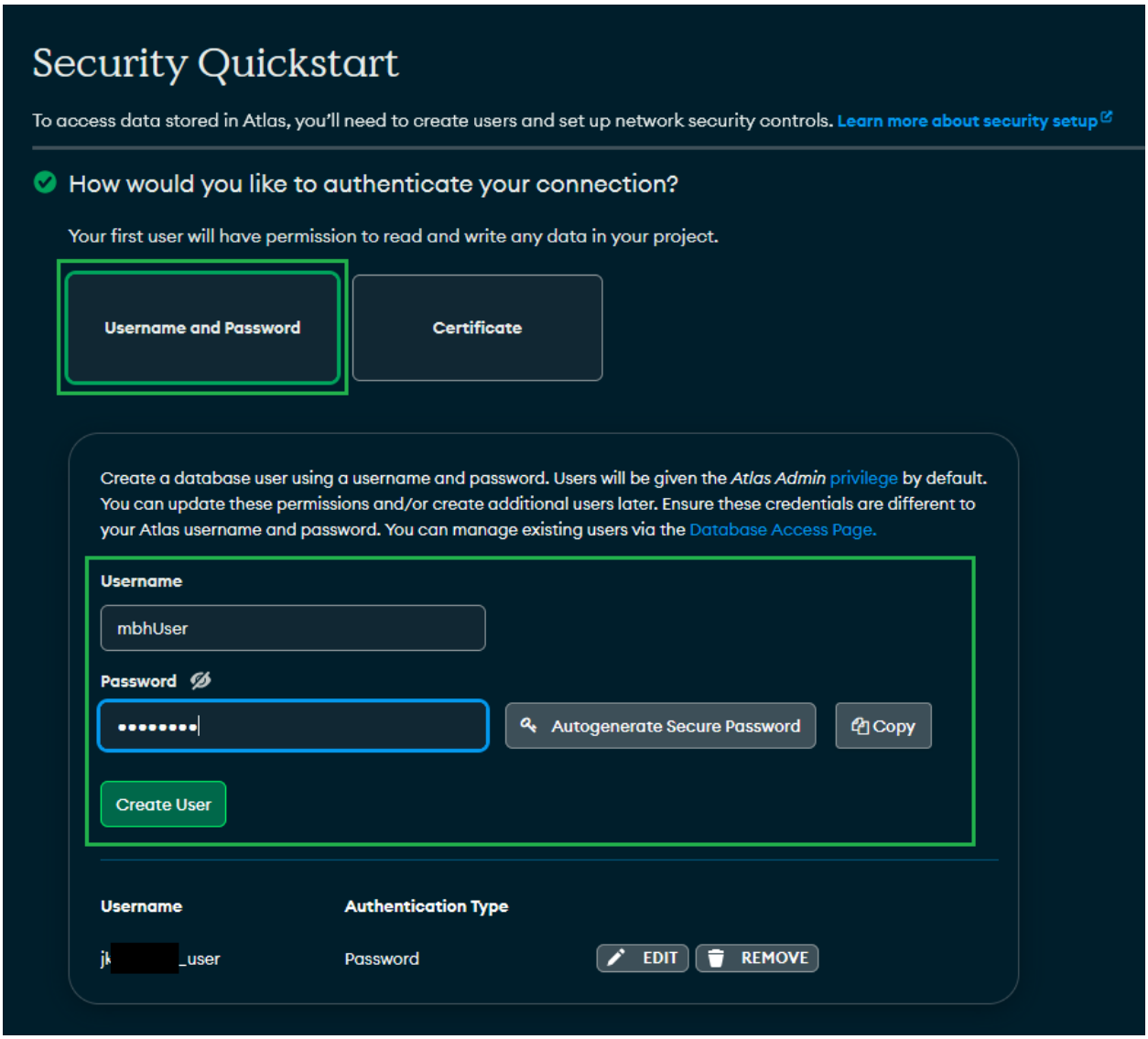
Once complete, cluster status will display as **Available**.

2.h.5 Configure Database Access (Create Database User)

A database user must be created before application connections are allowed.

1. Navigate to **Security > Database Access**.
2. Click **Add New Database User**.
3. Choose:
    - ✓ Authentication method: Password
4. Enter:
    - ✓ Username: mbhUser
    - ✓ Password: (generate secure password)
5. Under Database User Privileges:
    - ✓ Select **Read and write to any database**
6. Click **Add User**.

This username and password will be embedded in the MongoDB connection URI.



2.h.6 Configure Network Access (IP Access List)

MongoDB Atlas restricts connections by IP address.

1. Navigate to **Security > Network Access**.
2. Click **Add IP Address**.
3. Choose one of the following:

   For development: (not using for this project, but viable)

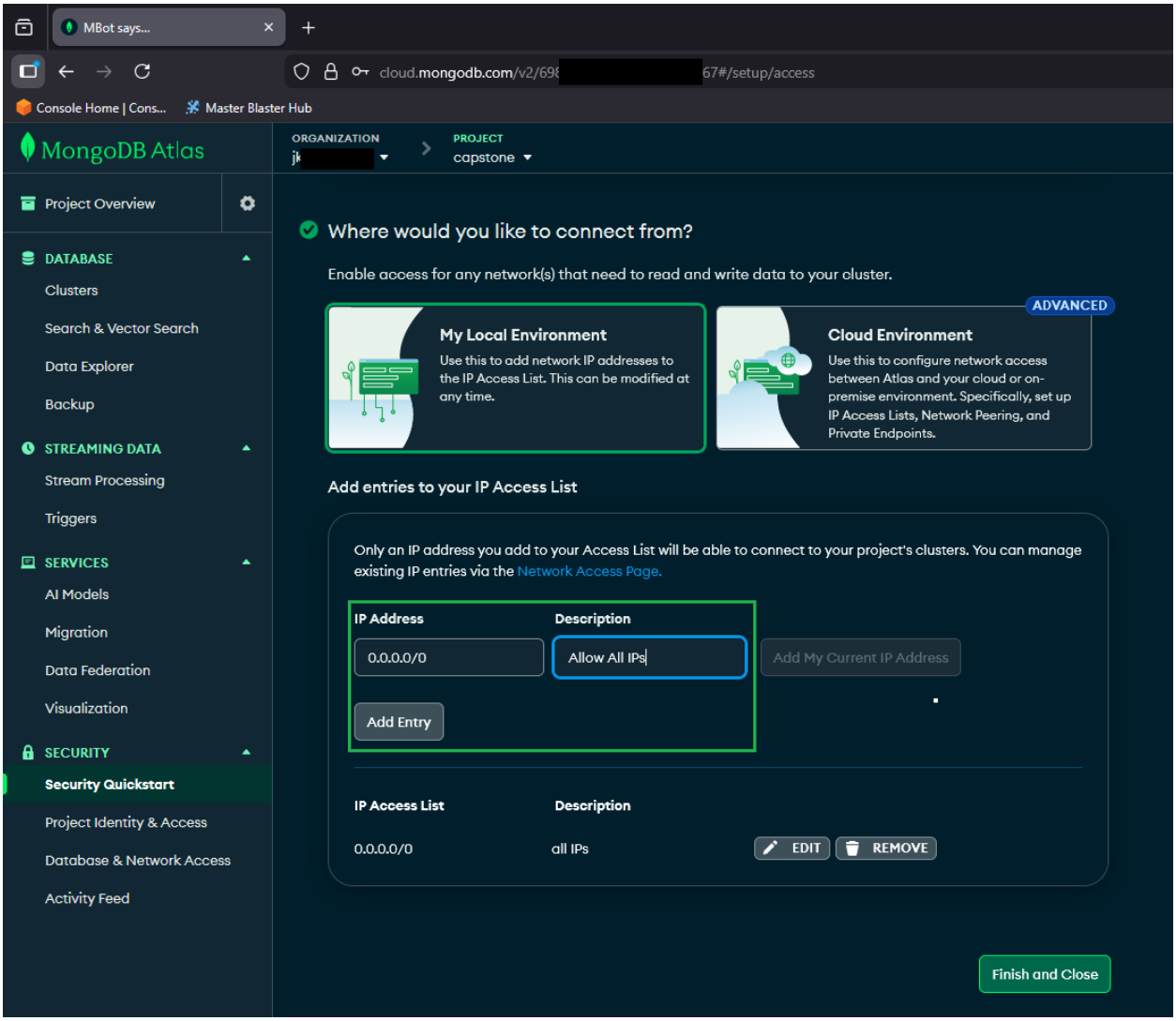
   ✓ Click **Add Current IP Address**

   For AWS ECS deployment: (use this selection)

   ✓ Select **Allow access from anywhere**

   ✓ Enter: 0.0.0.0/0
4. Click **Confirm**.

Allowing 0.0.0.0/0 permits connections from ECS Fargate containers, which use dynamic outbound IP addresses.



2.h.7 Obtain Connection String (URI)

1. Navigate to **Database > Clusters**.
2. Click **Connect** on the created cluster.
3. Select:
    - ✓ Connect your application
4. Choose:
    - ✓ Compass: no choice needed
    - ✓ Version: no choice needed

We are only concerned about viewing the connection string in this window.

5. Copy the provided connection string.

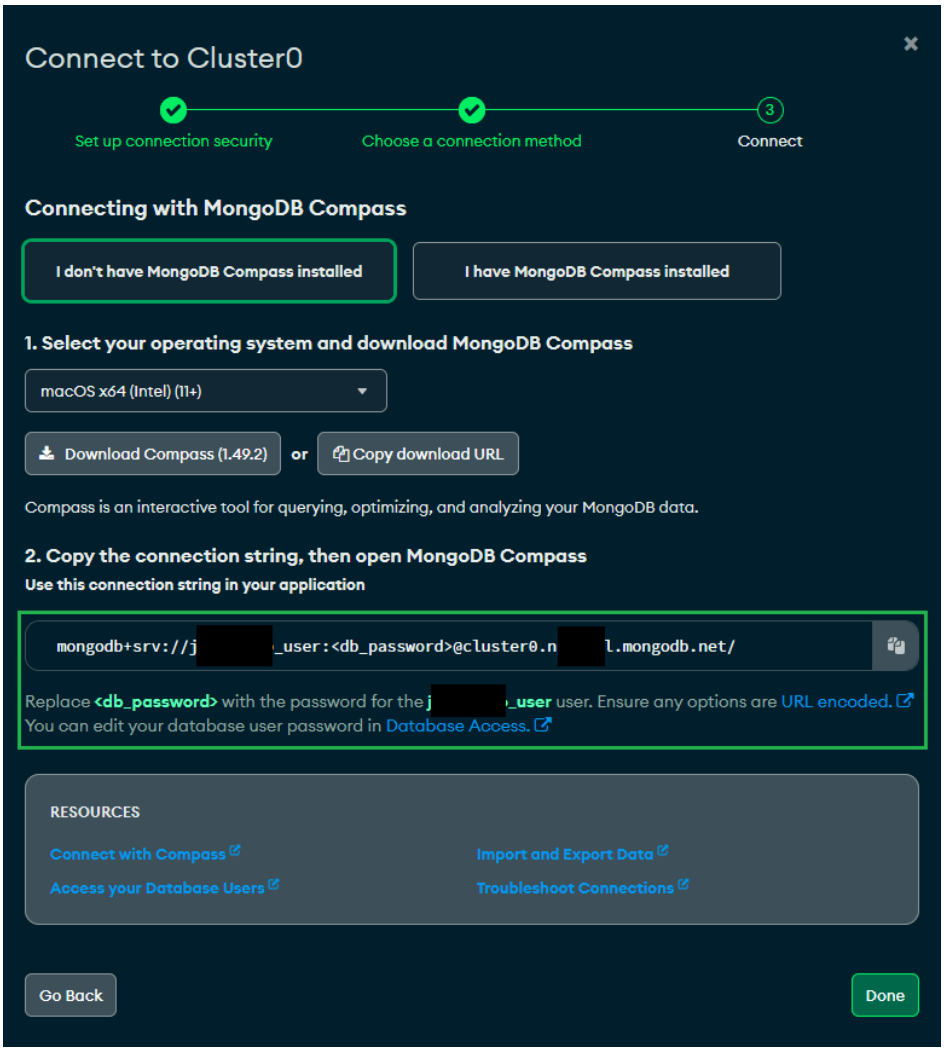
The connection string will appear similar to:
mongodb+srv://**<username>**:**<password>**@cluster0.nprtnel.mongodb.net/

Replace:

- ✓ **<username>** with mbhUser
- ✓ **<password>** with the database user password

You will be left with a string that resembles:
mongodb+srv://mbhUser:password@ cluster0.nprtnel.mongodb.net/

This will be URI connection string that will be used within the project code and environment variables which will be needed to connect. You will also need the database name: **pressurewash**



2.h.8 Configure Backend Environment Variables

The connection string must be stored as environment variables in the backend deployment environment.

The following variables are used in the Spring Boot application:

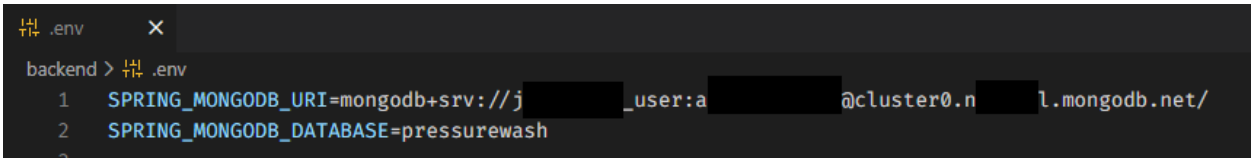
SPRING_MONGODB_URI
SPRING_MONGODB_DATABASE

These variables are configured:

- ✓ In local development via .env file
- ✓ In production via ECS task definition environment variables

The backend will use these values to establish a secure connection to MongoDB Atlas at runtime.

Locally, your MBH/backend/.env file should look like this:



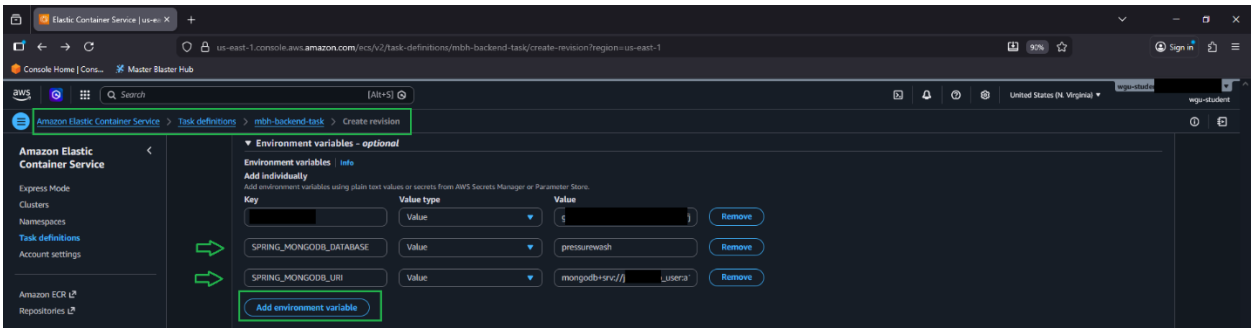
While here, let's go ahead and log back into AWS and set these values for the production environment.

2.h.8.a Update the ECS Task Definition via the AWS Console

1. Log in to the AWS Management Console.
2. Navigate to **Amazon ECS > Task Definitions**.
3. Select the task definition: **mbh-backend-task**
4. Click **Create new revision**.

Scroll down to the **Environment Variable** section and click **Add environment variable.** Enter both **Key:Value** pairs below.

Key: SPRING_MONGODB_DATABASE	Value: pressurewash
Key: SPRING_MONGODB_URI	Value: mongodb+srv://mbhUser:password@cluster0.nprtnel.mongodb.net/



5. Leave all other container settings unchanged.
6. Click **Create** to register the new task definition revision.

2.h.8.b Deploy Updated Task Definition

After creating the new revision:

1. Navigate to **Amazon ECS > Clusters**.
2. Select: **mbh-backend-cluster**
3. Open the **Services** tab.
4. Select: **mbh-backend-service**
5. From the **Deploy** dropdown arrow, Click **Update Service**.
6. Under Deployment configuration > Task definition revision, click the field and select the **Latest** option.
7. Leave all other settings unchanged.
8. Click **Update**.

ECS will perform a rolling deployment and replace the running task with a new task that includes the environment variables.

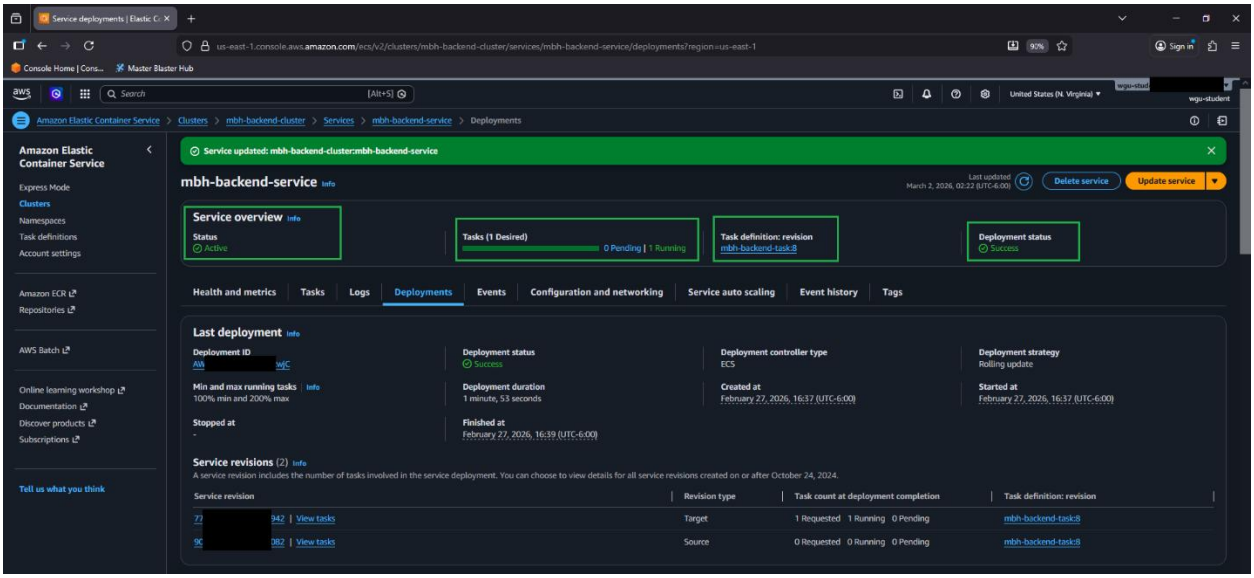
2.h.8.c Verify Deployment

After clicking **Update**, the console returns to the Service details page and displays a green banner indicating:

Service updated: mbh-backend-cluster:mbh-backend-service

Confirm the following:

- ✓ Service status: **Active**
- ✓ Deployment status: **Success**
- ✓ Tasks (1 Desired): **1 Running**
- ✓ Task definition revision reflects the newly created revision (for example, mbh-backend-task:8)



The backend container now reads **SPRING_MONGODB_URI** and **SPRING_MONGODB_DATABASE** directly from the ECS task environment and establishes a connection to MongoDB Atlas at startup.

2.i GitHub Repository, Secrets, and Workflows

GitHub is used for source code management and continuous integration/continuous deployment (CI/CD) automation. The repository stores both the frontend and backend code, along with workflow configuration files that automatically deploy the application when updates are pushed.

2.i.1 Create a GitHub Account

1. Navigate to:
   https://github.com
2. Select **Sign up**.
3. Enter:
   - ✓ Email address
   - ✓ Password
   - ✓ Username
4. Complete email verification.
5. Log in to the GitHub dashboard.

If using an existing account, ensure the account has administrative access to the target repository.

2.i.2 Clone the Repository

The project repository is hosted at:

https://github.com/jk377y/Master-Blaster-Hub

Install Git (If Not Already Installed)

1. Navigate to: https://git-scm.com/
2. Download and install Git.
3. Verify installation: **git –version** in your terminal

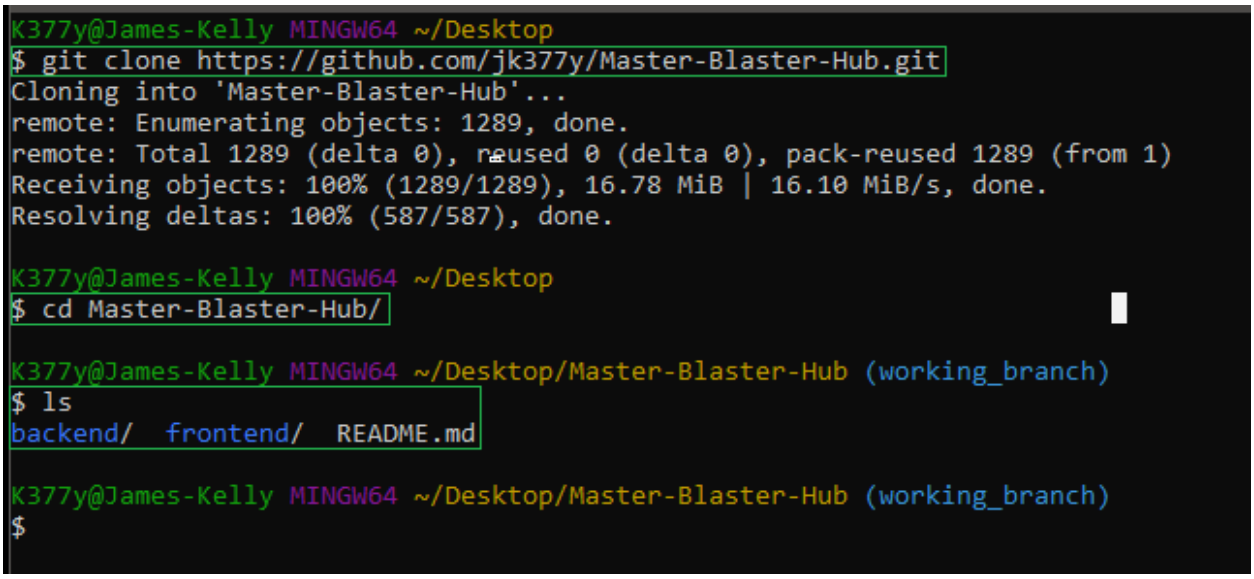
Clone the Repository Using HTTPS

1. Open a terminal or command prompt.
2. Navigate to the directory where you would like to store the project.
3. Execute the following command: git clone https://github.com/jk377y/Master-Blaster-Hub.git

After cloning completes,

navigate into the project directory: **cd Master-Blaster-Hub/**

verify root folder contents: **ls**



The repository will now exist locally on your machine and will include both the frontend and backend source code, along with the GitHub Actions workflow files used for automated deployment.

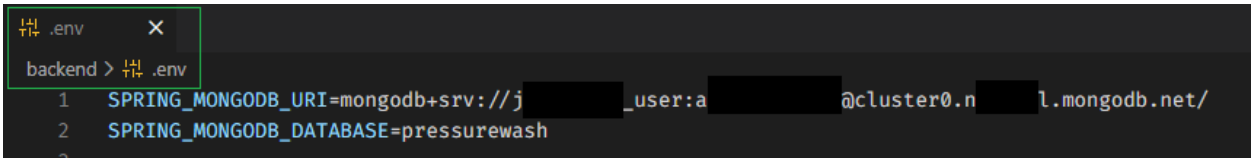
2.i.3 Configure Local Environment Variables

Environment variables are required for local development.

Backend Environment Variables

Create a file at: backend/.env

With the following data:



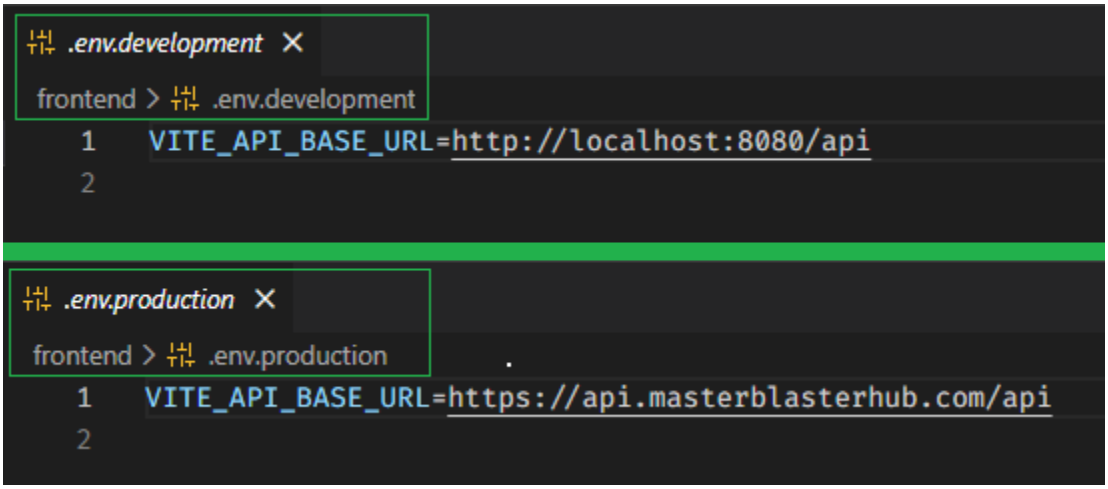
Use the uri and database values which you obtained in section "**2.h.7 Obtain Connection String (URI)**"

These variables are loaded by Spring Boot during local development.

Frontend Environment Variables

Create a file at: frontend/.env.development

With the following data:



These variables allow the frontend to target the correct backend API endpoint based on the running environment (local development or production).

2.i.4 Configure GitHub Repository Secrets

GitHub Actions workflows require secure credentials to deploy to AWS.

Navigate to Repository Secrets

1. Open the repository in GitHub.
2. Navigate to **Settings > Secrets and variables > Actions**
3. Select **New repository secret**.

Backend Secrets

These credentials correspond to the IAM user: **github-backend-deployer**
(obtained from **2.b.2 Backend Deployment IAM User**)

- ✓ **BACKEND_AWS_ACCESS_KEY_ID**
- ✓ **BACKEND_AWS_SECRET_ACCESS_KEY**

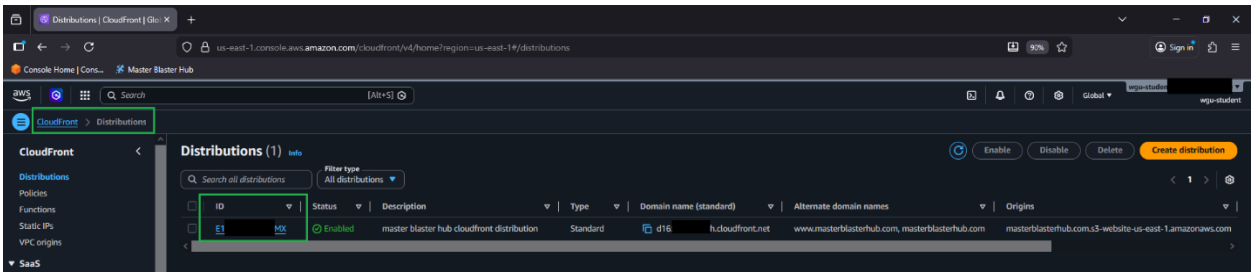
and

- ✓ **AWS_ACCOUNT_ID** (viewable on IAM Dashboard in the AWS console)

Frontend Secrets

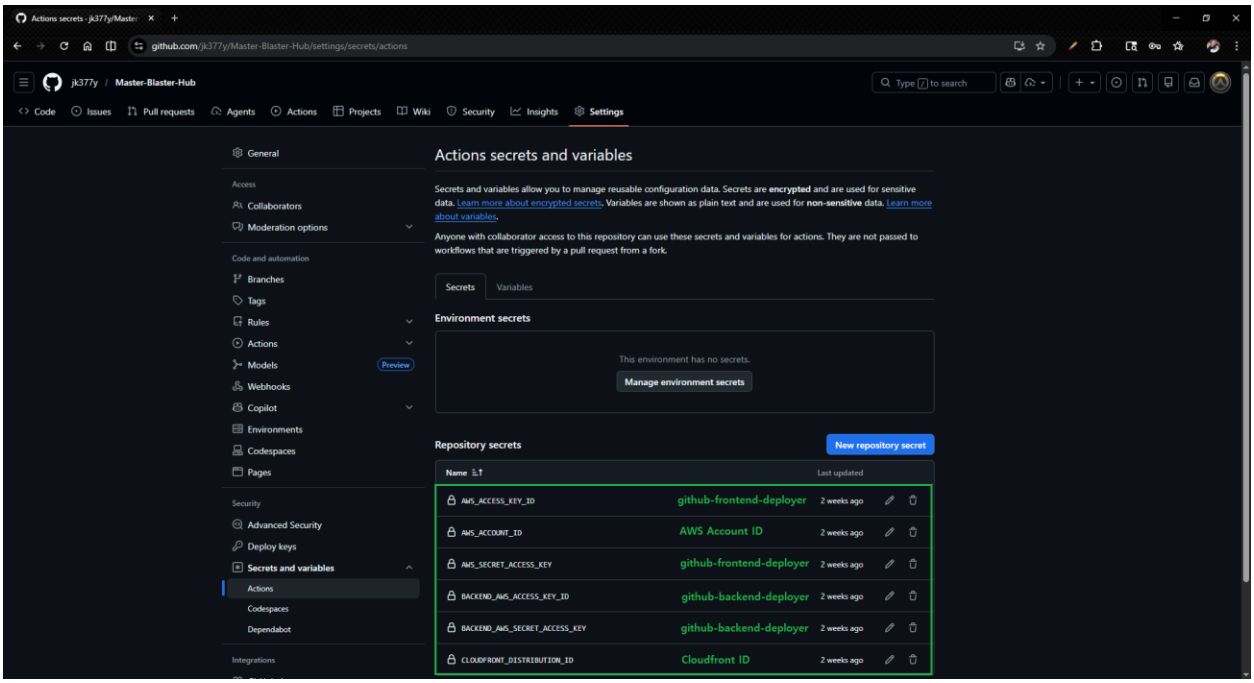
These credentials correspond to the IAM user: **github-frontend-deployer**
(obtained from **2.b.1 Frontend Deployment IAM User**)

- ✓ AWS_ACCESS_KEY_ID
- ✓ AWS_SECRET_ACCESS_KEY
- ✓ CLOUDFRONT_DISTRIBUTION_ID (found on AWS console; see image below)



These secrets are encrypted by GitHub and are not exposed in workflow logs.

The end result should look like this:



2.i.5 Enable GitHub Actions Workflows

The repository includes workflow files located at:

- ✓ .github/workflows/deploy-frontend.yml (see Appendix B)
- ✓ .github/workflows/deploy-backend.yml (see Appendix C)

GitHub automatically detects these files when pushed to the repository.

Workflow Trigger Conditions

Frontend workflow triggers when:

- ✓ Changes occur in frontend/**
- ✓ Push is made to working_branch

Backend workflow triggers when:

- ✓ Changes occur in backend/**
- ✓ Push is made to working_branch

2.i.6 CI/CD Deployment Process

Frontend Deployment Workflow

The frontend workflow performs:

- ✓ Install dependencies
- ✓ Build production bundle

- ✓ Authenticate to AWS
- ✓ Sync compiled assets to S3
- ✓ Invalidate CloudFront cache

This results in automatic frontend deployment.

Backend Deployment Workflow

The backend workflow performs:

- ✓ Authenticate to AWS
- ✓ Log in to Amazon ECR
- ✓ Build Docker image
- ✓ Tag image with ECR repository URI
- ✓ Push image to ECR
- ✓ Force ECS service redeployment

This results in automatic container update in ECS.

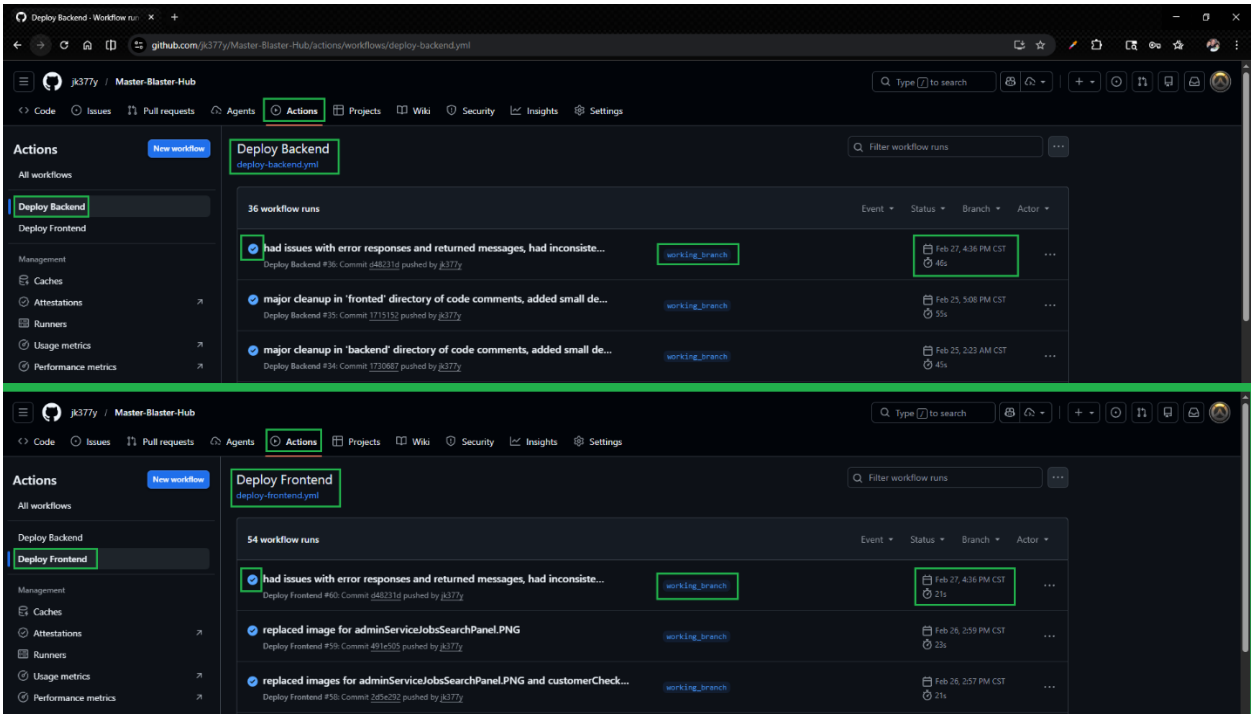
2.i.7 Verify Workflow Execution

To confirm CI/CD automation:

1. Navigate to the repository in GitHub.
2. Select the **Actions** tab.
3. Confirm:
    - ✓ Workflow run status: **Success**
    - ✓ All steps complete without errors

Successful execution confirms that:

- ✓ The frontend is deployed to S3 and distributed via CloudFront.
- ✓ The backend image is pushed to ECR and redeployed in ECS.



2.j Local Development Environment Setup

2.j.1 Node.js Installation

Node.js is required for frontend development, dependency management, and production build generation. The project uses Node Package Manager (npm) to install dependencies and execute build scripts for the React frontend.

The latest **Long-Term Support (LTS)** version of Node.js is recommended to ensure compatibility and stability.

Install Node.js

1. Navigate to: https://nodejs.org/
2. Download the **LTS version** appropriate for your operating system.
3. Run the installer and accept the default configuration options unless customization is required.
4. Complete the installation.

Verify Installation

After installation:

1. Open a terminal or command prompt.
2. Run the following command**: node --version**

A version number should be displayed (for example, v20.x.x or later).

3. Verify npm installation: **npm --version**

A version number should be returned.

Successful output from both commands confirms that Node.js and npm are correctly installed and available in the system PATH.



2.j.2 Java Development Kit (JDK)

The Java Development Kit (JDK) is required to build and run the Spring Boot backend application. The backend is developed using Java and must be compiled before execution or containerization.

This project is configured for:

Java Version 21

A JDK version compatible with Java 21 must be installed.

Install Java Development Kit (JDK)

1. Navigate to one of the following official sources:
   - ✓ Oracle JDK: https://www.oracle.com/java/technologies/downloads/
   - ✓ OpenJDK distribution (recommended): https://adoptium.net/
2. Select:
   - ✓ Version 21
   - ✓ The appropriate operating system
   - ✓ Installer package format (MSI, PKG, or equivalent)
3. Download and run the installer.
4. Complete installation using default settings unless customization is required.

Verify Installation

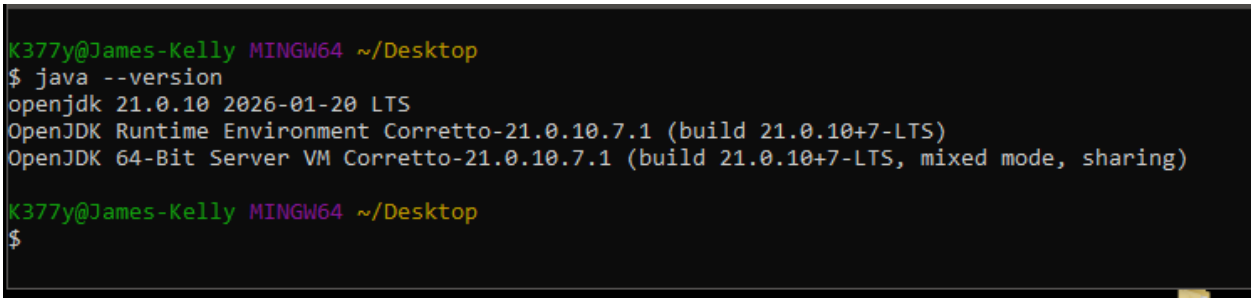
After installation:

1. Open a terminal or command prompt.

2. Run the following command: **java --version**

The output should display:

- ✓ Java version 21
- ✓ Vendor information
- ✓ Runtime environment details



2.j.3 Apache Maven

Apache Maven is required for managing the backend build lifecycle. It handles dependency resolution, compilation, testing, and packaging of the Spring Boot application.

This project uses Maven to:

- ✓ Download required dependencies
- ✓ Compile Java source code
- ✓ Package the backend into an executable JAR file
- ✓ Prepare the application for containerization

Install Apache Maven

1. Navigate to the official Maven website:

https://maven.apache.org/download.cgi

2. Download the latest stable binary ZIP archive.
3. Extract the contents to a permanent directory (for example: C:\Program Files\Apache\Maven or /usr/local/apache-maven).
4. Configure environment variables:
   - ✓ Create a system variable named MAVEN_HOME pointing to the Maven installation directory.
   - ✓ Add %MAVEN_HOME%\bin (Windows) or $MAVEN_HOME/bin (macOS/Linux) to the system PATH.
5. Save changes and restart the terminal or command prompt.

Verify Installation

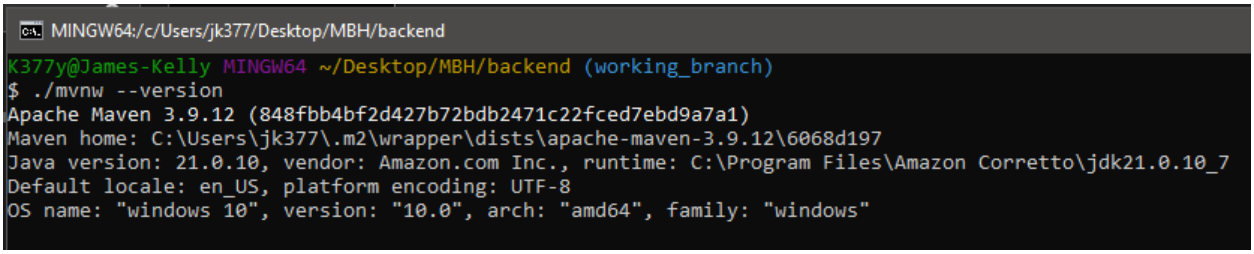
After installation:

1. Open a terminal or command prompt.
2. Run: **mvn --version**

The output should display:

- Apache Maven version
- Java version (should reflect JDK 21)
- Operating system information

Note: This image demonstrates Maven execution from the **backend** project directory. Maven is installed locally on the system and accessible within this directory.



2.j.4 Docker Desktop

Docker Desktop is required to build and test container images locally before pushing them to Amazon Elastic Container Registry (ECR).

This project uses Docker to containerize the Spring Boot backend application prior to deployment to Amazon ECS (Fargate).

Install Docker Desktop

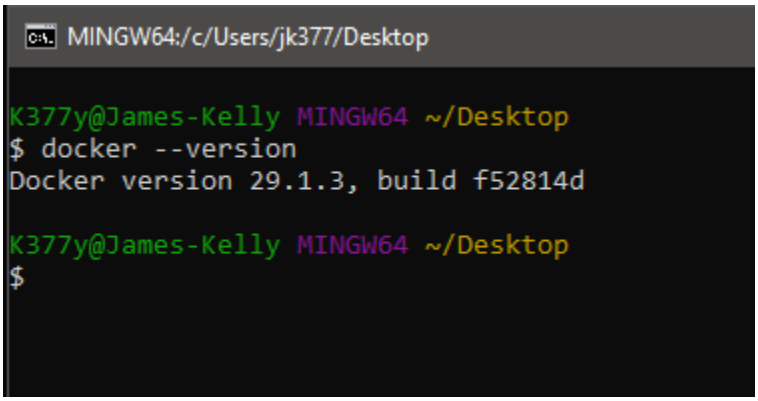
1. Navigate to the official Docker website: https://www.docker.com/products/docker-desktop/
2. Download Docker Desktop for your operating system.
3. Run the installer.
4. During installation:
   - ✓ Accept default configuration settings unless customization is required.
   - ✓ If prompted, enable WSL 2 integration (Windows users).
5. Complete installation and restart the system if required.

Verify Installation

After Docker Desktop is running:

1. Open a terminal or command prompt.
2. Run the following command: **docker --version**

The output should display the installed Docker version.



2.k Summary of Access Permissions

The individual performing the system setup and deployment must have appropriate administrative privileges across all required platforms. Insufficient permissions will prevent successful configuration of cloud infrastructure, CI/CD automation, and database connectivity.

The following access levels are required.

2.k.1 AWS Account Permissions

The setup administrator must have full administrative privileges within the AWS account used for deployment.

Minimum required capabilities include the ability to:

- ✓ Create and manage Amazon ECS clusters and services
- ✓ Create and manage Amazon ECR repositories
- ✓ Create and manage Application Load Balancers
- ✓ Create and modify Target Groups
- ✓ Create and modify Security Groups
- ✓ Manage IAM roles and policies
- ✓ Create and manage Amazon S3 buckets
- ✓ Configure CloudFront distributions
- ✓ Modify Route 53 DNS records
- ✓ Request and validate ACM certificates

AWS IAM Deployment Roles

While administrative privileges are required during initial infrastructure provisioning, deployment automation is executed using two dedicated IAM users with scoped permissions:

- ✓ **github-frontend-deployer**

- ✓ **github-backend-deployer**

These IAM users were created specifically to support GitHub Actions workflows and follow the principle of least privilege.

The **github-frontend-deployer** user has permissions to:

- ✓ Upload and manage objects in the S3 frontend bucket
- ✓ Create CloudFront invalidations

The **github-backend-deployer** user has permissions to:

- ✓ Authenticate with Amazon ECR
- ✓ Push container images
- ✓ Update Amazon ECS services
- ✓ Pass required IAM roles for ECS task execution

These scoped IAM users provide sufficient permissions to manage automated deployments without granting full administrative access.

2.k.2 GitHub Repository Permissions

Administrative access to the GitHub repository is required to:

- ✓ Clone the repository
- ✓ Configure repository secrets (for AWS credentials and deployment variables)
- ✓ Create and modify GitHub Actions workflows
- ✓ Enable GitHub Actions within repository settings
- ✓ Approve and manage workflow executions

Minimum required role:

- ✓ Repository Owner or Admin

Without repository administrative access, CI/CD automation cannot be configured.

2.k.3 MongoDB Atlas Permissions

Administrative access to the MongoDB Atlas organization and project is required to:

- ✓ Create the organization
- ✓ Create the project
- ✓ Provision the database cluster
- ✓ Configure network access rules (IP access list)
- ✓ Create database users
- ✓ Retrieve the connection string (MongoDB URI)

Required role:

- ✓ Organization Owner or Project Owner

Without appropriate Atlas permissions, the backend application cannot connect to the database.

2.k.4 Domain and DNS Permissions

The setup administrator must have DNS modification privileges for the registered domain.

This is required to:

- ✓ Create or modify DNS records
- ✓ Configure Route 53 hosted zones
- ✓ Point domain records to CloudFront and the Application Load Balancer
- ✓ Validate ACM certificates via DNS

Required access:

- ✓ Domain registrar administrative access
  or
- ✓ Route 53 hosted zone administrative access

Without DNS modification privileges, the application cannot be publicly accessible via custom domain names.

3. Application Execution and Validation

3.a Running the Backend Application Locally

After completing all prerequisite account configuration, environment variable setup, and local software installation, the backend application may be executed locally for validation and development purposes.

Running the backend locally confirms that:

- ✓ The Java Development Kit (JDK) is installed correctly
- ✓ Maven is configured properly
- ✓ Environment variables are accessible
- ✓ The MongoDB Atlas connection is functioning
- ✓ The Spring Boot application starts without errors

3.a.1 Navigate to Backend Directory

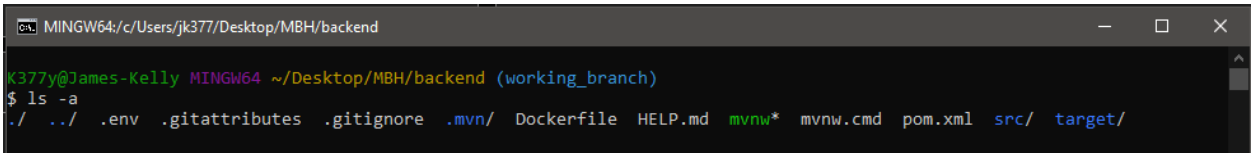
Open a terminal or command prompt.

From the backend/, run: **ls -a**

Confirm that the directory contains:

- ✓ pom.xml
- ✓ src/

- ✓ Dockerfile
- ✓ .env (created in Section 2.i.3)



3.a.2 Start the Spring Boot Application

The backend may be started using the Maven Wrapper included in the project.

If using the Maven Wrapper (recommended for consistency):

From backend/, run:

- ✓ Windows (Command Prompt): **mvnw.cmd spring-boot:run**
- ✓ Windows (Git Bash): **./mvnw spring-boot:run**
- ✓ macOS/Linux: **./mvnw spring-boot:run**

Maven will:

- ✓ Download required dependencies (if not already cached)
- ✓ Compile the Java source code
- ✓ Initialize the Spring Application Context
- ✓ Establish the MongoDB connection
- ✓ Start the embedded Tomcat web server
- ✓ Launch the Spring Boot application on port 8080

3.a.3 Confirm Successful Startup

When the application starts successfully, the console output should indicate:

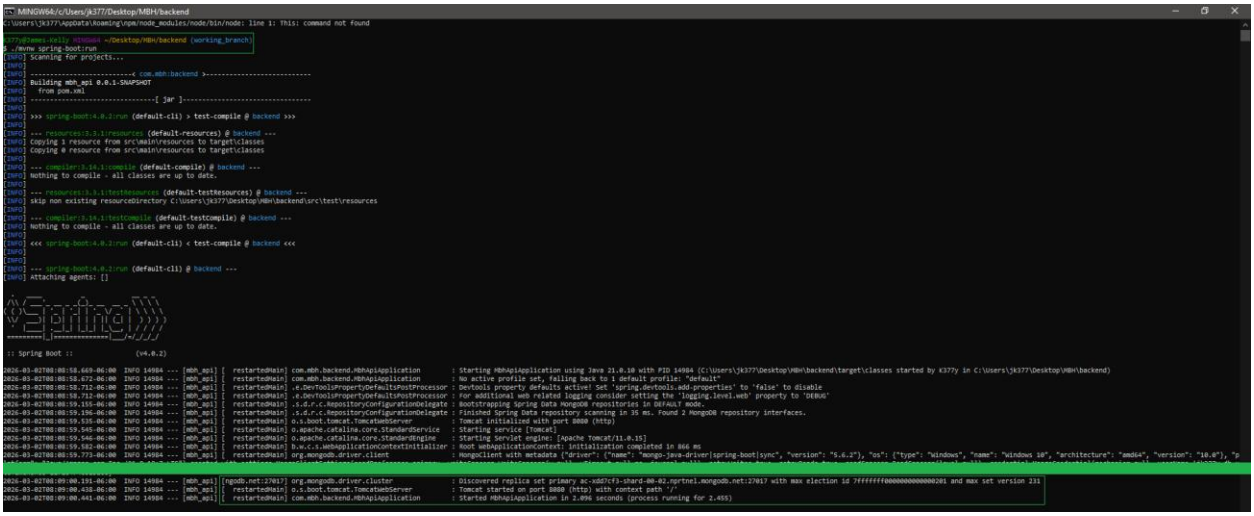
- ✓ Spring Boot startup banner
- ✓ Application running on port 8080
- ✓ No startup errors
- ✓ Successful MongoDB connection

Look for a message similar to:

Tomcat started on port(s): 8080 (http)

Started BackendApplication in X seconds

If environment variables are configured correctly, no database connection errors should appear.



3.a.4 Verify Health Endpoint

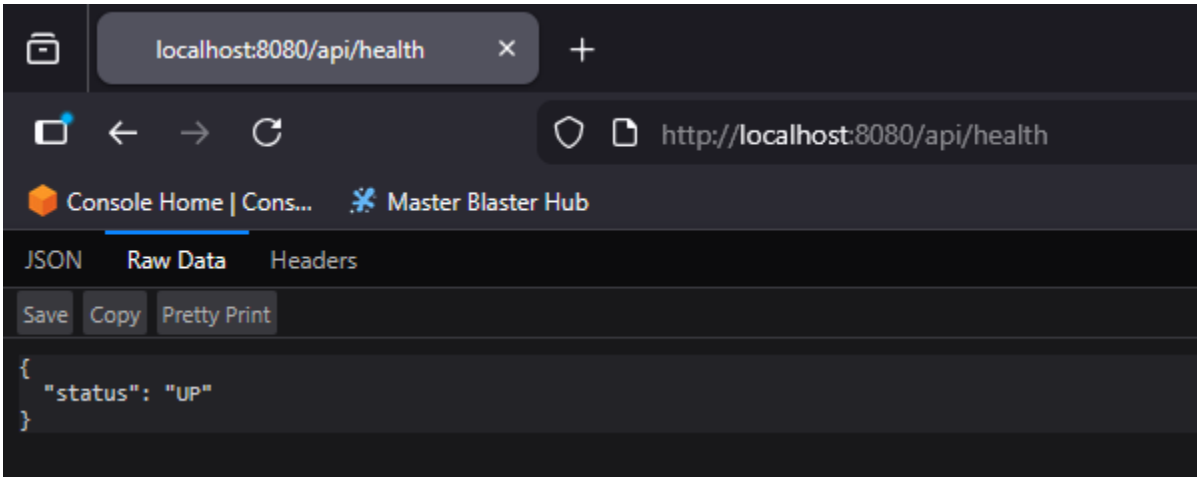
Open a web browser and navigate to: **http://localhost:8080/api/health**

A successful response should return a JSON object similar to:

```
{
 "status": "UP"
}
```

This confirms:

- ✓ The backend server is running
- ✓ The application context loaded successfully
- ✓ The health endpoint is reachable
- ✓ Local networking is functioning correctly



3.a.5 Build and Run Docker Image Locally

In production, the backend application runs as a Docker container within Amazon ECS (Fargate). Before pushing container images to Amazon ECR, the image may be built and validated locally.

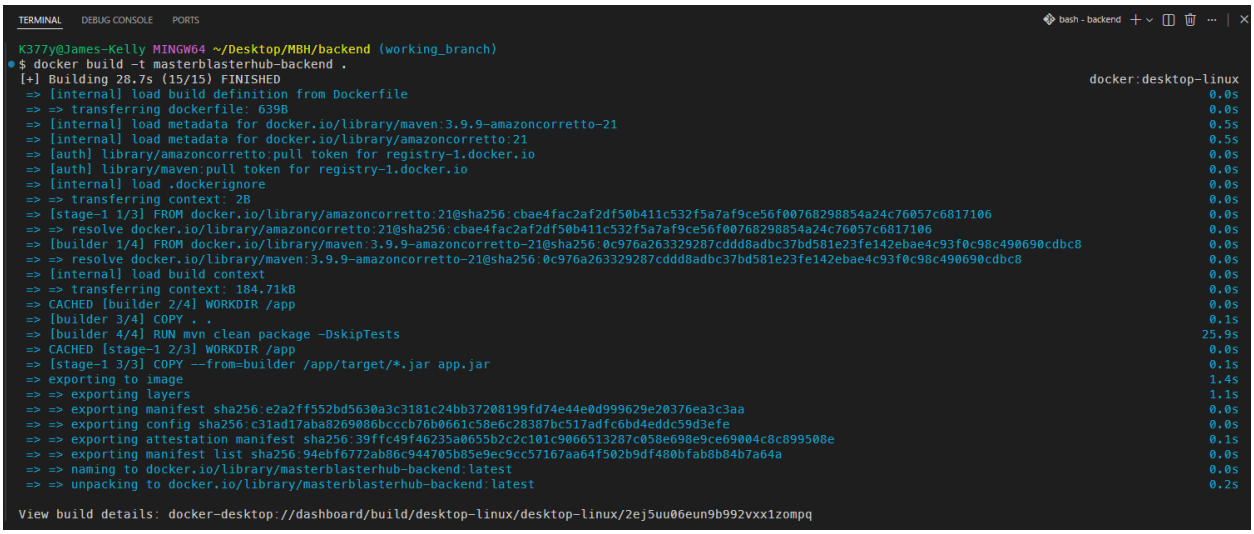
Build the Docker Image

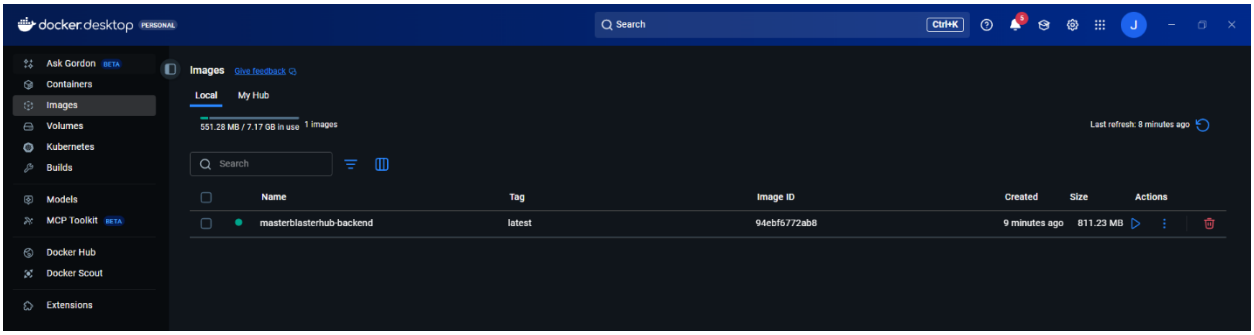
From the backend/, run: **docker build -t masterblasterhub-backend .**

Docker will:

- ✓ Use the multi-stage Dockerfile
- ✓ Build the Spring Boot application
- ✓ Package it into a lightweight runtime image
- ✓ Tag the image as masterblasterhub-backend

Successful completion confirms the Dockerfile is valid.

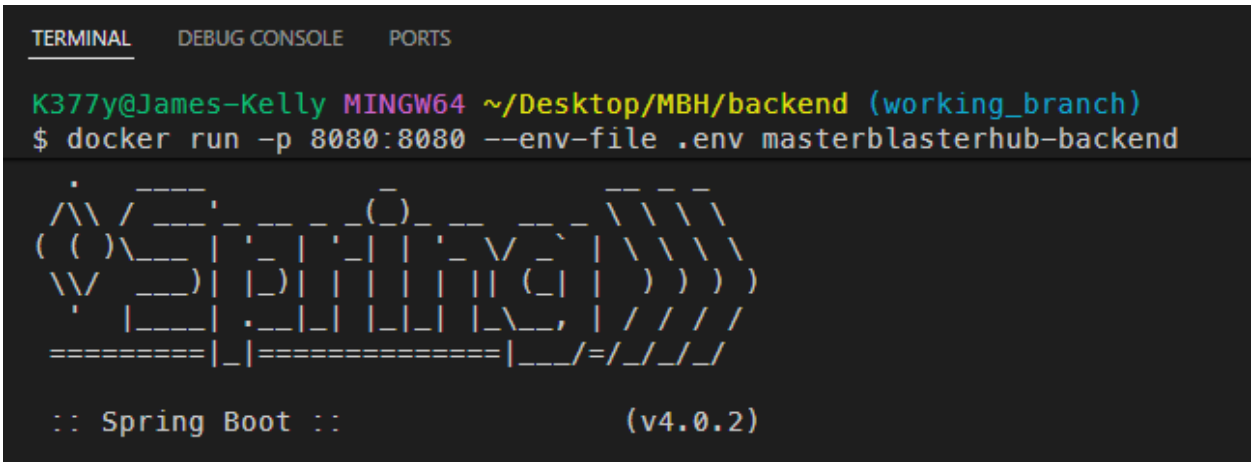




To run the Docker image locally, you need to load the environment variables. From the backend/, run:

docker run -p 8080:8080 --env-file .env masterblasterhub-backend

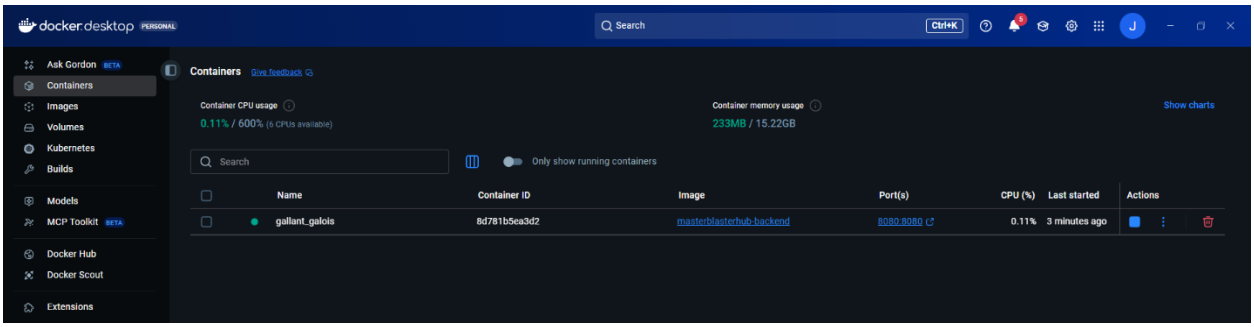
You will see the familiar Spring Boot application startup prompts.



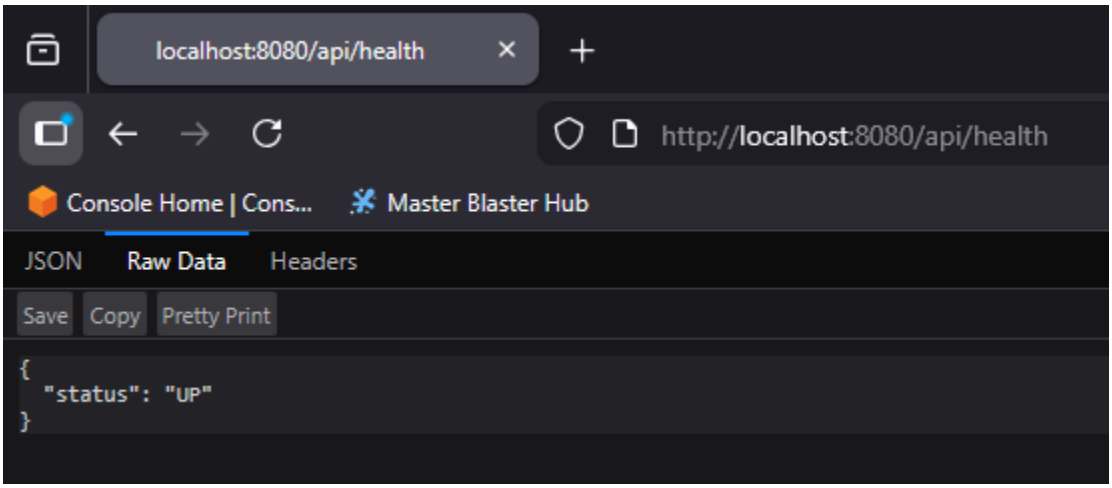
This command:

- ✓ Creates and starts a new container
- ✓ Maps container port 8080 to local port 8080
- ✓ Loads environment variables from backend/.env
- ✓ Launches the Spring Boot application inside the container

Docker Desktop will display the running container, but no manual start action is required.



Once it is online, you will be able to test the backend api enpoint again.



3.a.6 Expected Local Backend Behavior

When running via Maven:

- ✓ The application executes directly on the host machine
- ✓ Environment variables are loaded via Spring Boot configuration
- ✓ Tomcat runs on port 8080

When running via Docker:

- ✓ The application executes inside an isolated container
- ✓ Environment variables are injected using --env-file
- ✓ Port 8080 is exposed via container port mapping
- ✓ Docker Desktop displays container status

In both scenarios:

- ✓ The /api/health endpoint returns status UP
- ✓ MongoDB Atlas connection is established
- ✓ No startup exceptions occur

At this stage, the backend is fully operational in the local development environment and ready to be integrated with the frontend application.

3.b Running the Frontend Application Locally

After verifying that the backend application executes correctly (both via Maven and Docker), the frontend application may be started in local development mode.

Running the frontend locally confirms that:

- ✓ Node.js and npm are installed correctly
- ✓ Project dependencies install without error
- ✓ Vite development server starts successfully
- ✓ Environment variables are loaded properly
- ✓ Frontend-to-backend communication is functioning

3.b.1 Navigate to Frontend Directory

Open a terminal or command prompt.

From the frontend/, run: **ls -a**

Confirm the directory contains:

- ✓ package.json
- ✓ vite.config.js
- ✓ src/
- ✓ .env.development

The presence of package.json confirms that the Node project is properly initialized.

```
K377y@James-Kelly MINGW64 ~/Desktop/MBH/frontend (working_branch)
$ ls -a
./   .env.development  .gitignore  eslint.config.js  node_modules/  package-lock.json  src/
../  .env.production   dist/       index.html        package.json   public/            vite.config.js
```

3.b.2 Install Frontend Dependencies

From frontend/, run: **npm install**

This command:

- ✓ Reads dependency definitions from package.json
- ✓ Downloads required packages into the node_modules directory
- ✓ Generates or updates package-lock.json

Successful completion confirms:

- ✓ Node.js is installed
- ✓ npm is functioning
- ✓ Project dependencies resolve without conflicts

No error messages should appear during installation.

```
K377y@James-Kelly MINGW64 ~/Desktop/MBH/frontend (working_branch)
$ npm install

added 162 packages, and audited 163 packages in 15s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

3.b.3 Start the Vite Development Server

From frontend/, run: **npm run dev**

Vite will:

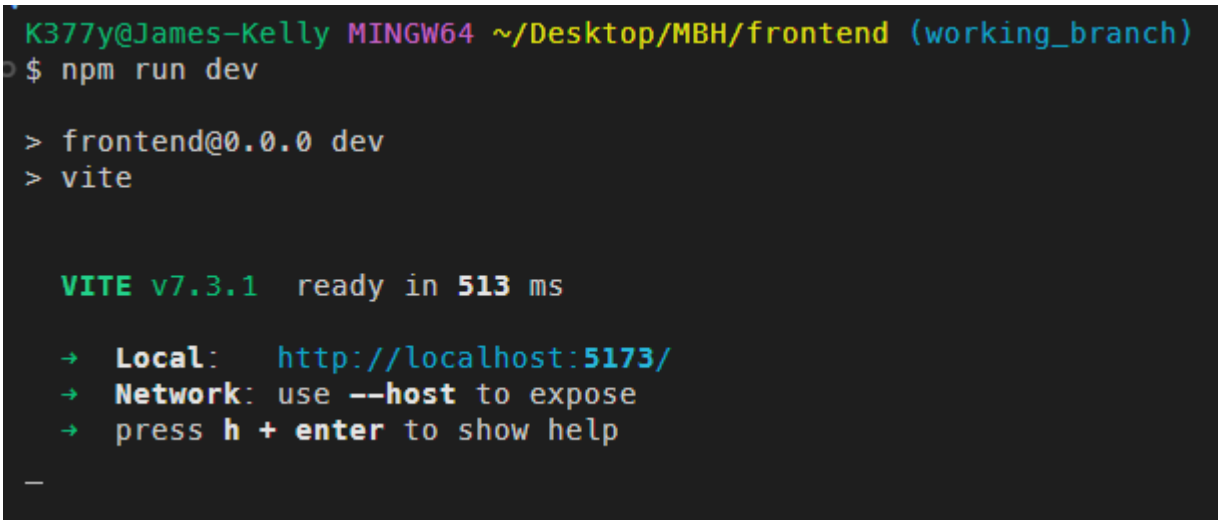
- ✓ Compile the React application
- ✓ Start a development server
- ✓ Enable hot module replacement (HMR)

- ✓ Display a local access URL

The terminal should display output similar to:

- ✓ Local: http://localhost:5173/ (most likely for this setup)
- ✓ Network: http://192.xxx.x.x:5173/

This confirms that the development server is running.



3.b.4 Verify Frontend Accessibility

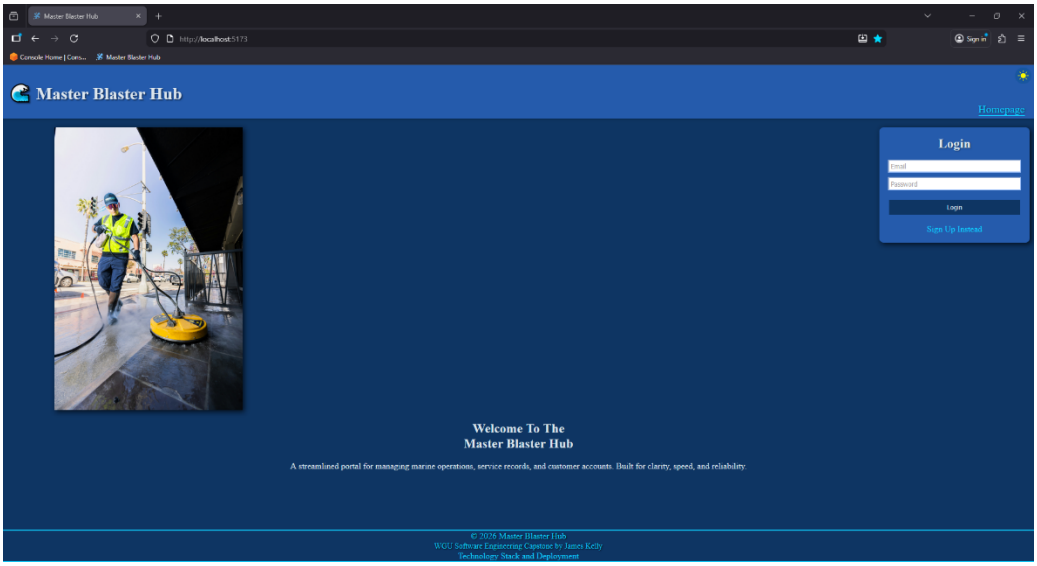
Open a web browser and navigate to: http://localhost:5173

The application homepage should load successfully.

Confirm:

- ✓ No console errors appear in the browser developer tools
- ✓ Theme toggle functions
- ✓ Login and Signup screens render correctly

This verifies that the frontend build process and local development server are functioning properly.



3.b.5 Confirm API Communication with Local Backend

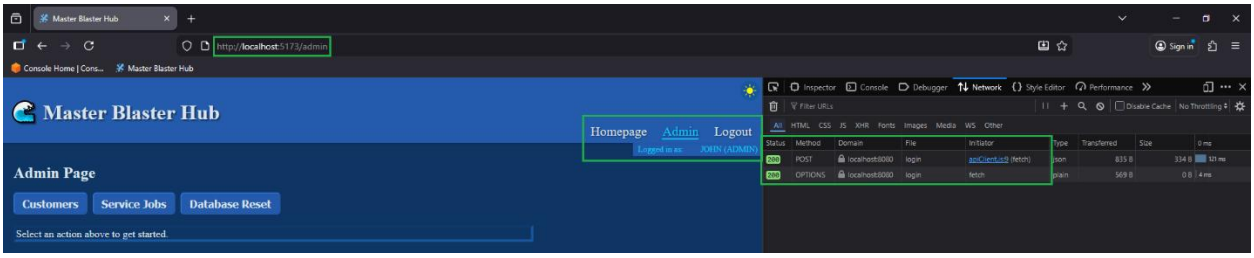
The frontend environment variable VITE_API_BASE_URL (defined in frontend/.env.development) should be set to:
http://localhost:8080/api

To validate frontend-to-backend communication:

- ✓ Ensure the backend application is running on port 8080.
- ✓ From the frontend application, perform an action that triggers an API request (for example, login or accessing a protected route).
- ✓ Open the browser developer tools, go to the Network tab.

  Confirm:

  - ✓ Requests are sent to: **http://localhost:8080/api/...**
  - ✓ Responses return status 200 (or expected status codes).
  - ✓ No CORS errors occur.



This confirms:

- ✓ Environment variables are loaded correctly by Vite
- ✓ The frontend is targeting the correct backend endpoint
- ✓ Cross-origin configuration is properly implemented

- ✓ Backend and frontend services are communicating successfully

3.b.6 Expected Local Frontend Behavior

When running locally:

- ✓ The frontend development server listens on port 5173 (default Vite port).
- ✓ API requests are routed to http://localhost:8080/api.
- ✓ Changes to frontend source files trigger automatic browser reload via hot module replacement.
- ✓ Authentication and protected routes function when the backend is running.

At this stage, the full application stack is operational locally:

- ✓ MongoDB Atlas (cloud database)
- ✓ Spring Boot backend (local or Docker container)
- ✓ React frontend via Vite development server

Successful execution confirms that the development environment is correctly configured and that both application layers integrate as designed prior to cloud deployment.

3.c Production Environment Deployment Validation

After confirming that the application runs successfully in the local development environment, the final validation phase verifies that the cloud-based production deployment is functioning correctly.

This section confirms that:

- ✓ CI/CD automation executes successfully
- ✓ Docker images are built and deployed to Amazon ECR
- ✓ ECS services update properly
- ✓ CloudFront distributes updated frontend assets
- ✓ HTTPS is configured correctly
- ✓ Backend health checks pass
- ✓ Frontend-to-backend communication works in production

This phase represents the formal **Production Validation and Testing** stage of the deployment lifecycle.

3.c.1 Trigger Deployment via GitHub

To initiate a production deployment:

- ✓ Make a small test change to either the frontend or backend code.
- ✓ Commit the change locally.

- ✓ Push the commit to the working_branch.
  1. git add .
  2. git commit -m "Test production deployment validation"
  3. git push origin working_branch

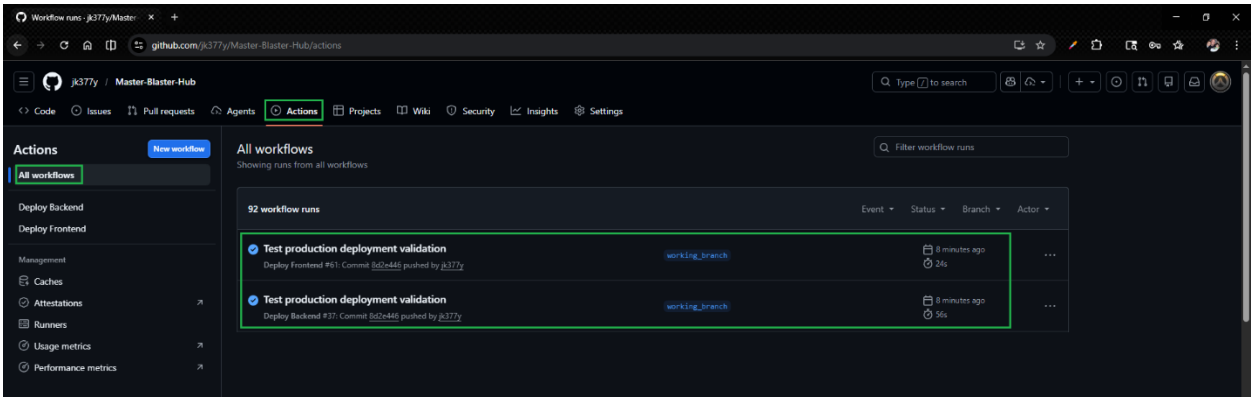
Pushing to working_branch triggers the GitHub Actions workflows defined in:

- ✓ .github/workflows/deploy-frontend.yml (Appendix B)
- ✓ .github/workflows/deploy-backend.yml (Appendix C)

3.c.2 Verify GitHub Actions Workflow Execution

1. Navigate to the GitHub repository.
2. Select the **Actions** tab.
3. Confirm that the appropriate workflow runs automatically.
   Verify:
   - ✓ Workflow status displays **Success**
   - ✓ All steps complete without error
   - ✓ No failed jobs appear in the execution log

   Successful execution confirms:
   - ✓ AWS authentication succeeded
   - ✓ Docker image built successfully
   - ✓ Image was pushed to ECR
   - ✓ ECS service update command executed
   - ✓ Frontend assets were deployed to S3
   - ✓ CloudFront invalidation was created



3.c.3 Verify Backend Deployment in AWS

After workflow completion, log in to the AWS Management Console.

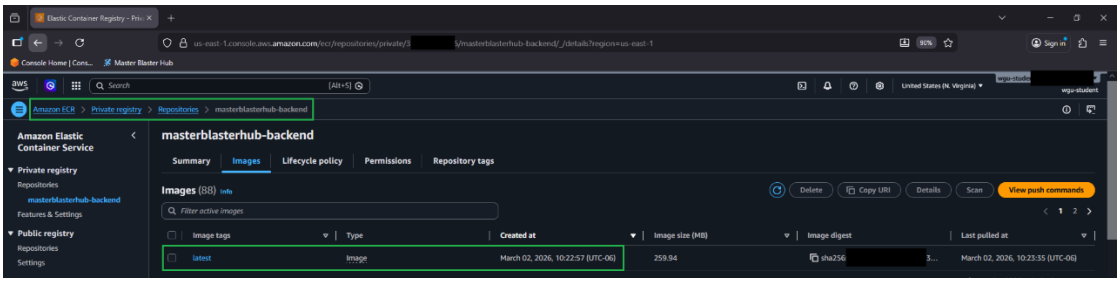
Confirm New Image in Amazon ECR

Navigate to: Amazon ECR > Repositories > **masterblasterhub-backend**

Confirm:

- A new image tag (typically latest) exists
- The push timestamp reflects the recent deployment

This confirms the Docker image was successfully published.

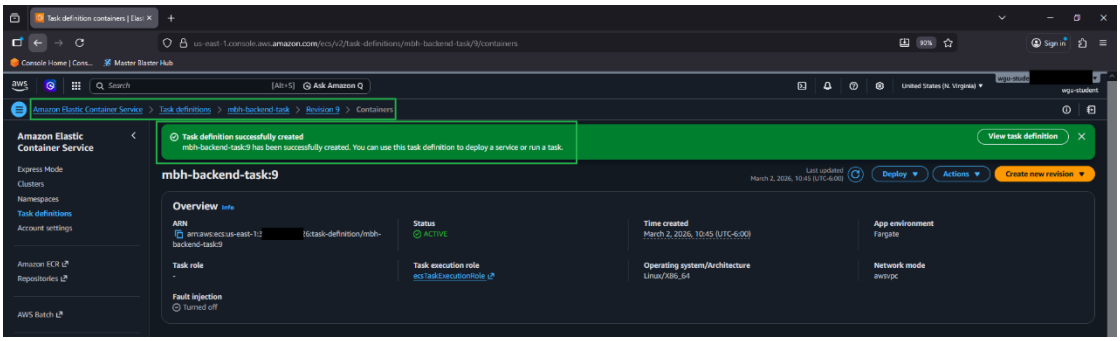


Perform ECS Task Revision

Navigate to: **Amazon ECS > Task definitions > mbh-backend-task**

Click and confirm: **Create New Revision**

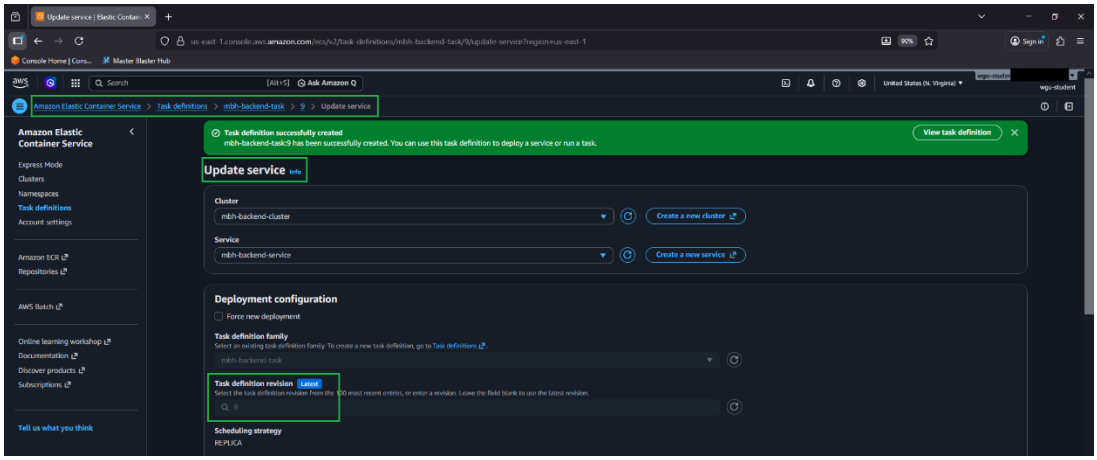
- ✓ No changes are needed.
- ✓ Scroll down to the bottom and click **Create**.



Perform Service Update

Click the **Deploy** dropdown

- ✓ Click: **Update Service**
- ✓ No changes are needed. If you look at Deployment configuration > Task definition revision, you will see the latest revision number.
- ✓ Scroll down to the bottom and click **Update**.



Confirm ECS Service Update

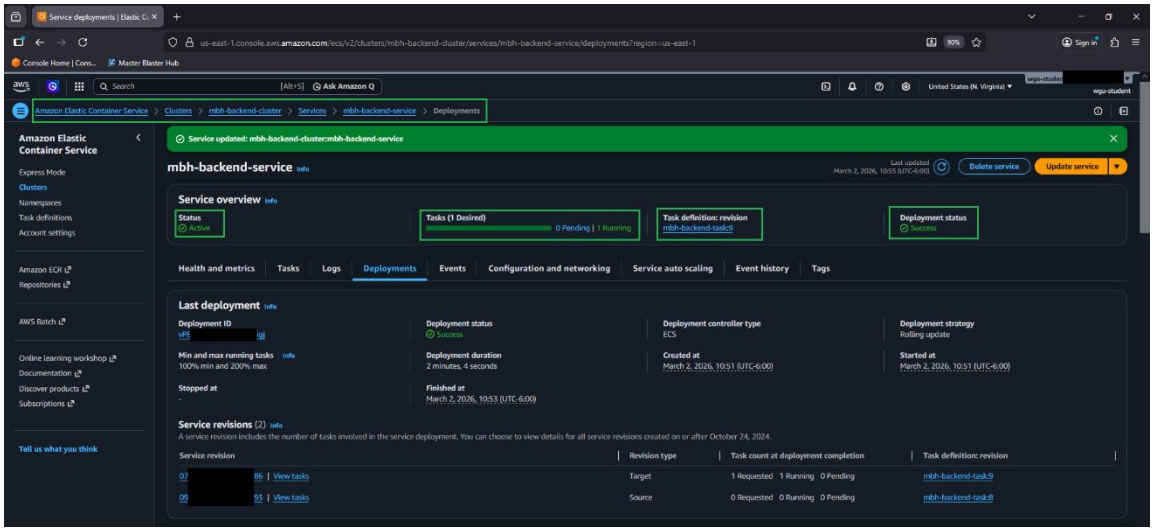
Navigate to: **Amazon ECS > Clusters > mbh-backend-cluster > Services > mbh-backend-service > Deployments**

Confirm:

- ✓ Service status: **Active**
- ✓ Deployment status: **Completed**
- ✓ Desired tasks: 1
- ✓ Running tasks: 1

Select the Deployments tab and confirm:

- ✓ A new deployment revision was created
- ✓ The previous task was replaced successfully
- ✓ No failed deployments appear



3.c.4 Verify Frontend Deployment

Navigate to: **Amazon S3 > Bucket > masterblasterhub.com**

Confirm:

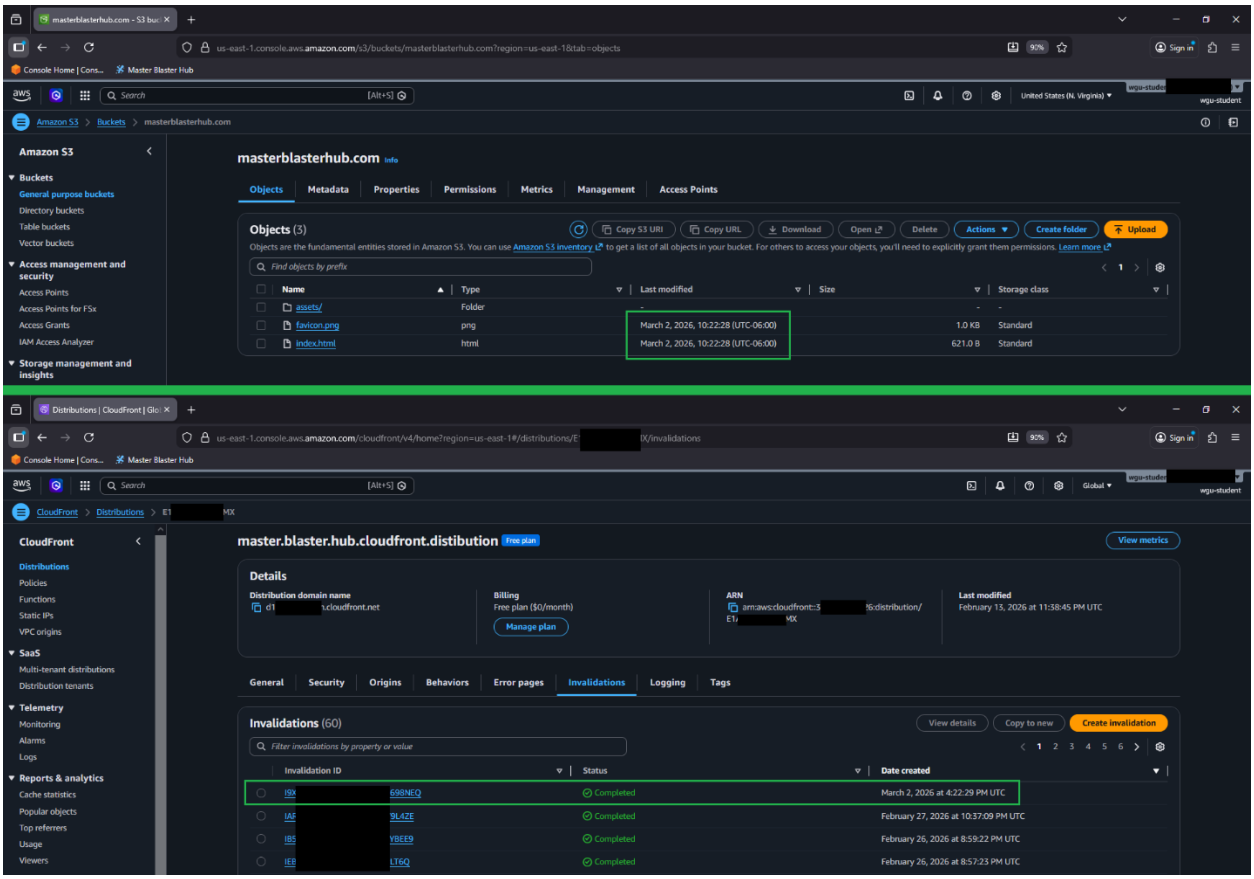
- ✓ Recently modified files reflect the latest deployment time

Then navigate to: **Amazon CloudFront > Distributions**

Confirm:

- ✓ Distribution status: **Enabled**
- ✓ Most recent invalidation completed successfully

CloudFront invalidation ensures new frontend assets are served immediately.



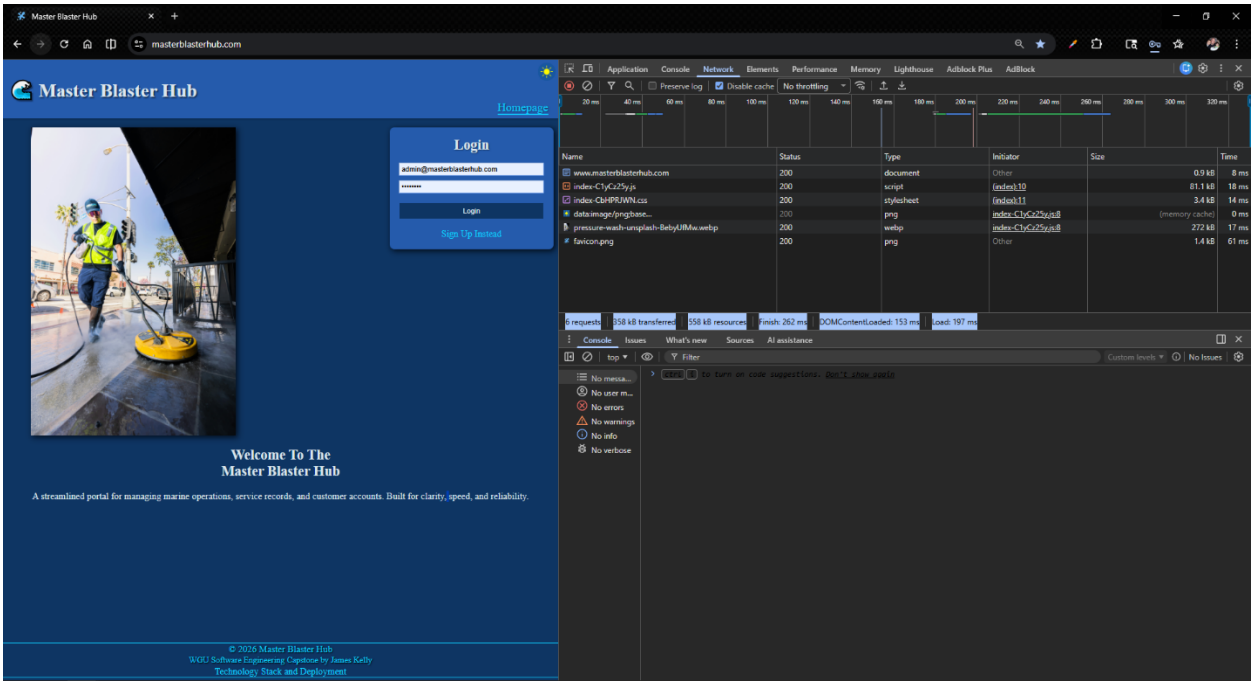
3.c.5 Validate Production URLs

Open a web browser and verify the following endpoints:

Frontend Application: https://www.masterblasterhub.com

Confirm:

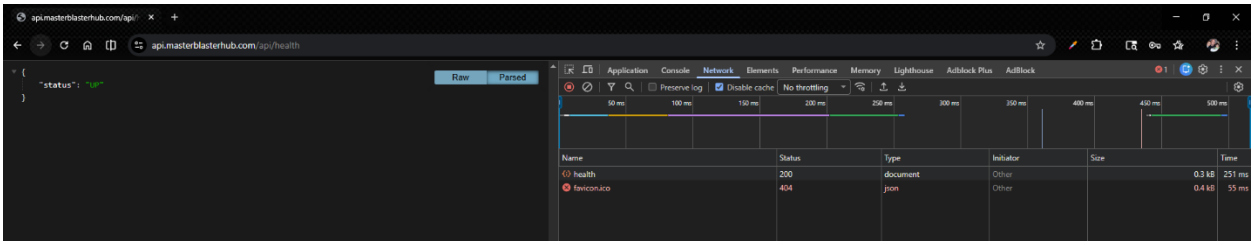
- ✓ Page loads successfully
- ✓ HTTPS certificate is valid (secure padlock visible)
- ✓ No mixed content warnings



Backend Health Endpoint: https://api.masterblasterhub.com/api/health

Expected response:

```
{
  "status": "UP"
}
```



This confirms:

- ✓ Domain resolution via Route 53
- ✓ TLS termination via ACM
- ✓ Load balancer routing
- ✓ ECS container execution

- ✓ Spring Boot application health

3.c.6 Validate Full Application Functionality

Perform a functional test in production:

- ✓ Log in using valid credentials
- ✓ Navigate to MyPortal or Admin page
- ✓ Perform an action that triggers API communication

Open browser developer tools > Network tab.

Confirm:

- ✓ Requests are sent to https://api.masterblasterhub.com/api/...
- ✓ Responses return expected HTTP status codes
- ✓ No CORS errors occur
- ✓ No 502 or 504 gateway errors occur

This confirms:

- ✓ Frontend and backend are communicating over HTTPS
- ✓ Cross-origin configuration is correct
- ✓ Database connectivity is functioning
- ✓ Production environment variables are loaded correctly

3.c.7 Expected Production Behavior

A successful production validation confirms:

- ✓ CI/CD automation functions as designed
- ✓ Docker images build and deploy automatically
- ✓ ECS tasks replace previous versions without downtime
- ✓ Load balancer health checks pass
- ✓ CloudFront distributes updated frontend assets
- ✓ HTTPS is enforced for all traffic
- ✓ The full application stack operates in the cloud

At this stage, the Master Blaster Hub application is fully operational in a secure, automated production environment. Congratulations!!!

APPENDIX A – Git Branch History

branchHistory_SWECapstone_JamesKelly.pdf

APPENDIX B – deploy-frontend.yml

File location: MBH/.github/workflows/deploy-frontend.yml

```yaml
name: Deploy Frontend

# Triggers only when frontend files change on working_branch
on:
  push:
    branches:
      - working_branch
    paths:
      - 'frontend/**'

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      # Pull repository code
      - name: Checkout repository
        uses: actions/checkout@v3

      # Install frontend dependencies
      - name: Install dependencies
        working-directory: frontend
        run: npm install

      # Build production bundle
      - name: Build frontend
        working-directory: frontend
        run: npm run build

      # Configure AWS credentials
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      # Upload build output to S3
      - name: Deploy to S3
        run: aws s3 sync frontend/dist s3://masterblasterhub.com --delete

      # Clear CloudFront cache so new files are served
      - name: Invalidate CloudFront cache
        run: |
          aws cloudfront create-invalidation \
            --distribution-id ${{ secrets.CLOUDFRONT_DISTRIBUTION_ID }} \
            --paths "/*"

```

APPENDIX C – deploy-backend.yml

File location: MBH/.github/workflows/deploy-backend.yml

```yaml
name: Deploy Backend

# Triggers deployment when working_branch is updated
on:
  push:
    branches:
      - working_branch
    paths:
      - 'backend/**'

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      # Pull repository code
      - name: Checkout repository
        uses: actions/checkout@v3

      # Configure AWS credentials using repository secrets
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.BACKEND_AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.BACKEND_AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      # Authenticate Docker to Amazon ECR
      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      # Build backend Docker image and tag for ECR
      - name: Build Docker image
        run: |
          docker build -t masterblasterhub-backend ./backend
          docker tag masterblasterhub-backend:latest \
            ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-
backend:latest

      # Push image to Amazon ECR repository
      - name: Push Docker image to ECR
        run: |
          docker push \
            ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.us-east-1.amazonaws.com/masterblasterhub-
backend:latest

      # Trigger ECS service to pull new image and redeploy
      - name: Force ECS service redeploy
        run: |
          aws ecs update-service \
            --cluster mbh-backend-cluster \
            --service mbh-backend-service \
            --force-new-deployment

```